

AMBA and AXI Connect Example

PWM Control

[*Reference*] ■

Table of Contents

axi_pwm_exam.xpr

➤ SoC Peripheral RTC Design Project (프로젝트 최종 계획서 제출)

1. Project Exam

1) Exam_Washing_Machine

2) Exam_Microblaze_Washing_Machine_Block_Design

2. led_btn_Exam

3. AXI_My_IP_LED_Exam

4. AXI_My_IP_FND_Exam

5. AXI_RTC_Exam

6. **AXI_PWM**

7. AXI_LCD

8. Servo_Motor_Exam

9. DHT11_Module_Exam

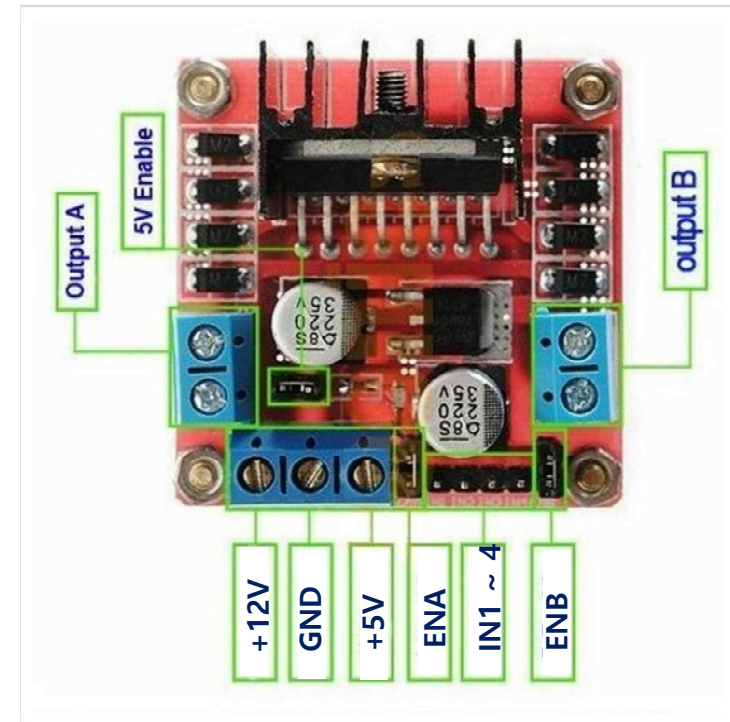
10. Ultrasonic Module Exam

➤ 모터 구동을 위한 모듈 : **L298N**(모터 드라이버)[1/2]

- **L298N 기능** : 1. DC, 스텝 모터의 방향 및 속도 제어
 - 2. H-브리지 구조: 모터를 양방향으로 제어가능. 즉, 모터의 회전방향을 제어하거나 정지
 - 3. 두개의 모터를 독립적으로 제어. 두개의 출력 핀 제공(Output A, Output B)
 - 4. 주로 PWM신호를 사용하여 모터 속도를 조절

▪ **L298N** 핀 구성 :

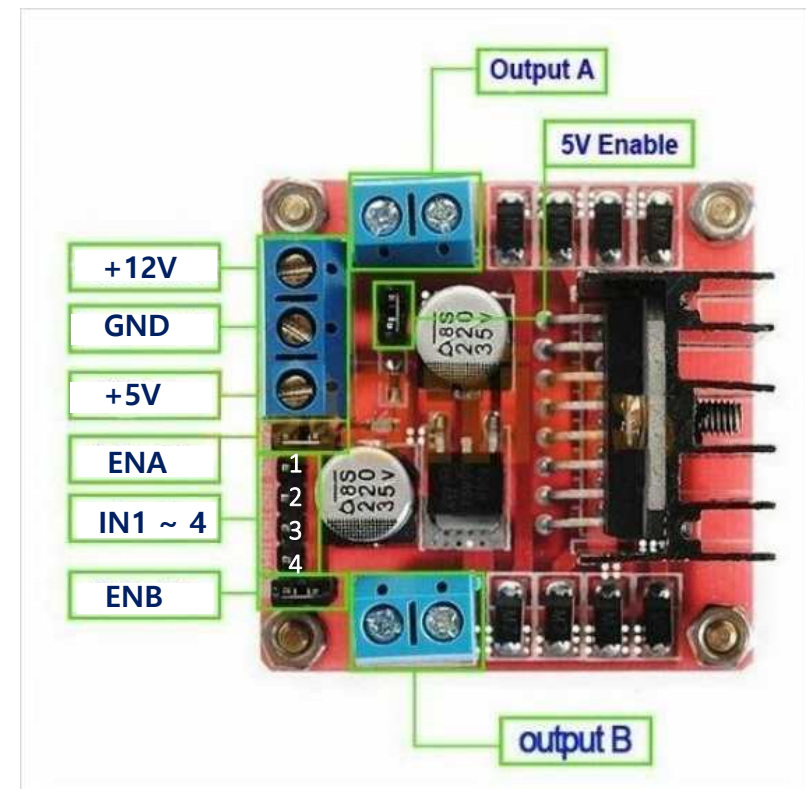
1. **IN1, IN2** : 첫 번째 모터의 회전 방향 제어
2. **IN3, IN4** : 두 번째 모터의 회전 방향 제어
3. **OUTA** : 첫 번째 모터의 출력 핀(모터와 연결)
4. **OUTB** : 두 번째 모터의 출력 핀(모터와 연결)
5. **ENA** : 첫 번째 모터의 속도 제어를 위한 PWM 입력 핀
6. **ENB** : 두 번째 모터의 속도 제어를 위한 PWM 입력 핀
7. **+12V** : 모터 전원 입력(+5V ~ 35V)
8. **+5V** : 내부 논리 회로 전원 입력 (+5V)
9. **GND** : 공통 접지



➤ 모터 구동을 위한 모듈 : **L298N**(모터 드라이버)[2/2]

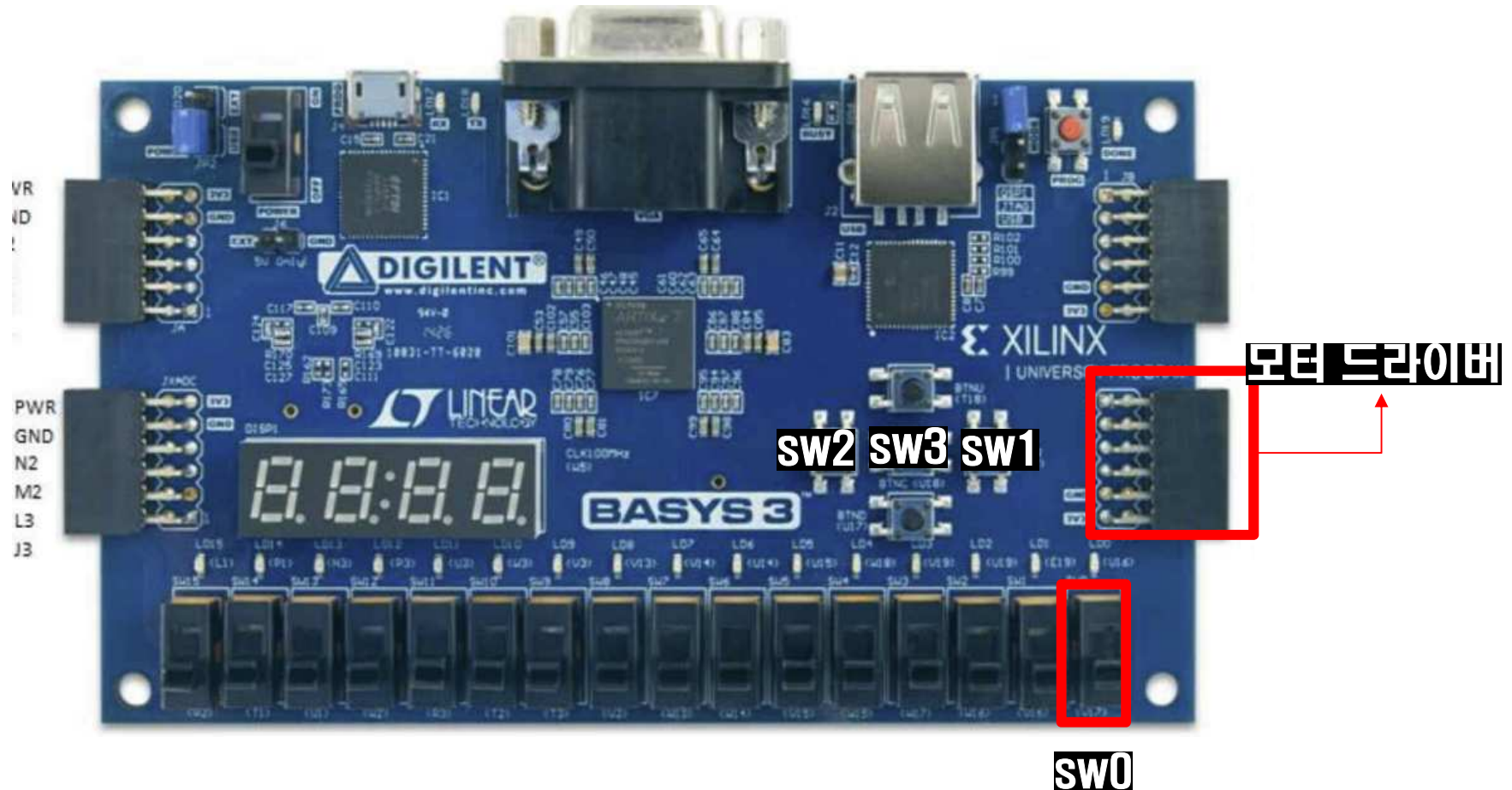
▪ 모터 구동을 위한 결선

1. **IN1, IN2**: **DC MOTER** 결선(**DC MOTOR**방향제어)
2. **OUTA**: **DC MOTER** 결선 (**MOTOR** 출력 핀)
3. **ENA** : **DC MOTER** 결선(속도제어)
4. **+12V** : 외부전원 +5V인가
5. **+5V** : 결선X
6. **GND** : 외부전원과 **BASYS3**을 공통 접지



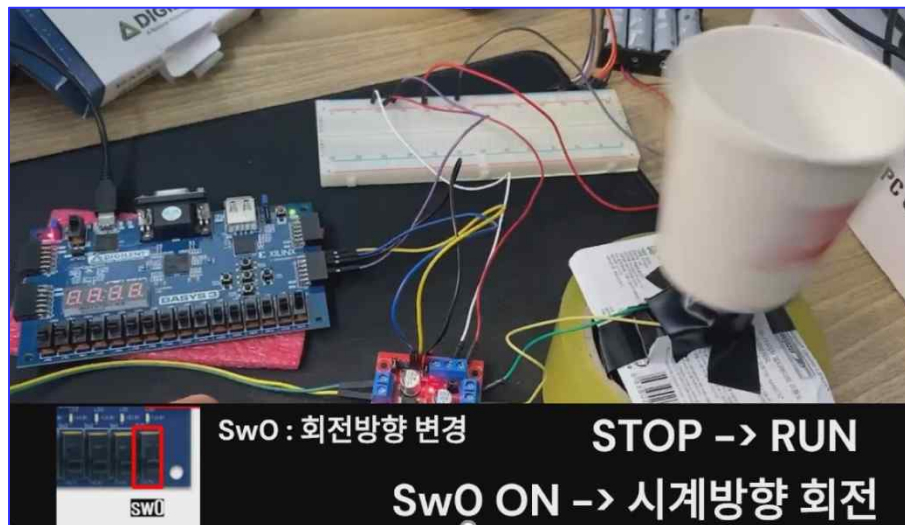
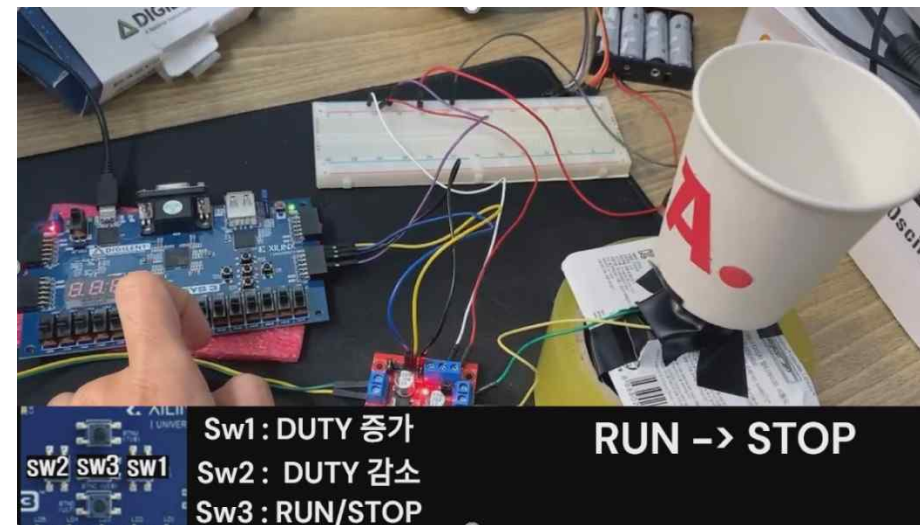
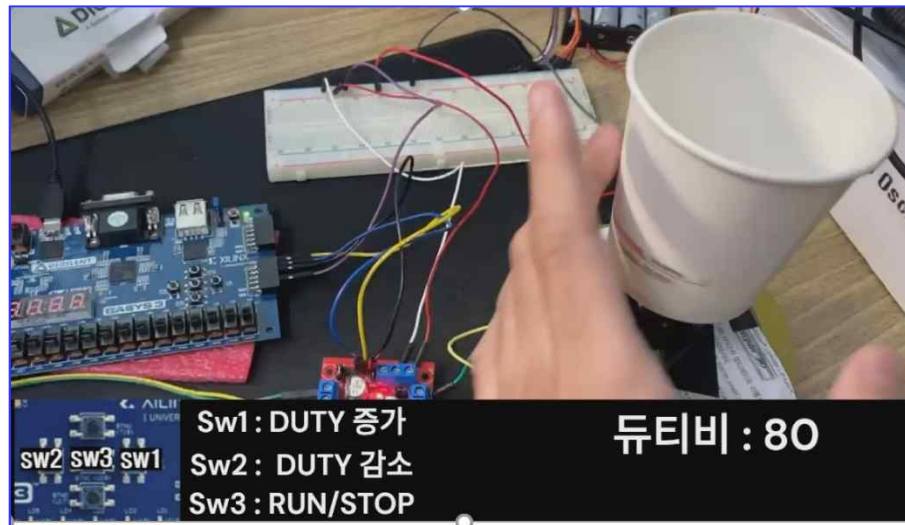
➤ Design Goal

- ***GPIO를 활용한 PWM Control 실습***

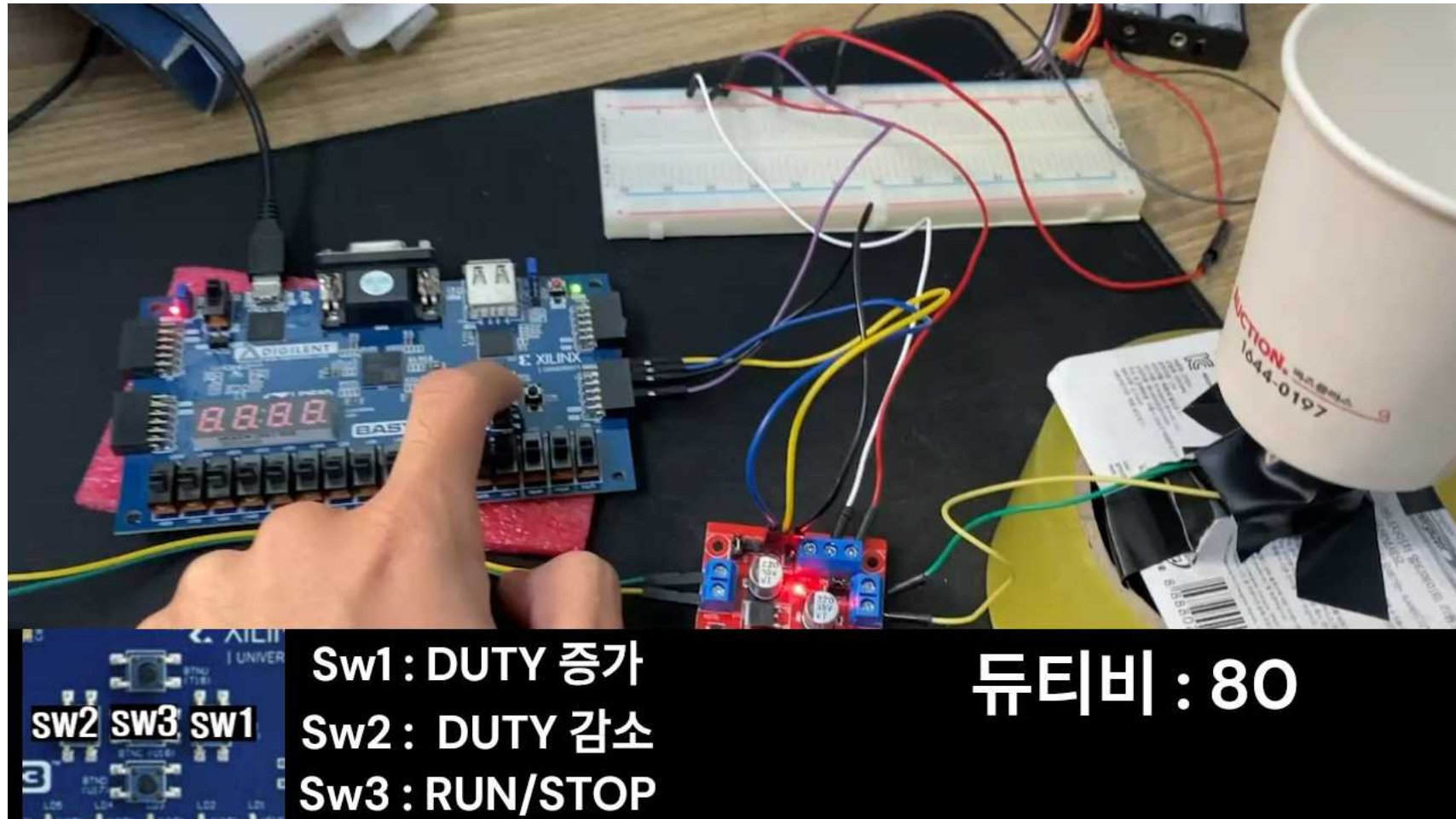


- ***SW0*** : 기본상태에서 0b01, 토글일때 0b10
- ***SW1*** : PWM 듀티비 증가 ***SW2*** : PWM 듀티비 감소 ***SW3*** : RUN/STOP

➤ AXI Connect ➔ PWM Control ➔ Program Device and Implementation ➔ **Check The Result : Image**



- AXI Connect ➔ PWM Control ➔ Program Device and Implementation ➔ **Check The Result : Video**

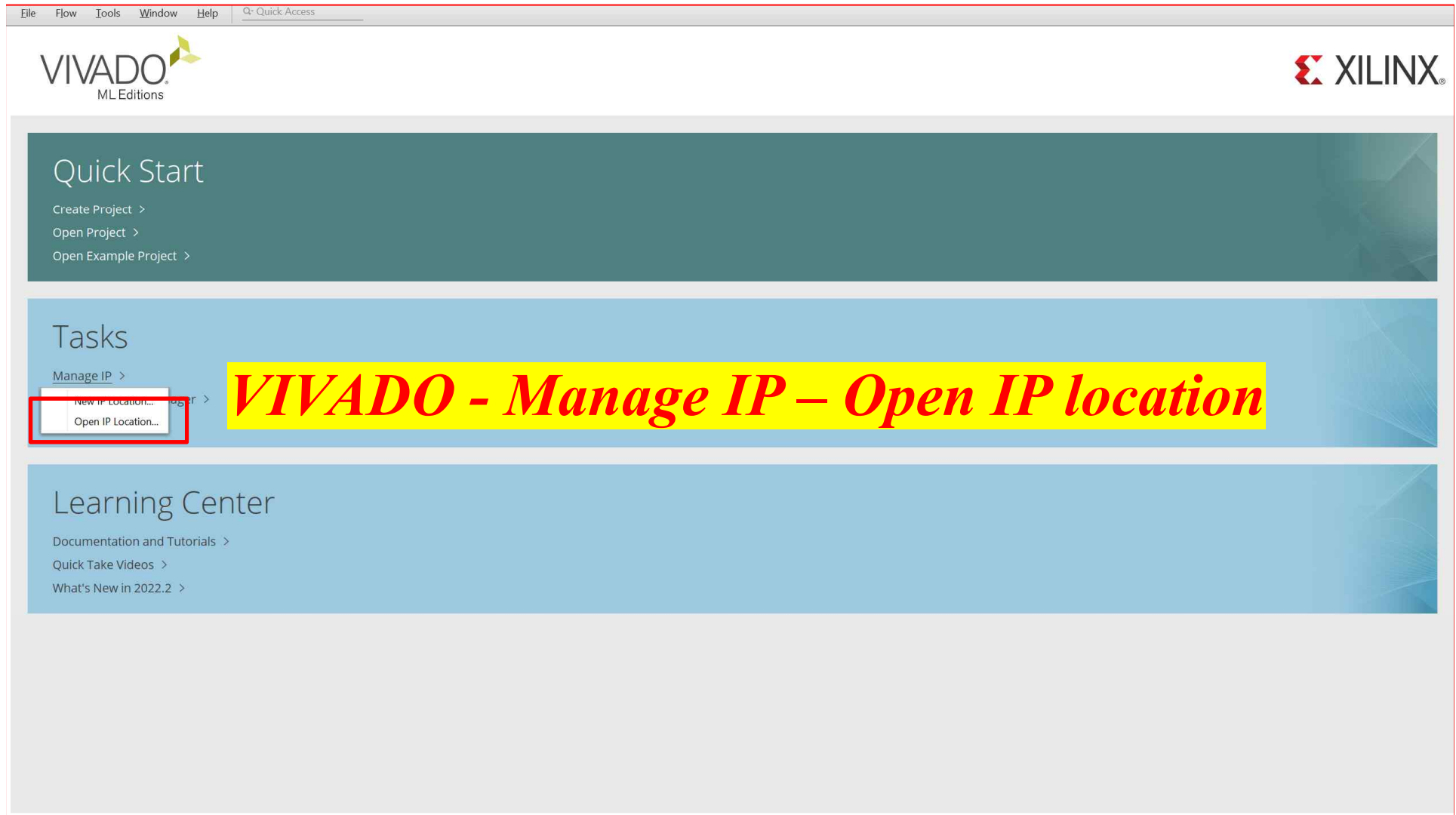


AMBA and AXI Connect Example

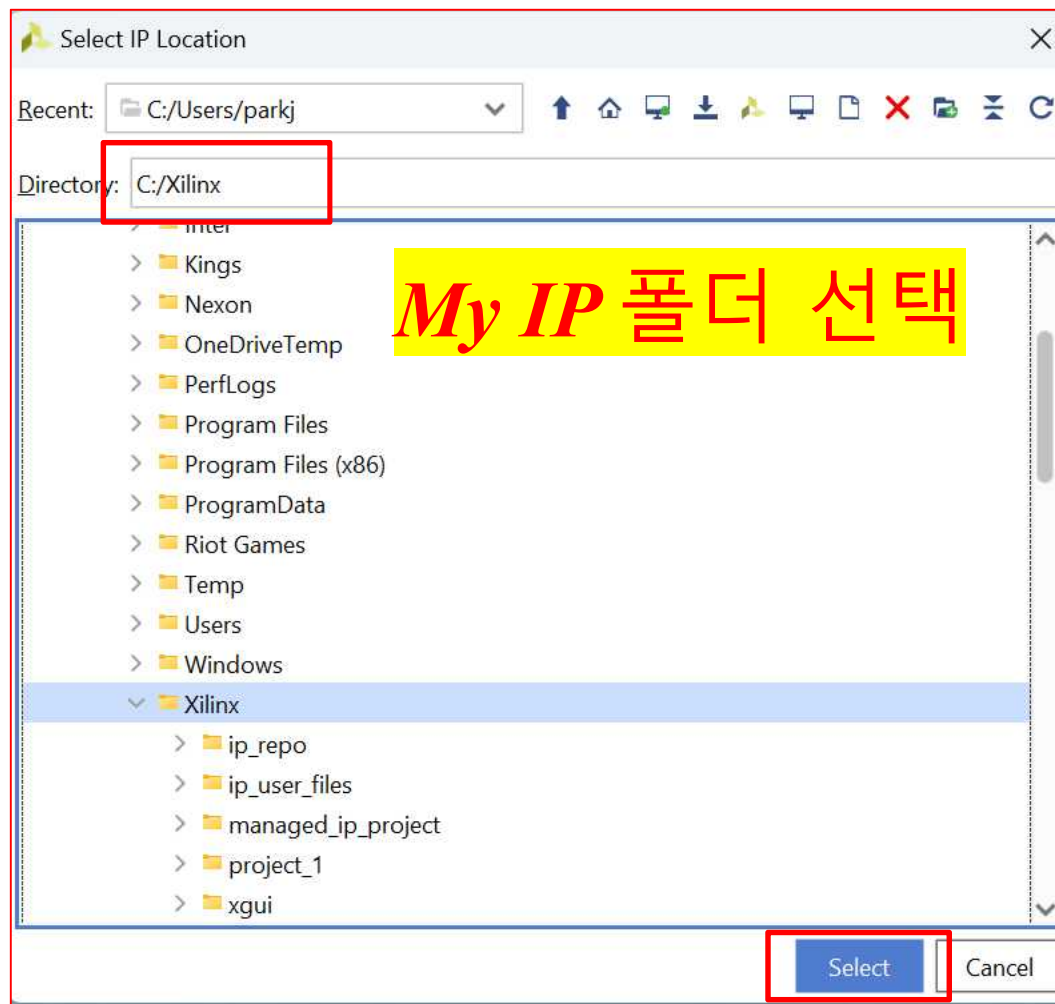
PWM Control

[*Create IP*]

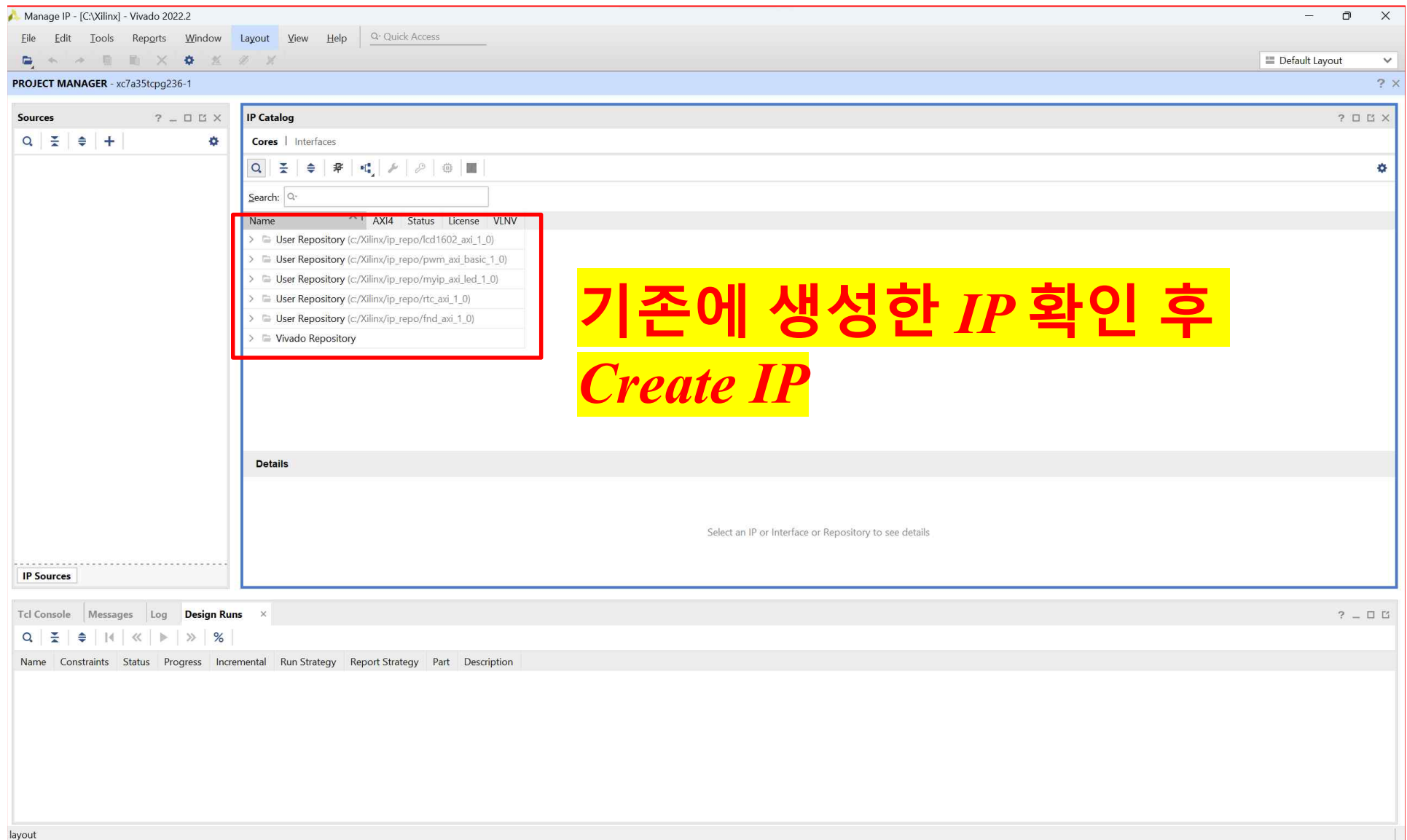
➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



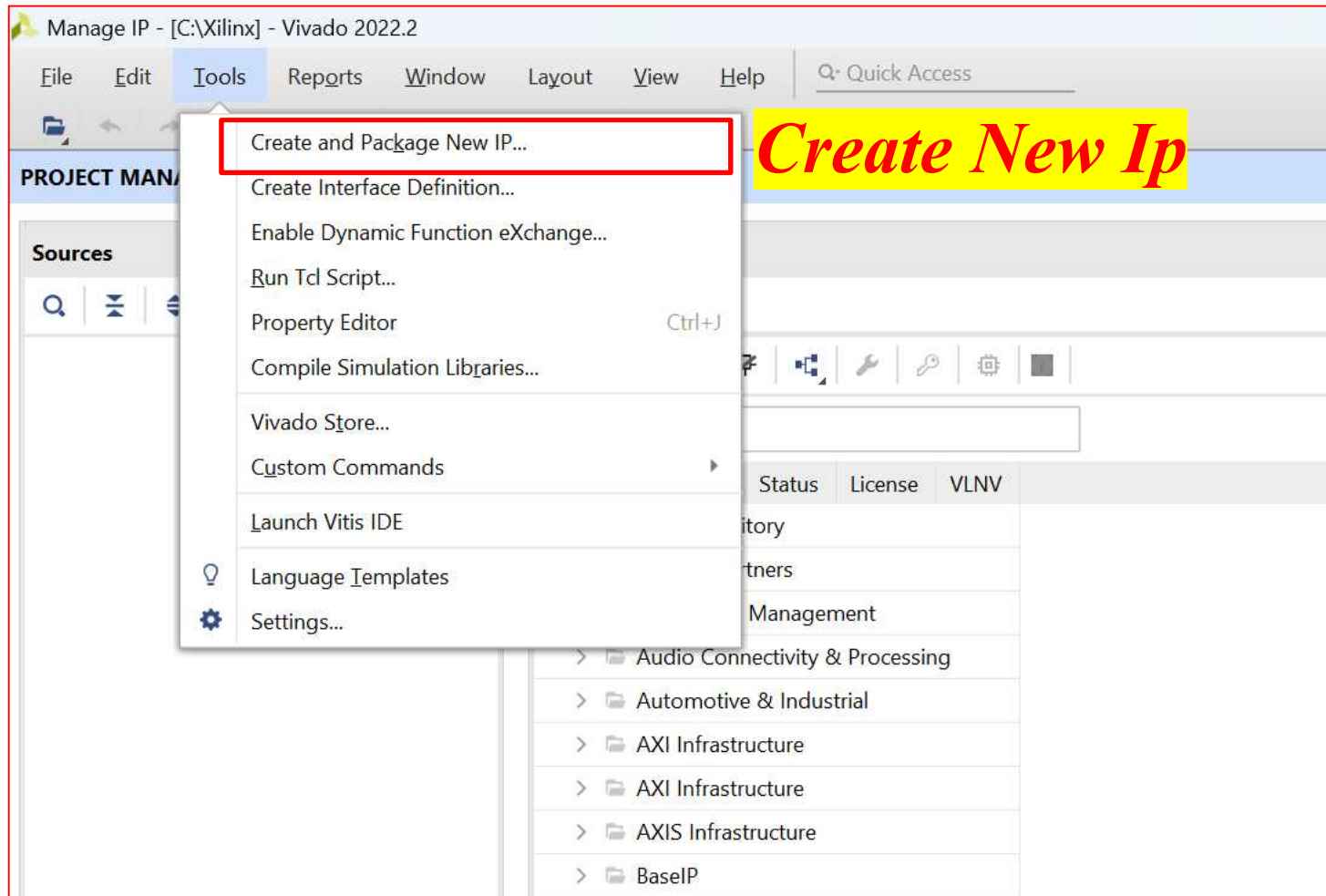
The screenshot shows the Vivado 2022.2 Manage IP window. The IP Catalog is open, displaying a list of user repositories. A red box highlights the following repositories:

- User Repository (c:/Xilinx/ip_repo/lcd1602_axi_1_0)
- User Repository (c:/Xilinx/ip_repo/pwm_axi_basic_1_0)
- User Repository (c:/Xilinx/ip_repo/myip_axi_led_1_0)
- User Repository (c:/Xilinx/ip_repo/rtc_axi_1_0)
- User Repository (c:/Xilinx/ip_repo/fnd_axi_1_0)
- Vivado Repository

A yellow text box with the following Korean text is overlaid on the right side of the IP Catalog:

기존에 생성한 *IP* 확인 후
Create IP

➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control

Create and Package New IP

Create Peripheral, Package IP or Package a Block Design
Please select one of the following tasks.

Packaging Options

- ☐ Package your current project
Use the project as the source for creating a new IP Definition.
- ☐ Package a block design from the current project
Choose a block design as the source for creating a new IP Definition.
Select a block design: axi_connect_1
- ☐ Package a specified directory
Choose a directory as the source for creating a new IP Definition.

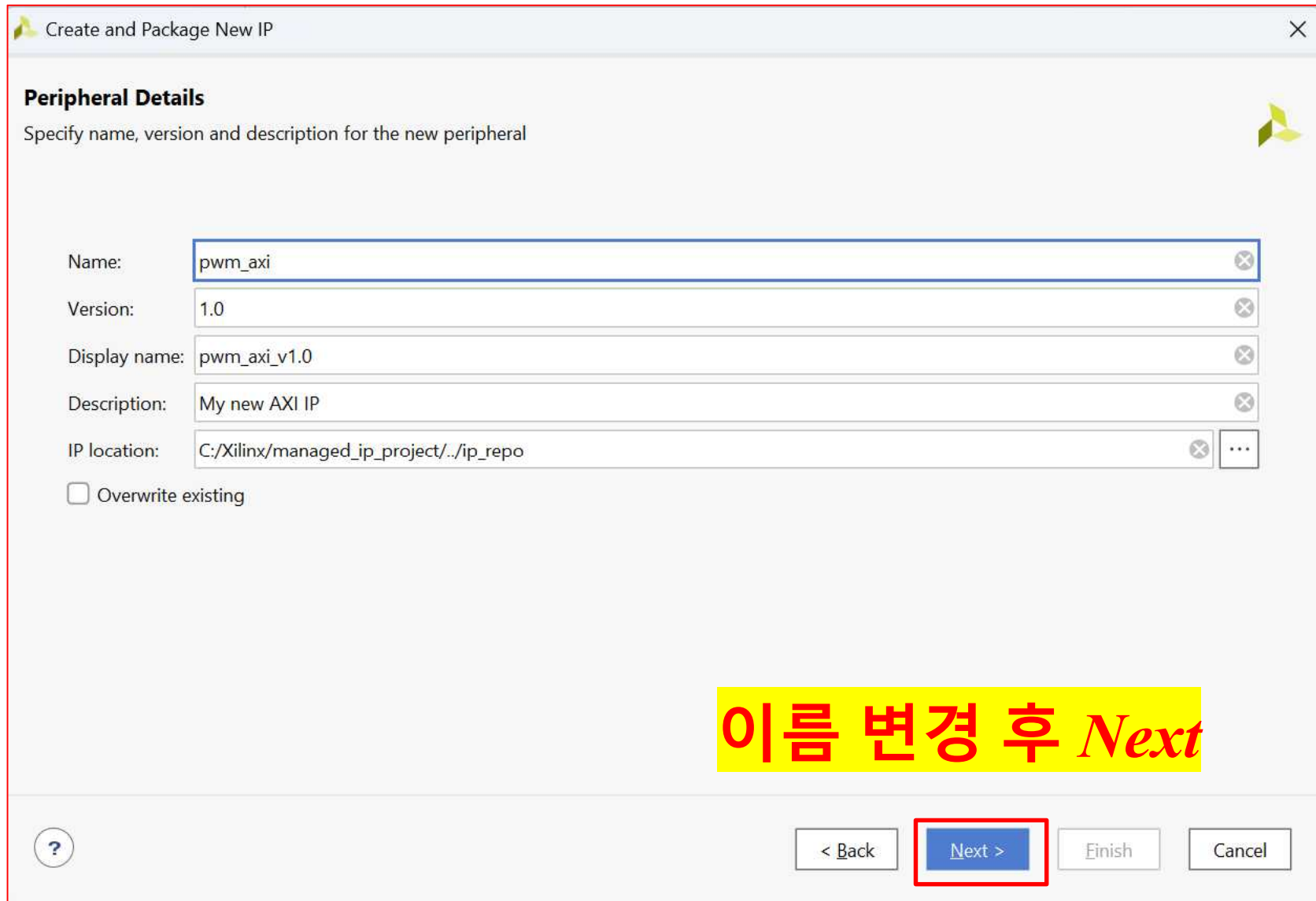
Create AXI4 Peripheral

- ☒ Create a new AXI4 peripheral
Create an AXI4 IP, driver, software test application, IP Integrator AXI4 VIP simulation and debug demonstration design.

Create New Ip

< Back Next > Finish Cancel

➤ AXI Connect ➔ PWM Control



Create and Package New IP

Peripheral Details
Specify name, version and description for the new peripheral

Name: pwm_axi

Version: 1.0

Display name: pwm_axi_v1.0

Description: My new AXI IP

IP location: C:/Xilinx/managed_ip_project/./ip_repo

☐ Overwrite existing

이름 변경 후 *Next*

< Back Next > Finish Cancel

➤ AXI Connect ➔ PWM Control

Create and Package New IP

Add Interfaces
Add AXI4 interfaces supported by your peripheral

☐ Enable Interrupt Support

Interfaces

- S00_pwm_AXI

S00_pwm_AXI
pwm_axi_v1.0

Name: S00_pwm_AXI

Interface Type: Lite

Interface Mode: Slave

Data Width (Bits): 32

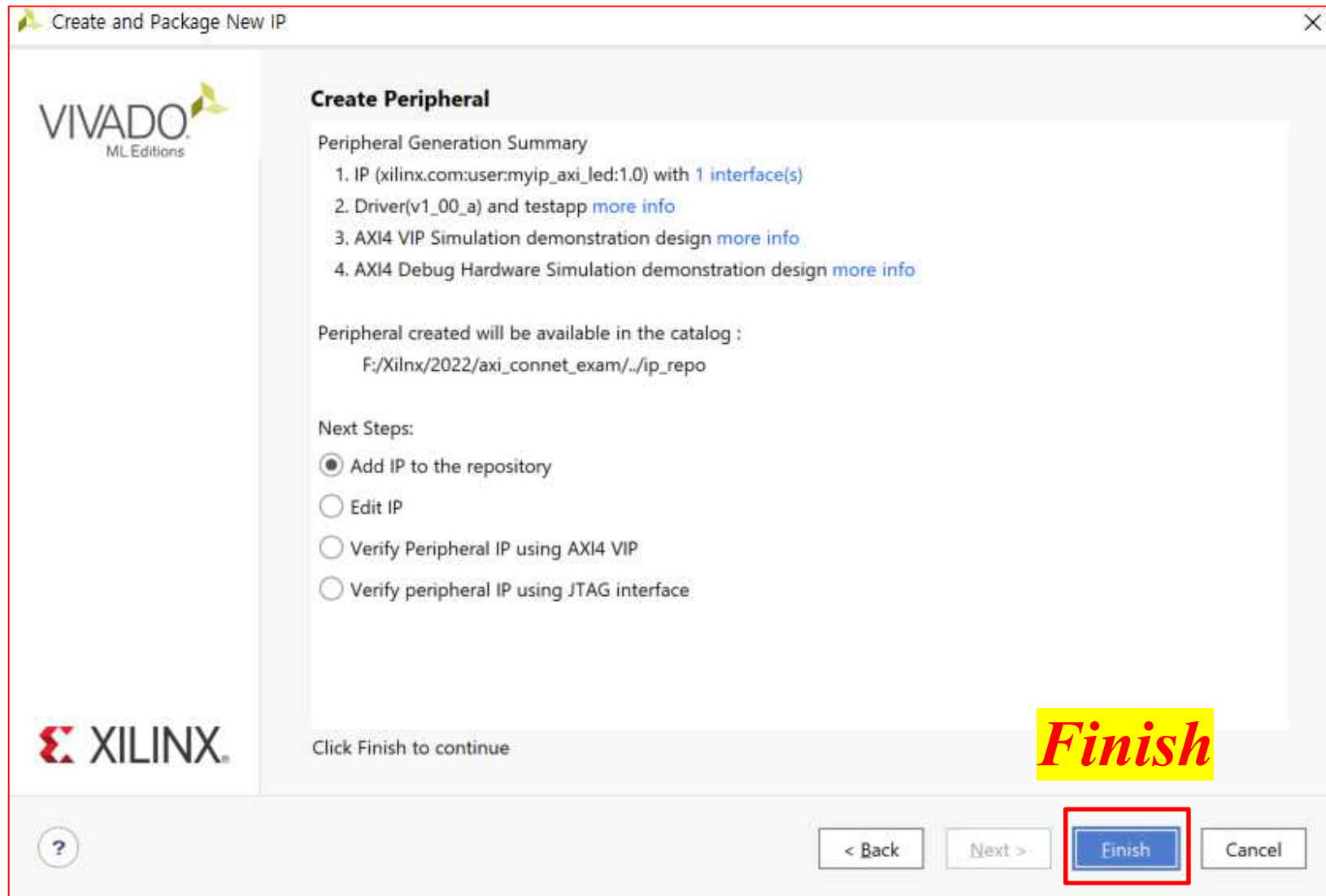
Memory Size (Bytes): 64

Number of Registers: 4 [4..512]

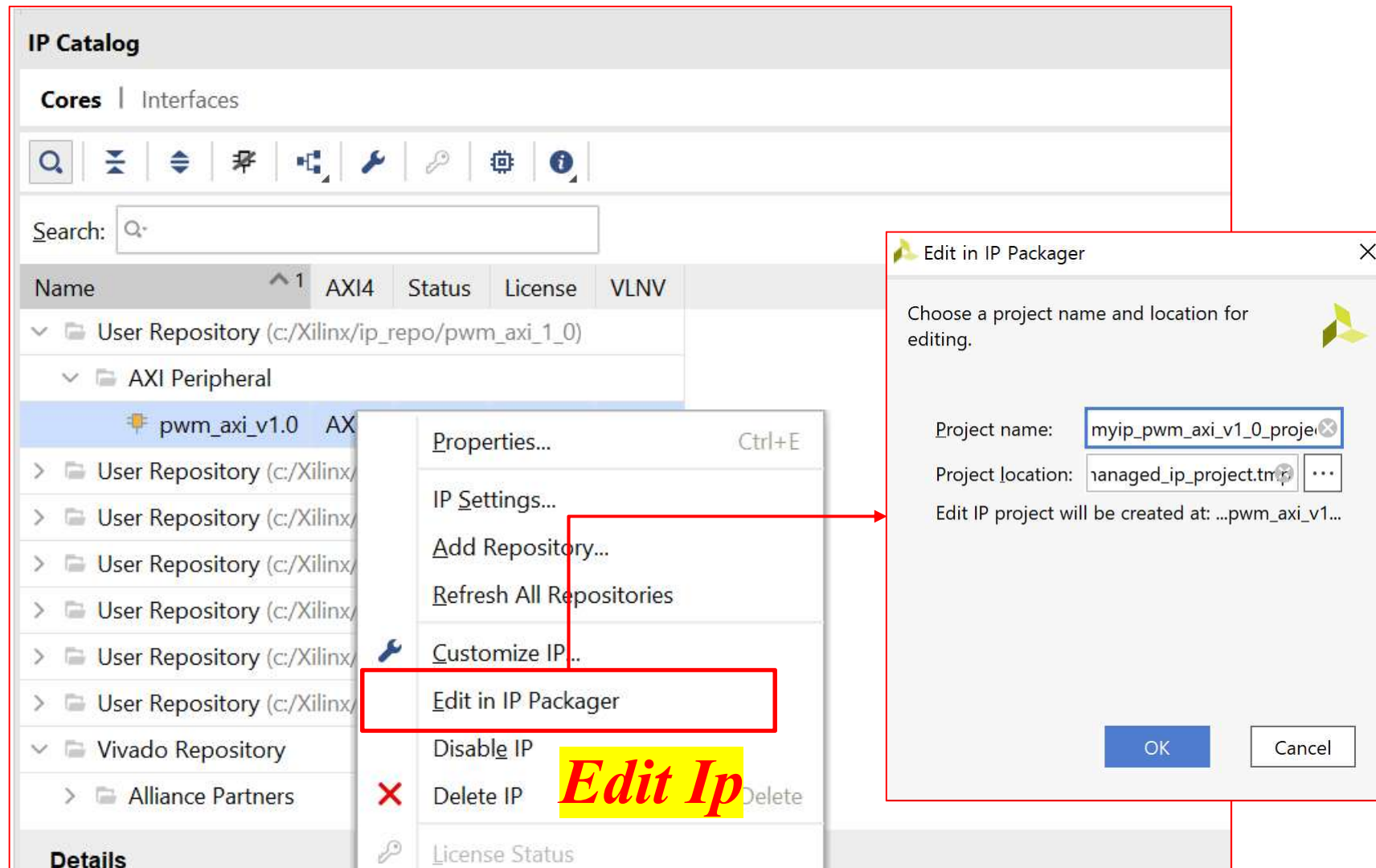
이름 변경 후 Next

< Back Next > Finish Cancel

➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



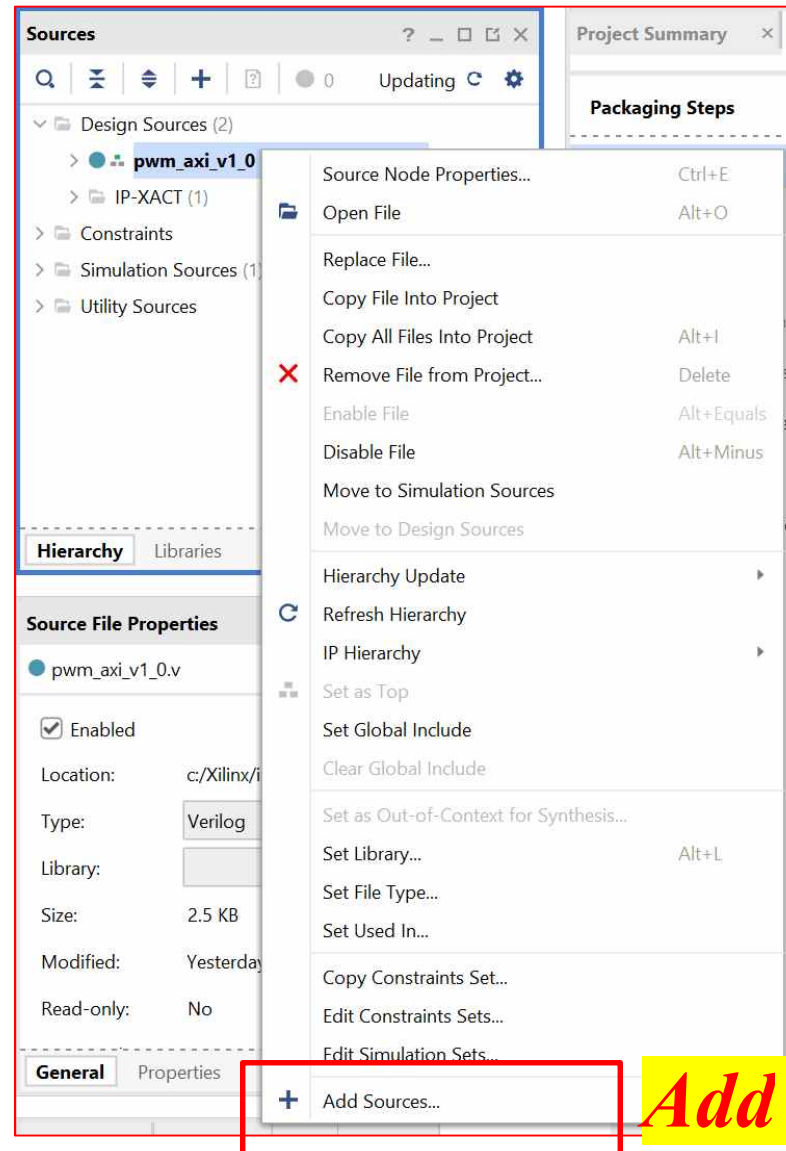
➤ AXI Connect ➔ PWM Control

The screenshot displays the Xilinx Vivado IDE interface. On the left, the 'Sources' window shows a project hierarchy with 'pwm_axi_v1_0 (pwm_axi_v1_0.v) (1)' highlighted under 'Design Sources (2)'. A yellow box with red text '저장된 IP 모듈 확인' (Check saved IP module) is overlaid on the 'Sources' window. The 'Project Summary' window is open, showing the 'Package IP - pwm_axi' tab. The 'Identification' tab is selected, displaying the following details:

Identification	
Vendor:	xilinx.com
Library:	user
Name:	pwm_axi
Version:	1.0
Display name:	pwm_axi_v1.0
Description:	My new AXI IP
Vendor display name:	
Company url:	
Root directory:	c:/Xilinx/ip_repo/pwm_axi_1_0
Xml file name:	c:/Xilinx/ip_repo/pwm_axi_1_0/component.xml

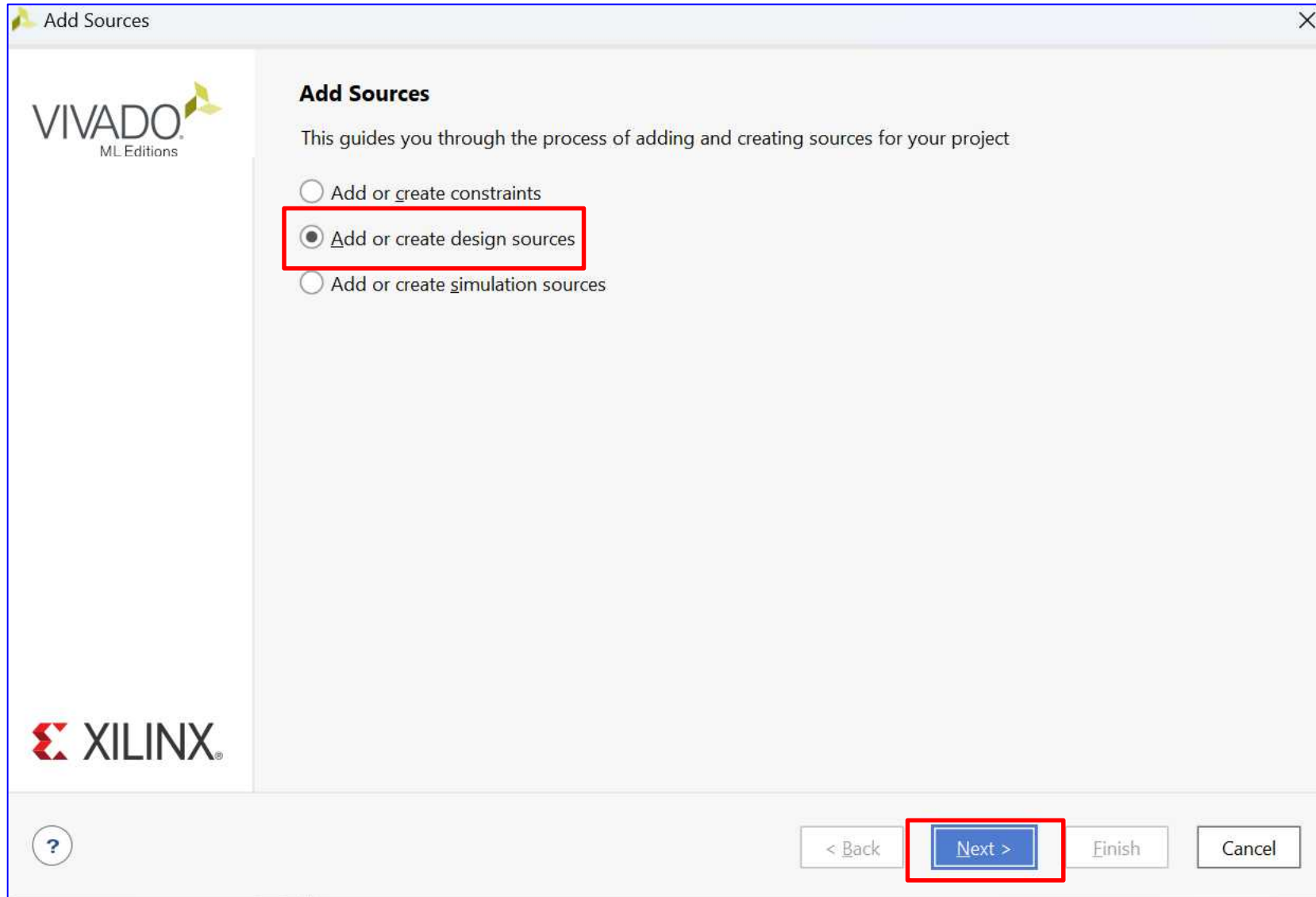
Below the 'Identification' tab, the 'Categories' section shows a list of categories with 'AXI_Peripheral' selected.

➤ AXI Connect ➔ PWM Control

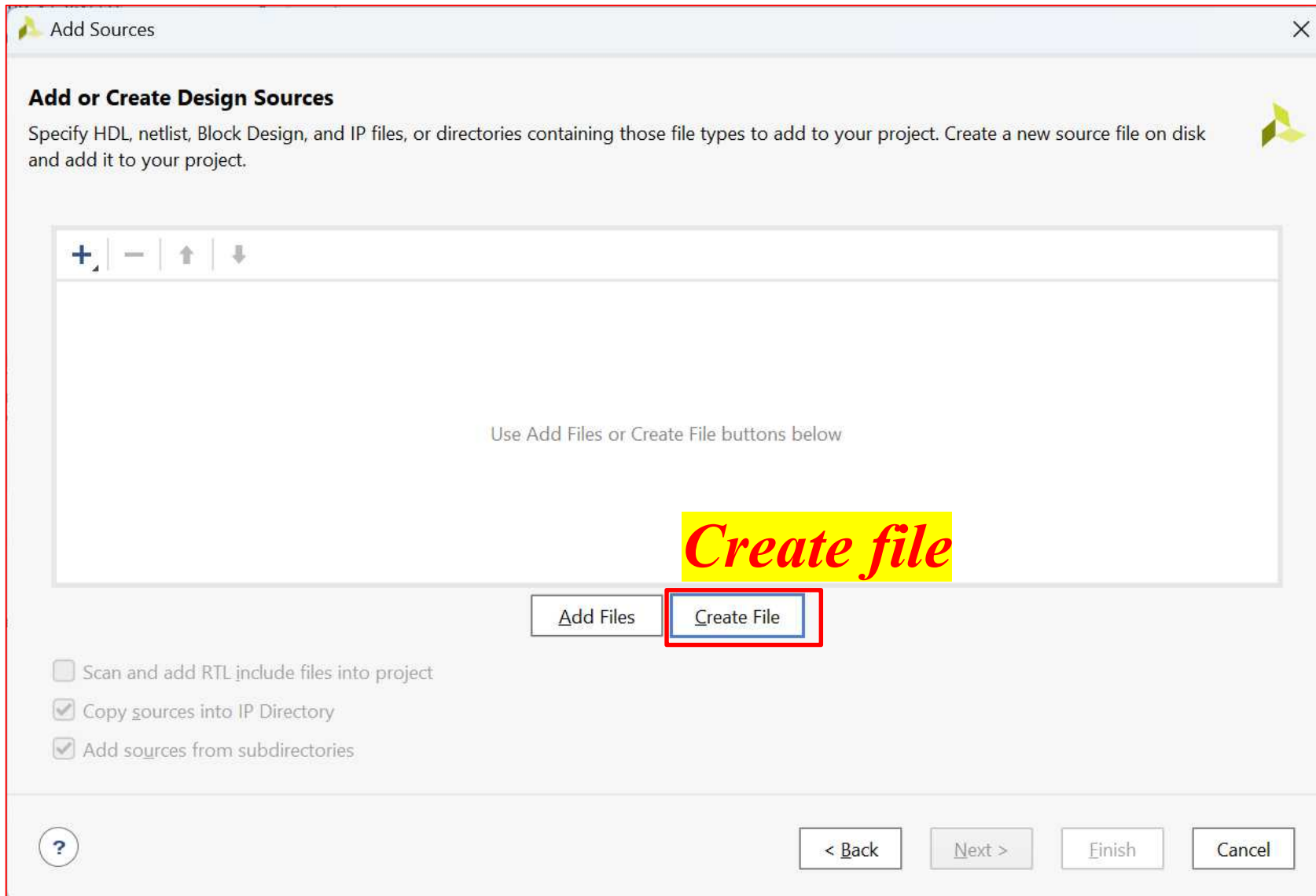


Add Sources

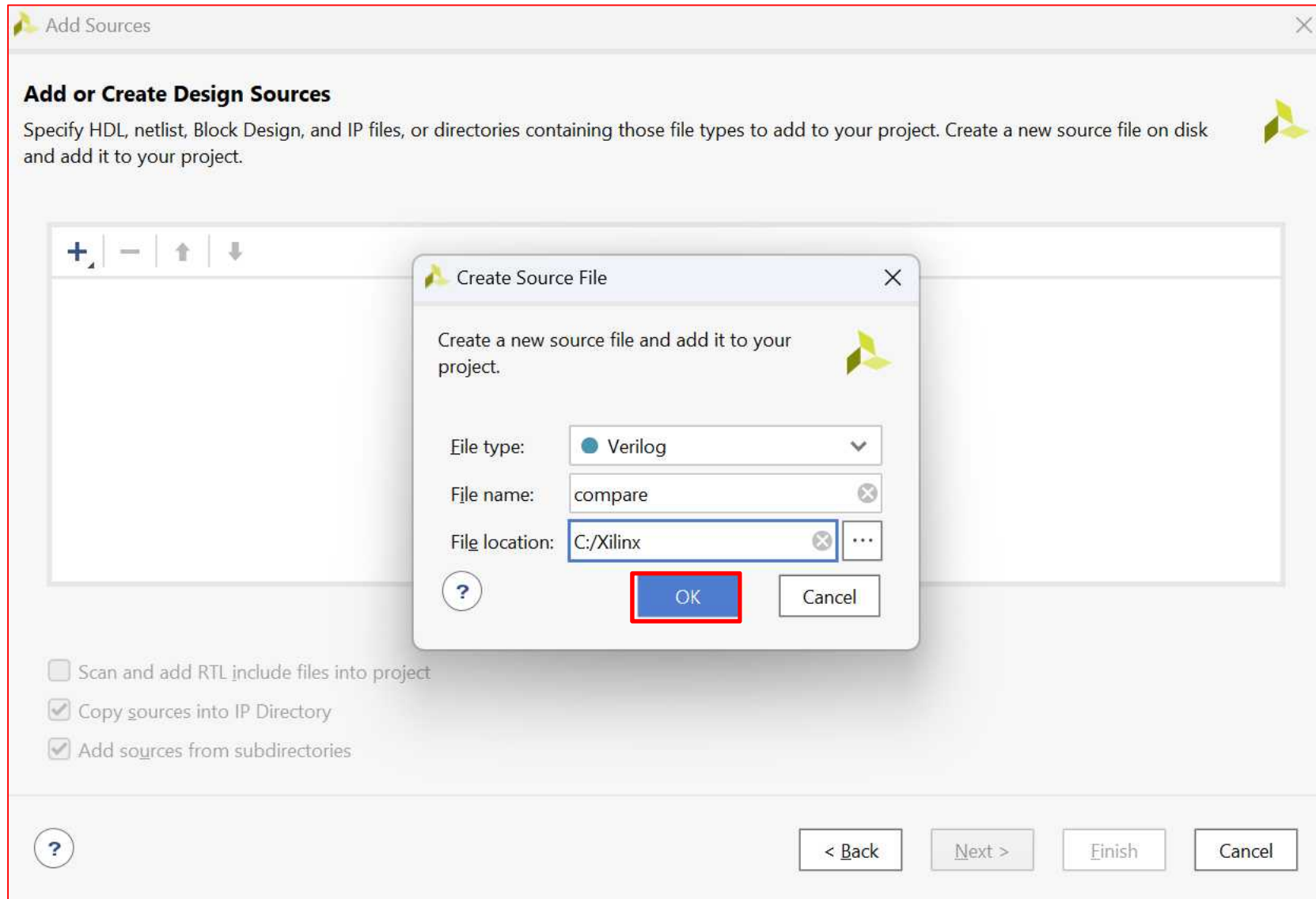
➤ AXI Connect ➔ PWM Control



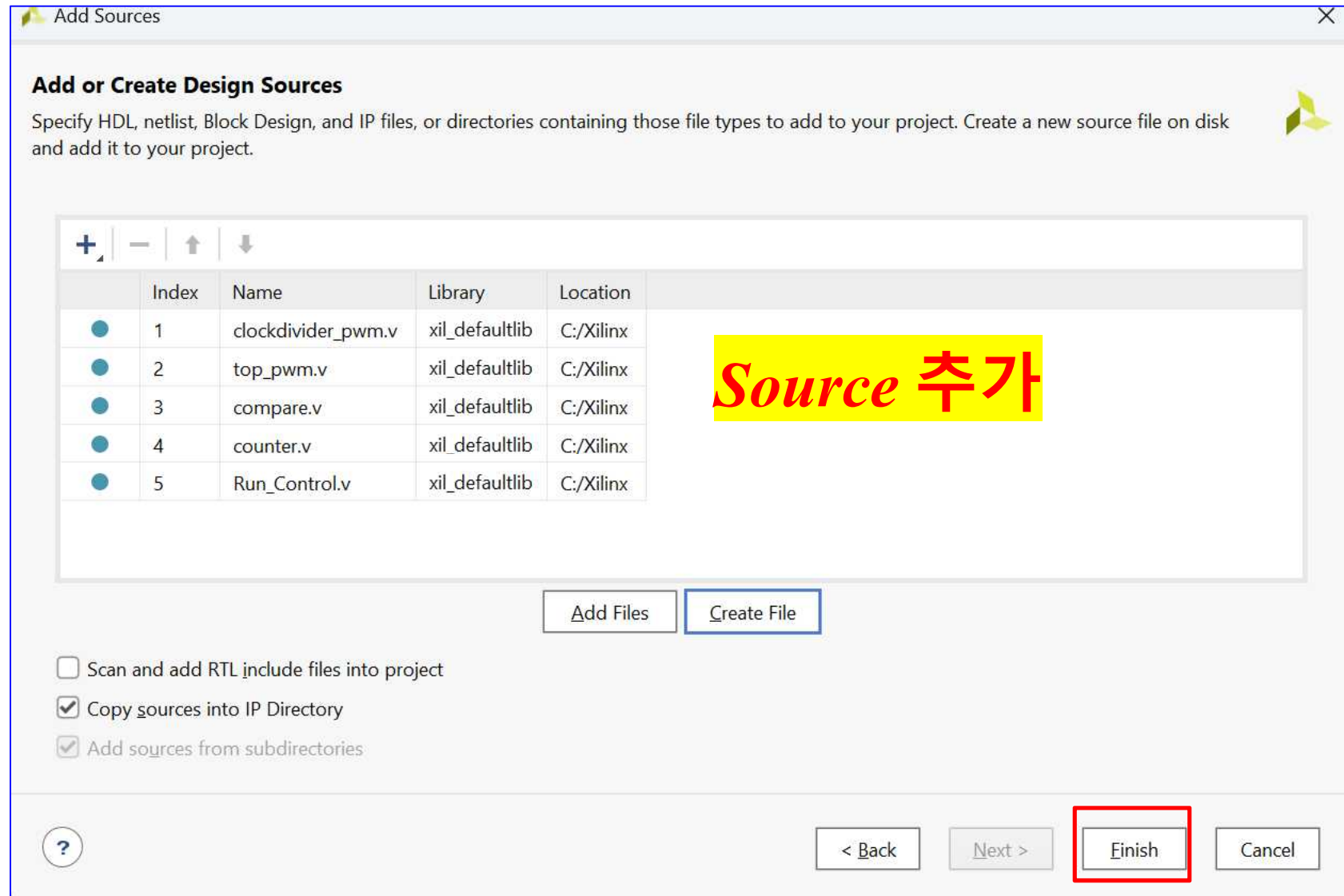
➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control

➔ **top_pwm**

axi_pwm_exam.xpr

```
1. `timescale 1ns / 1ps
2. module top_pwm(
3.   reset, clk, i_ocr, i_en, i_dir, o_dir,
     o_motor_pwm, w_counter, w_clkout
4. );
5.   input reset;
6.   input clk;
7.   input [7:0] i_ocr;
8.   input i_en;
9.   input [1:0] i_dir;
10.  output [1:0] o_dir;
11.  output o_motor_pwm;
12.  output [7:0] w_counter;
13.  output w_clkout;
14.  clockdivider_pwm U0 (
15.    .inclk(clk),
16.    .reset(reset),
17.    .clk_out(w_clkout)
18. );
```

```
19. counter U1 (
20.   .inclk(w_clkout),
21.   .reset(reset),
22.   .out_counter(w_counter)
23. );
24.   wire [7:0] w_counter;
25.   wire [7:0] i_ocr;
26.   wire [1:0] i_dir;
27.   wire [1:0] o_dir;
28.  compare U2 (
29.    .count(w_counter),
30.    .i_ocr(i_ocr),
31.    .i_en(i_en),
32.    .out_pwm(o_motor_pwm)
33. );
34.  Run_Control U3(
35.    .i_direction(i_dir),
36.    .o_direction(o_dir),
37.    .i_en(i_en)
38. );
39. endmodule
```

➤ AXI Connect ➔ PWM Control

➔ **clockdivider_pwm**

```
1. `timescale 1ns / 1ps
2. module clockdivider_pwm (
3.     input inclk,
4.     input reset,
5.     output reg clk_out
6. );
7.     reg [25:0] cnt = 0;
8.     always @(posedge inclk, negedge reset) begin
9.         if (~reset) begin
10.             cnt <= 0;
11.             clk_out = 0;
12.         end else begin
13.             if (cnt == (49)) begin
14.                 cnt <= 0;
15.                 case(clk_out)
16.                     1'b0 : clk_out <= 1;
17.                     1'b1 : clk_out <= 0;
18.                 endcase
```

```
19.     end else begin
20.         cnt <= cnt + 1;
21.     end
22. end
23. end
24. endmodule
```


AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control

➔ counter

```
1. `timescale 1ns / 1ps
2. module counter (
3.     input inclk,
4.     input reset,
5.     output [7:0] out_counter
6. );
7. reg [7:0] count=0;
8. assign out_counter = count;
9. always @(posedge inclk, negedge reset) begin
10.     if (~reset) begin
11.         count <= 0;
12.     end else begin
13.
14.         if(count >= 99) begin
15.             count <= 0;
16.         end else begin
17.             count = count + 1;
18.         end
19.     end
20. end
21. endmodule
```

axi_pwm_exam.xpr

➤ AXI Connect ➔ PWM Control

➔ compare

```
1. `timescale 1ns / 1ps

2. module compare(
3.     count, i_ocr, out_pwm, i_en
4. );
5.
6.     input [7:0] count;
7.     input [7:0] i_ocr;
8.     input i_en;
9.     output out_pwm;

10. reg pwm = 0;

11. assign out_pwm = pwm;

12. always @(*) begin

13.     if(i_en) begin // i_en == 1

14.         if(count < i_ocr) begin
15.             pwm = 1'b1;
16.         end else begin
17.             pwm = 1'b0;
18.         end

19.     end else begin // i_en == 0
20.         pwm = 1'b0;
21.     end
22. end

23. endmodule
```

➤ AXI Connect ➔ PWM Control

➔ **Run_Control**

```
1. `timescale 1ns / 1ps
2. module Run_Control(
3.     input [1:0] i_direction,
4.     input i_en,
5.     output [1:0] o_direction
6. );
7.
8.     reg [1:0] r_dir;
9.     assign o_direction = r_dir;
10.
11.     always @(*) begin
12.         if(i_en==1'b1)
13.             case(i_direction)
14.                 2'b00 : r_dir = 2'b00;
15.                 2'b01 : r_dir = 2'b01;
16.                 2'b10 : r_dir = 2'b10;
17.                 2'b11 : r_dir = 2'b00;
18.                 default : r_dir = 2'b00;
```

```
19.     endcase
20.     else
21.         r_dir = 2'b00;
22.
23.     end
24. endmodule
```

AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control

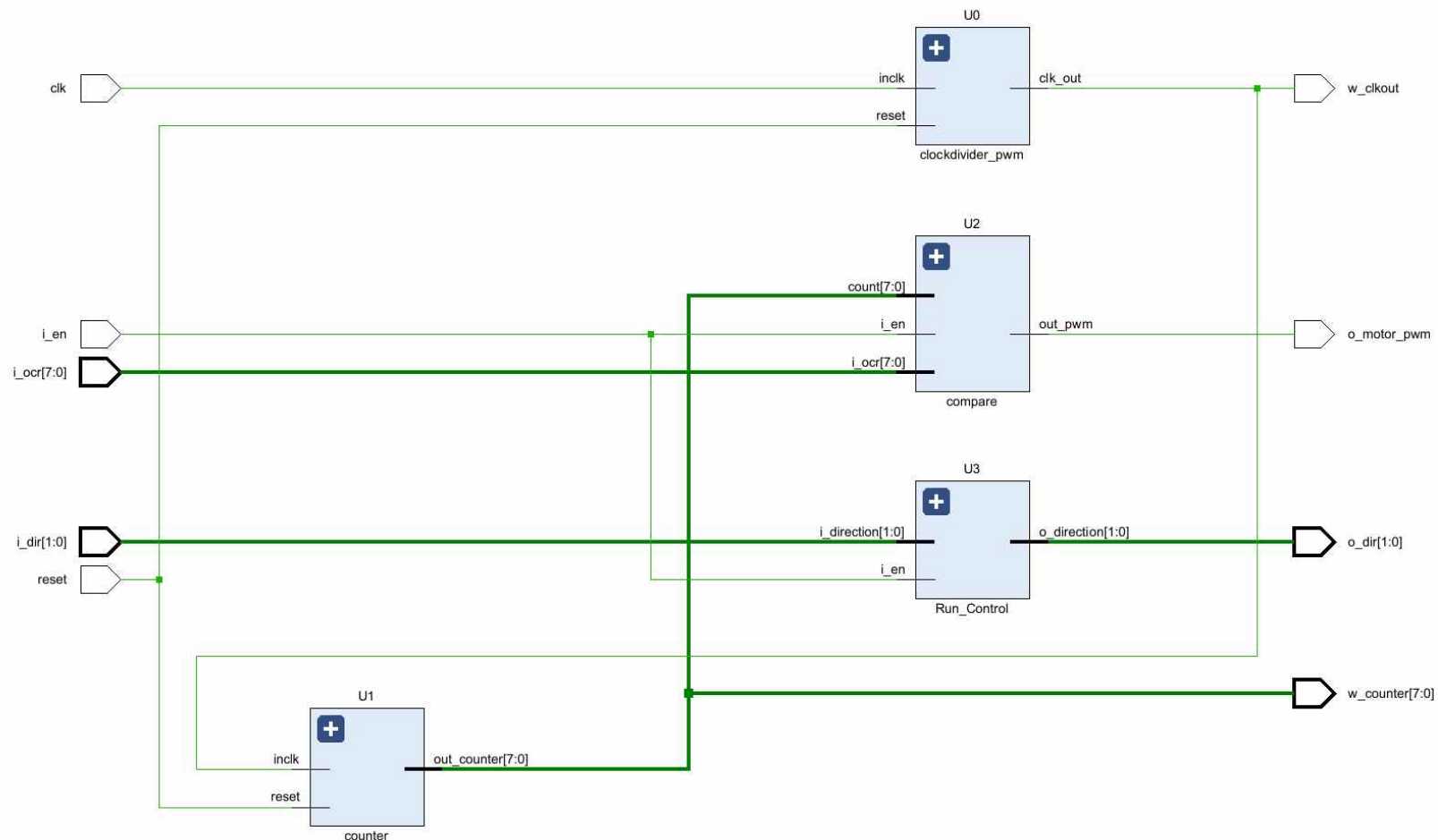
➔ **top_pwm**

axi_pwm_exam.xpr

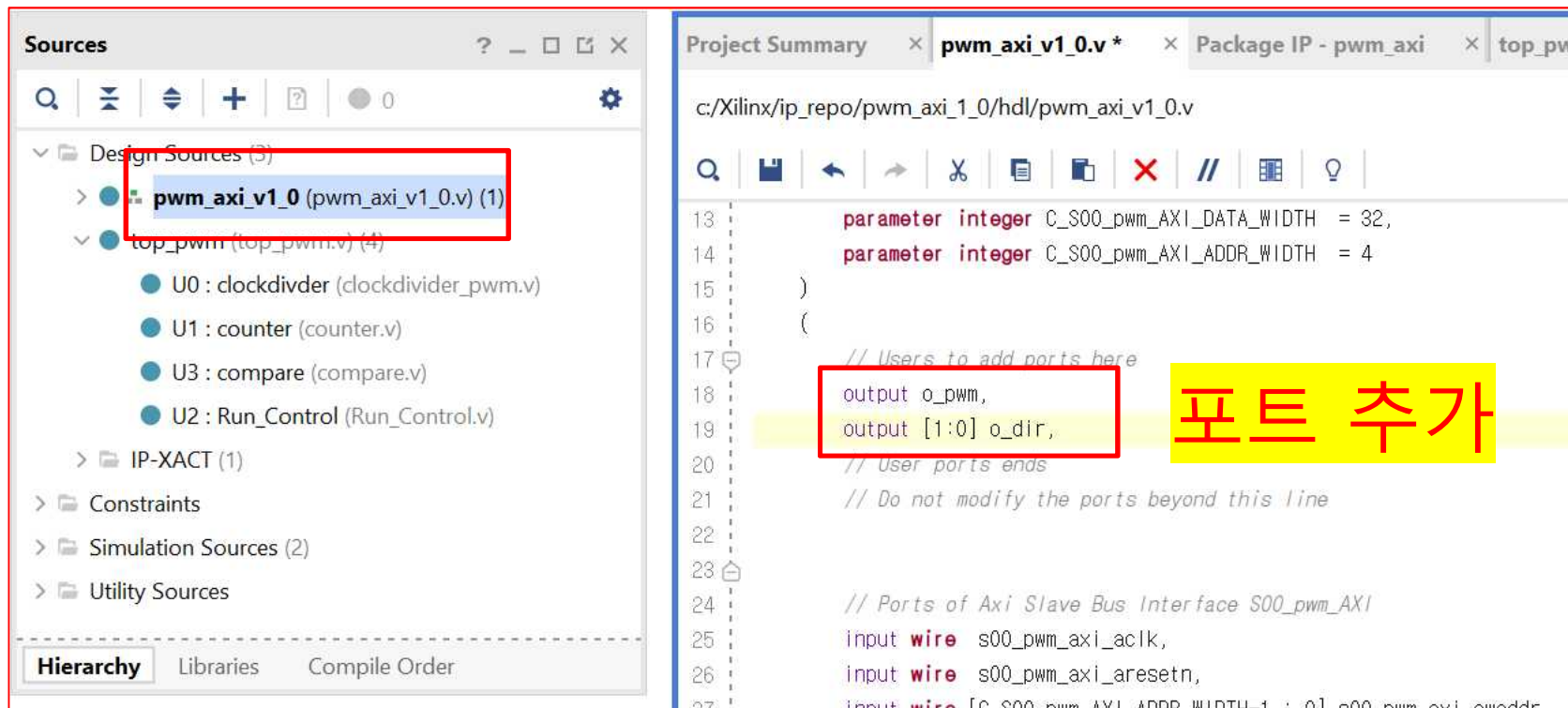
```
1. `timescale 1ns / 1ps
2. module top_pwm(
3.   reset, clk, i_ocr, i_en, i_dir, o_dir,
     o_motor_pwm, w_counter, w_clkout
4. );
5.   input reset;
6.   input clk;
7.   input [7:0] i_ocr;
8.   input i_en;
9.   input [1:0] i_dir;
10.  output [1:0] o_dir;
11.  output o_motor_pwm;
12.  output [7:0] w_counter;
13.  output w_clkout;
14.  clockdivider_pwm U0 (
15.    .inclk(clk),
16.    .reset(reset),
17.    .clk_out(w_clkout)
18.  );
```

```
19. counter U1 (
20.   .inclk(w_clkout),
21.   .reset(reset),
22.   .out_counter(w_counter)
23. );
24. wire [7:0] w_counter;
25. wire [7:0] i_ocr;
26. wire [1:0] i_dir;
27. wire [1:0] o_dir;
28. compare U2 (
29.   .count(w_counter),
30.   .i_ocr(i_ocr),
31.   .i_en(i_en),
32.   .out_pwm(o_motor_pwm)
33. );
34. Run_Control U3(
35.   .i_direction(i_dir),
36.   .o_direction(o_dir),
37.   .i_en(i_en)
38. );
39. endmodule
```

➤ AXI Connect ➔ PWM Control ➔ **top_pwm : RTL**



➤ AXI Connect ➔ PWM Control



The screenshot displays the Xilinx IDE interface. On the left, the 'Sources' window shows the project hierarchy. Under 'Design Sources (3)', the file 'pwm_axi_v1_0 (pwm_axi_v1_0.v) (1)' is highlighted with a red box. Below it, 'top_pwm (top_pwm.v) (4)' is expanded, showing components U0 (clockdivider), U1 (counter), U3 (compare), and U2 (Run_Control). At the bottom of the Sources window, the 'Hierarchy' tab is selected.

On the right, the 'Project Summary' window shows the Verilog code for 'pwm_axi_v1_0.v'. The file path is 'c:/Xilinx/ip_repo/pwm_axi_1_0/hdl/pwm_axi_v1_0.v'. The code includes parameter declarations for data and address widths. A red box highlights the output declarations: 'output o_pwm,' and 'output [1:0] o_dir,'. A yellow box with the Korean text '포트 추가' (Add Port) is overlaid on this section. Below the user ports, there are comments and declarations for the AXI slave bus interface ports.

```
13      parameter integer C_S00_pwm_AXI_DATA_WIDTH = 32,  
14      parameter integer C_S00_pwm_AXI_ADDR_WIDTH = 4  
15  )  
16  (  
17      // Users to add ports here  
18      output o_pwm,  
19      output [1:0] o_dir,  
20      // User ports ends  
21      // Do not modify the ports beyond this line  
22  
23      // Ports of Axi Slave Bus Interface S00_pwm_AXI  
24      input wire s00_pwm_axi_aclk,  
25      input wire s00_pwm_axi_aresetn,  
26      input wire [C_S00_pwm_AXI_ADDR_WIDTH-1 : 0] s00_pwm_axi_awaddr
```

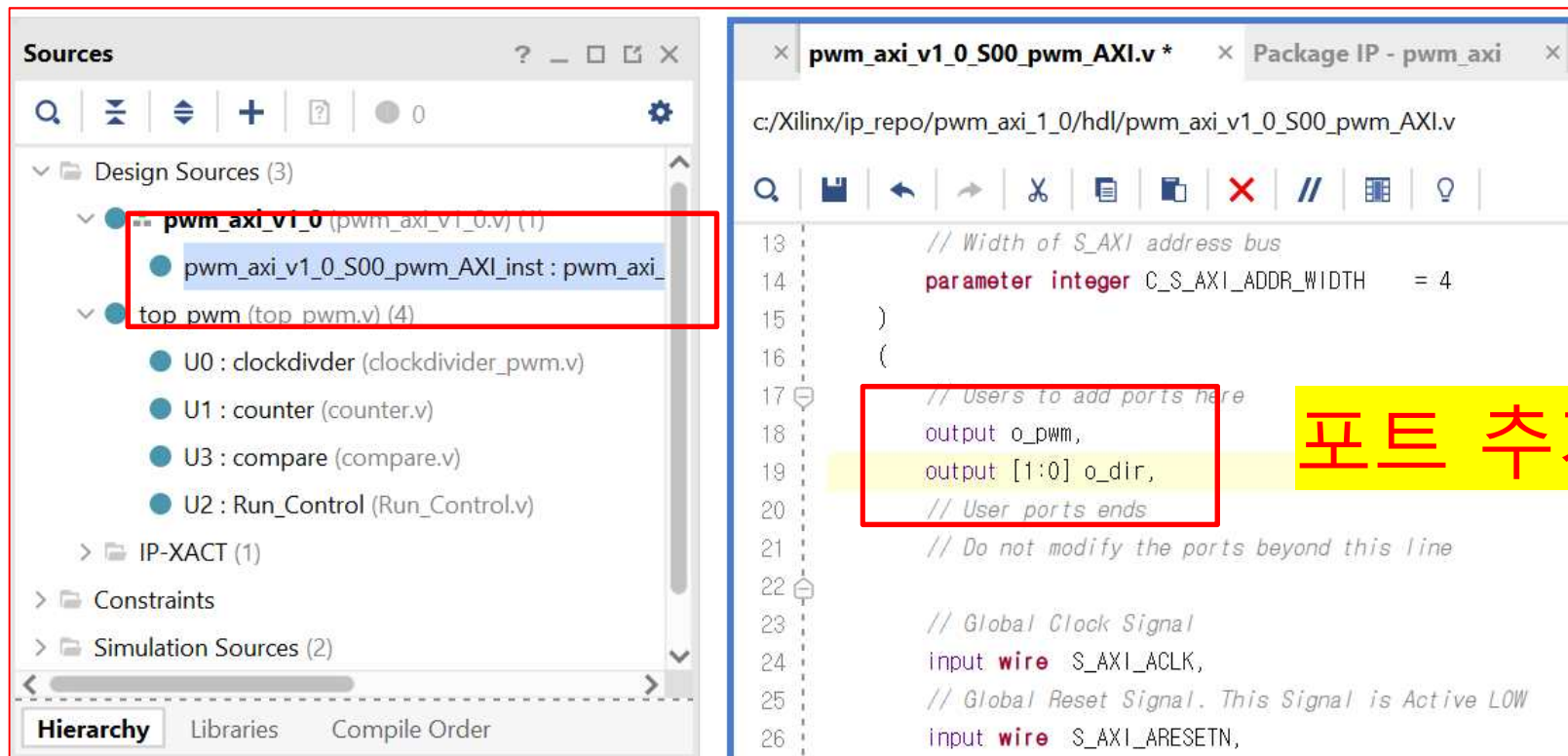
➤ AXI Connect ➔ PWM Control

The screenshot displays the Xilinx IDE interface. On the left, the 'Sources' window shows a project hierarchy. Under 'Design Sources (3)', the file 'pwm_axi_v1_0 (pwm_axi_v1_0.v) (1)' is highlighted with a red box. Below it, 'top_pwm (top_pwm.v) (4)' is expanded, showing components: 'U0 : clockdivider (clockdivider_pwm.v)', 'U1 : counter (counter.v)', 'U3 : compare (compare.v)', and 'U2 : Run_Control (Run_Control.v)'. At the bottom, the 'Hierarchy' tab is selected. On the right, the 'Project Summary' window shows the file 'pwm_axi_v1_0.v'. The code in this window is as follows:

```
45     input wire s00_pwm_axi_rready
46 );
47 // Instantiation of Axi Bus Interface S00_pwm_AXI
48 pwm_axi_v1_0_S00_pwm_AXI # (
49     .C_S_AXI_DATA_WIDTH(C_S00_pwm_AXI_DATA_WIDTH),
50     .C_S_AXI_ADDR_WIDTH(C_S00_pwm_AXI_ADDR_WIDTH)
51 ) pwm_axi_v1_0_S00_pwm_AXI_inst (
52     .o_pwm(o_pwm),
53     .o_dir(o_dir),
54     .S_AXI_ACLK(s00_pwm_axi_aclk),
55     .S_AXI_ARESETN(s00_pwm_axi_aresetn),
56     .S_AXI_AWADDR(s00_pwm_axi_awaddr),
57     .S_AXI_AWPROT(s00_pwm_axi_awprot),
58     .S_AXI_AWVALID(s00_pwm_axi_awvalid),
```

A red box highlights the instantiation arguments '.o_pwm(o_pwm),' and '.o_dir(o_dir),' on lines 52 and 53. A yellow box with the Korean text '추가' (Add) is placed next to these lines.

➤ AXI Connect ➔ PWM Control



The screenshot displays the Xilinx IDE interface. On the left, the 'Sources' window shows the project hierarchy. Under 'Design Sources (3)', the instance 'pwm_axi_v1_0_S00_pwm_AXI_inst : pwm_axi_v1_0' is highlighted with a red box. Below it, the 'top_pwm' block and its sub-components (U0: clockdivider, U1: counter, U3: compare, U2: Run_Control) are listed. The 'IP-XACT (1)' section shows the 'pwm_axi_v1_0' IP block. The 'Constraints' and 'Simulation Sources (2)' sections are also visible.

On the right, the 'pwm_axi_v1_0_S00_pwm_AXI.v' source file is open. The code defines the AXI interface and PWM control logic. A red box highlights the output ports 'o_pwm' and 'o_dir' in the '// Users to add ports here' section. A yellow box with the text '포트 추가' (Add Port) is overlaid on this section.

```
13 // Width of S_AXI address bus
14 parameter integer C_S_AXI_ADDR_WIDTH = 4
15 )
16 (
17 // Users to add ports here
18 output o_pwm,
19 output [1:0] o_dir,
20 // User ports ends
21 // Do not modify the ports beyond this line
22
23 // Global Clock Signal
24 input wire S_AXI_ACLK,
25 // Global Reset Signal. This Signal is Active LOW
26 input wire S_AXI_ARESETN,
```

➤ AXI Connect ➔ PWM Control

The screenshot displays the Xilinx IDE interface. On the left, the **Sources** window shows the project hierarchy. The **Design Sources (3)** section includes:

- pwm_axi_v1_0** (pwm_axi_v1_0.v) (1)
 - pwm_axi_v1_0_S00_pwm_AXI_inst : pwm_axi_v1_0_S00_pwm_AXI.v** (highlighted with a red box)
- top_pwm** (top_pwm.v) (4)
 - U0 : clockdivider (clockdivider_pwm.v)
 - U1 : counter (counter.v)
 - U3 : compare (compare.v)
 - U2 : Run_Control (Run_Control.v)
- IP-XACT (1)
- Constraints
- Simulation Sources (2)

Below the Sources window is the **Source File Properties** window for **pwm_axi_v1_0_S00_pwm_AXI.v**. It shows the file is **Enabled**, located at **c:/Xilinx/ip_repo/pwm_axi_1_0/hdl**, with a **Type** of **Verilog**, a **Size** of **14.1 KB**, and a **Modified** date of **Today at 14:17:23 PM**.

The main editor window shows the Verilog code for **pwm_axi_v1_0_S00_pwm_AXI.v**. The code is as follows:

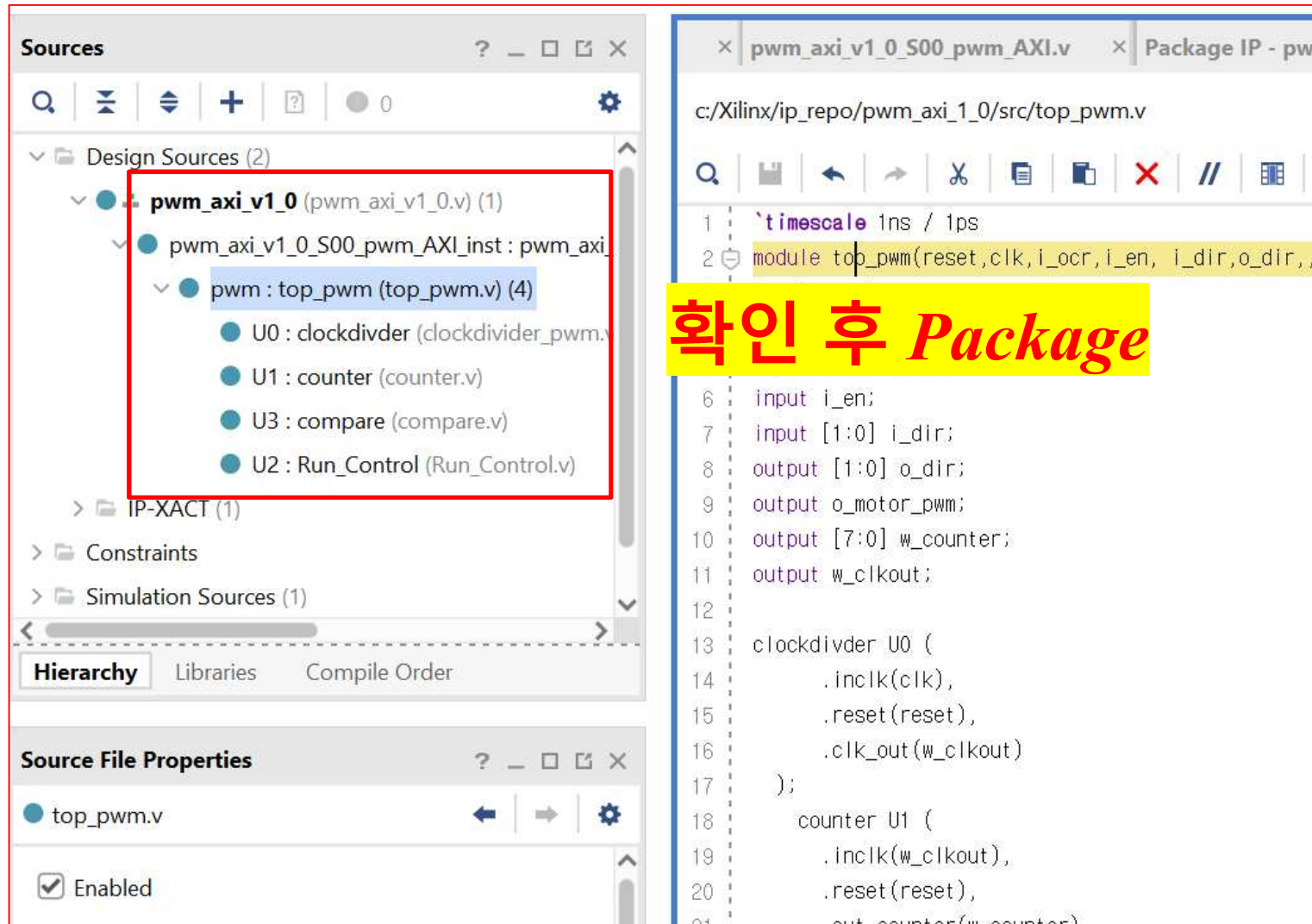
```

390 begin
391     // When there is a valid read address (S_AXI_ARVALID) with
392     // acceptance of read address by the slave (axi_arready),
393     // output the read data
394     if (slv_reg_rden)
395     begin
396         axi_rdata <= reg_data_out;    // register read data
397     end
398 end
399 end
400
401 // Add user logic here
402 top_pwm pwm(
403     .reset(S_AXI_ARESETN),
404     .clk(S_AXI_ACLK),
405     .i_ocr(i_ocr),
406     .i_en(i_en),
407     .i_dir(i_dir),
408     .o_dir(o_dir),
409     .o_motor_pwm(o_pwm)
410 );
411 wire i_en;
412 wire [1:0] i_dir;
413 wire [7:0] i_ocr;
414 assign i_en = slv_reg0;
415 assign i_dir = slv_reg1;
416 assign i_ocr = slv_reg2;
417 // User logic ends
418
419 endmodule
420

```

A red box highlights the user logic section (lines 401-416). A yellow box highlights the assignment statements (lines 414-416). A yellow box with the Korean text **추가** (Add) is placed next to the highlighted code.

➤ AXI Connect ➔ PWM Control



Sources

- Design Sources (2)
 - pwm_axi_v1_0 (pwm_axi_v1_0.v) (1)
 - pwm_axi_v1_0_S00_pwm_AXI_inst : pwm_axi_v1_0_S00_pwm_AXI.v (1)
 - pwm : top_pwm (top_pwm.v) (4)
 - U0 : clockdivider (clockdivider_pwm.v)
 - U1 : counter (counter.v)
 - U3 : compare (compare.v)
 - U2 : Run_Control (Run_Control.v)
- IP-XACT (1)
- Constraints
- Simulation Sources (1)

Source File Properties

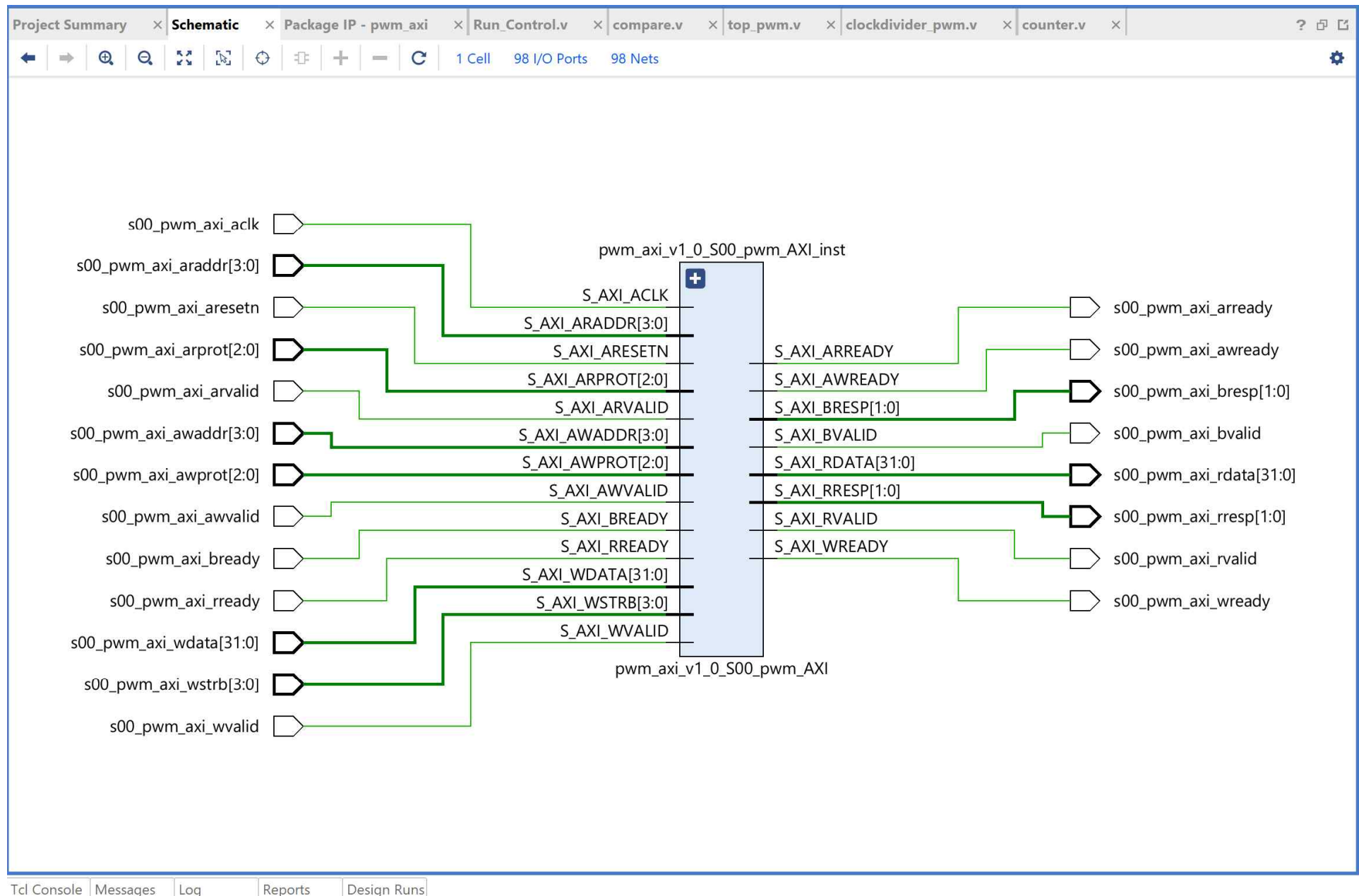
- top_pwm.v
 - Enabled

Package IP - pwm

```
c:/Xilinx/ip_repo/pwm_axi_1_0/src/top_pwm.v

1 `timescale 1ns / 1ps
2 module top_pwm(reset,clk,i_ocr,i_en, i_dir,o_dir,,
3
4
5
6 input i_en;
7 input [1:0] i_dir;
8 output [1:0] o_dir;
9 output o_motor_pwm;
10 output [7:0] w_counter;
11 output w_clkout;
12
13 clockdivider U0 (
14     .inclk(clk),
15     .reset(reset),
16     .clk_out(w_clkout)
17 );
18 counter U1 (
19     .inclk(w_clkout),
20     .reset(reset),
21     .out_counter(w_counter)
```

확인 후 Package



AMBA and AXI Connect Example

PWM Control

[*Create and Package New IP*]

➤ AXI Connect ➔ PWM Control

The screenshot shows the Vivado 2022.2 IDE interface. The 'Tools' menu is open, and 'Create and Package New IP...' is highlighted. The 'Package IP - axi-fnd' dialog is open, showing the following fields:

- Vendor: xilinx.com
- Library: user
- Name: axi-fnd
- Version: 1.0
- Display name: axi-fnd_v1.0
- Description: My new AXI IP
- Vendor display name:
- Company url:
- Root directory: c:/Xilinx/ip_repo/axi-fnd_1_0
- Xml file name: c:/Xilinx/ip_repo/axi-fnd_1_0/component.xml

The 'Categories' section shows 'AXI_Peripheral'.

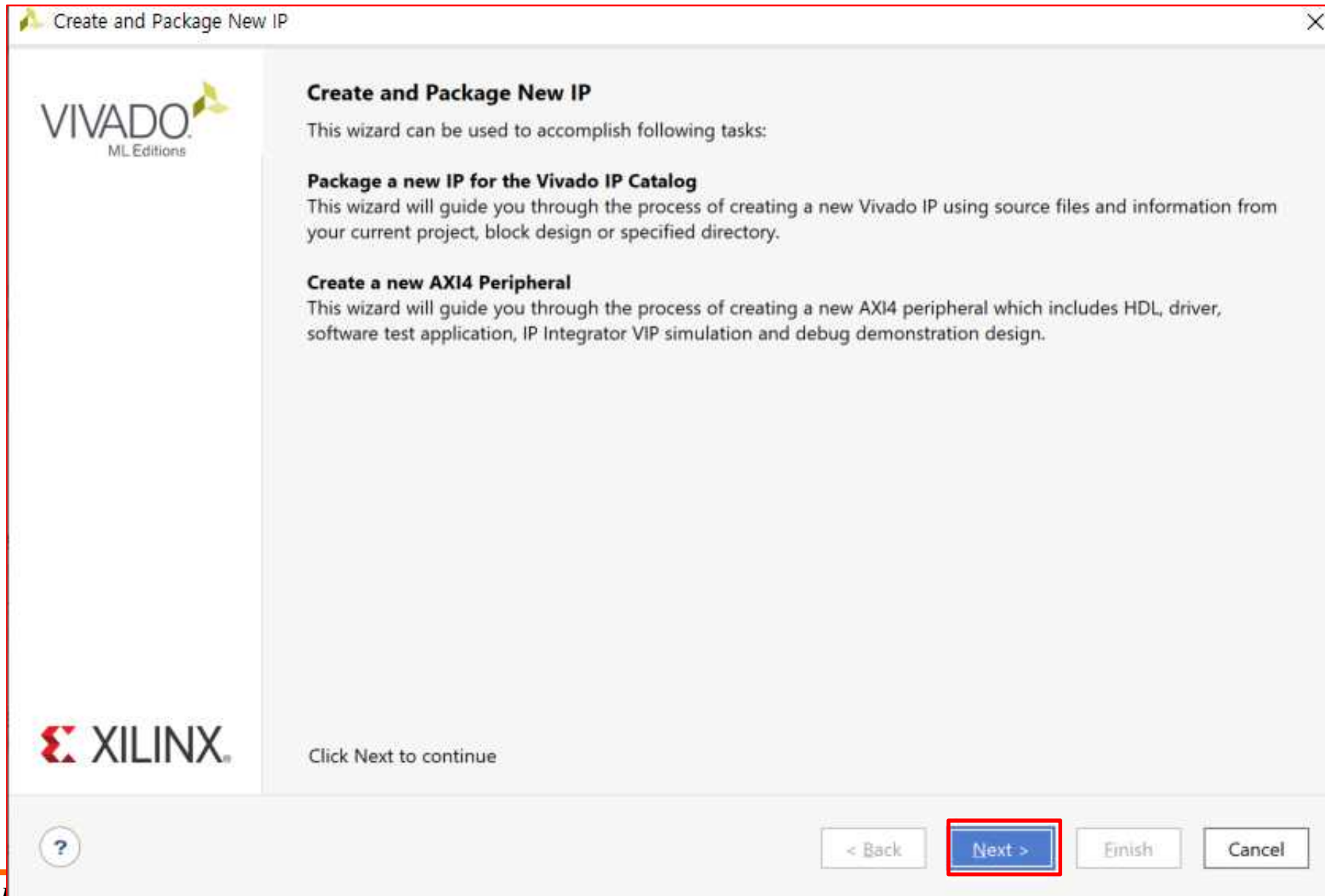
The 'Source File Properties' window is also open, showing details for the 'axi-fnd_v1_0.v' file:

- Location: c:/Xilinx/ip_repo/axi-fnd_1_0/hdl
- Type: Verilog
- Library:
- Size: 3.3 KB
- Modified: Today at 10:08:32 AM
- Read-only: No

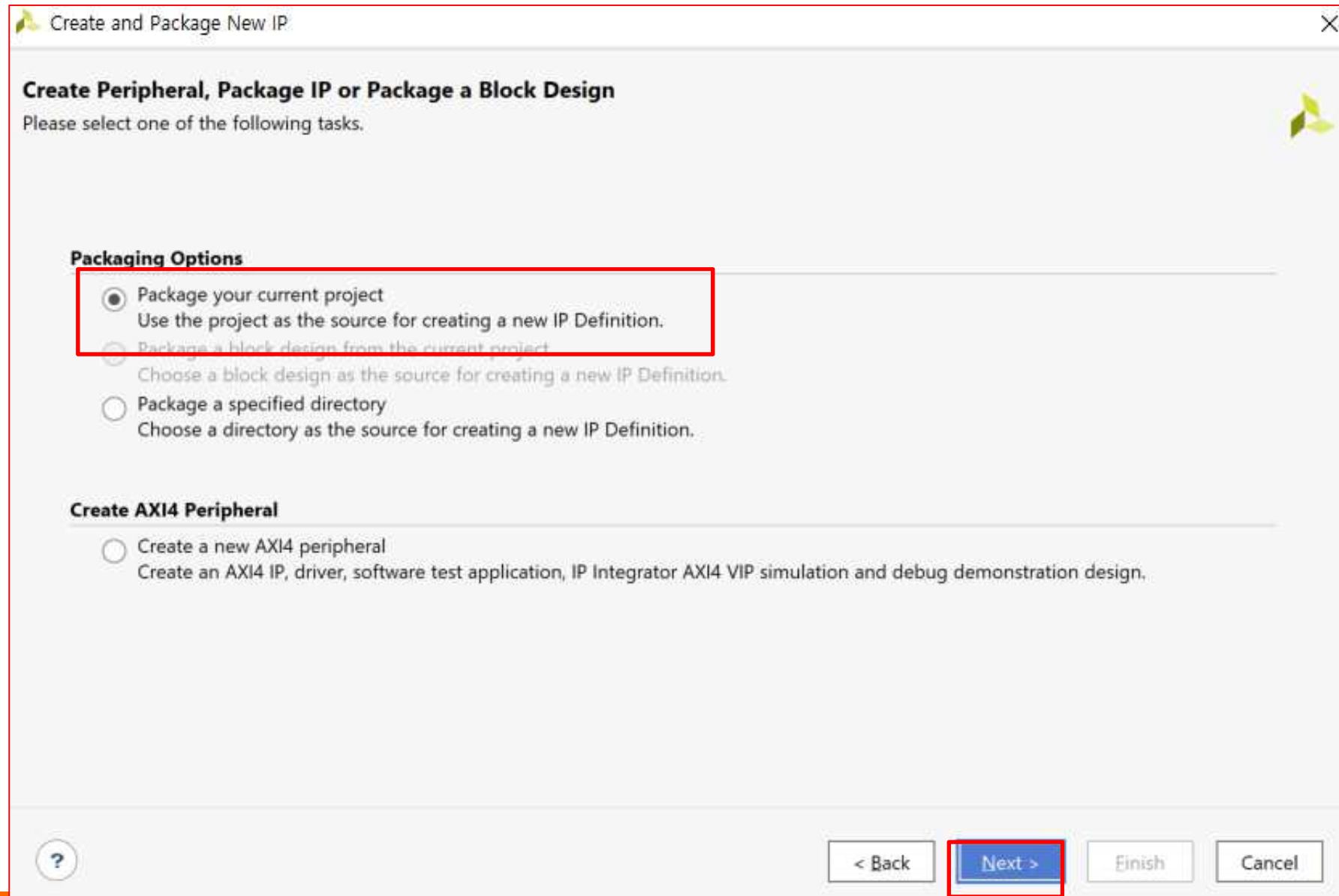
The 'Design Runs' table at the bottom shows the following data:

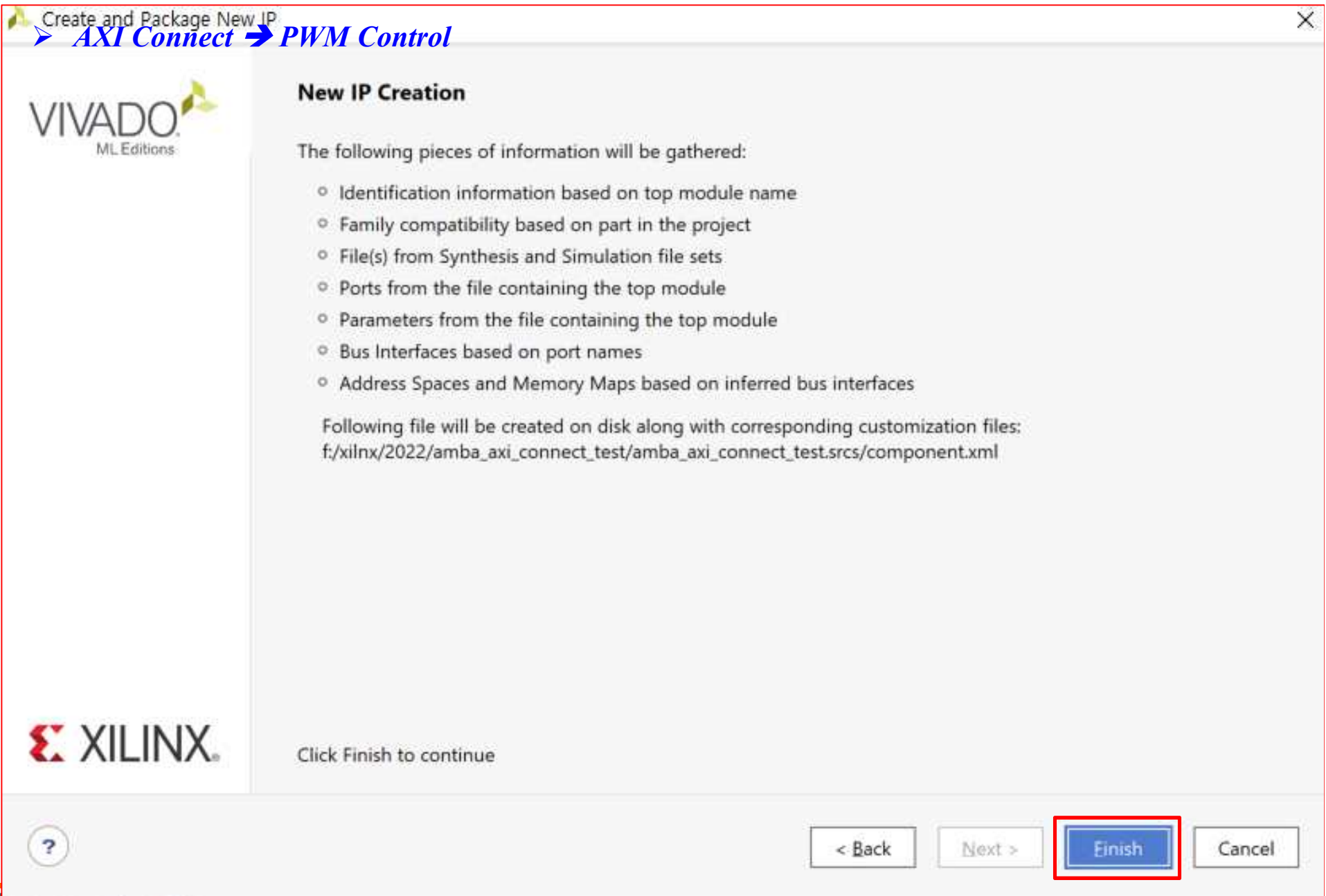
Name	Constrai...	Status	...	W...	T...	Total P...	Failed Ro...	Methodol...	RQA S...	QoR Suggest...	...	B...	U...	S...	Ela...	Run Strategy	Report Stra
synth_1	constrs_1	Not started														Vivado Synthesis Defaults (Vivado Synthesis 2022)	Vivado Syn
impl_1	constrs_1	Not started														Vivado Implementation Defaults (Vivado Implementation 2022)	Vivado Imp

➤ AXI Connect ➔ PWM Control

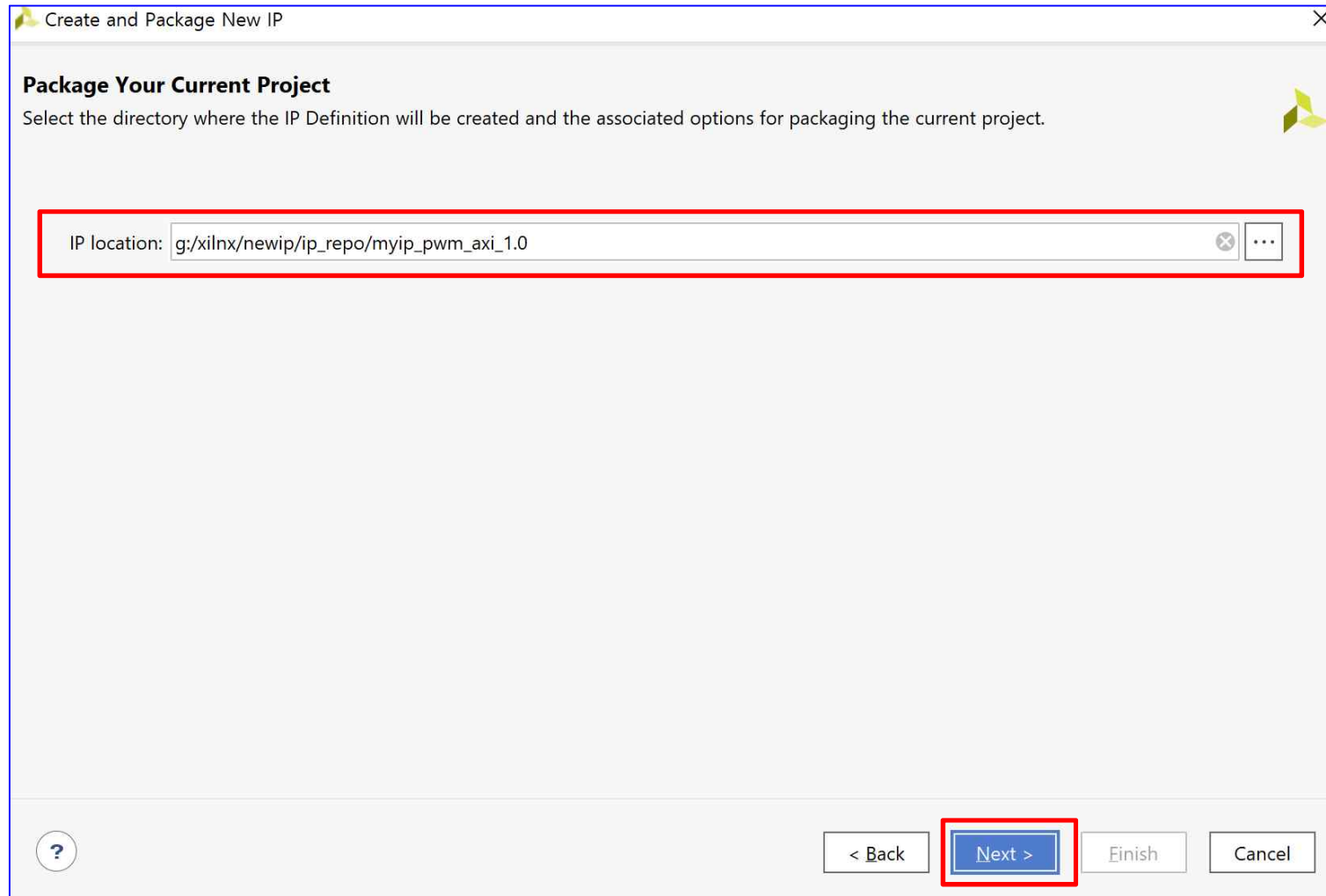


➤ AXI Connect ➔ PWM Control





➤ AXI Connect ➔ PWM Control

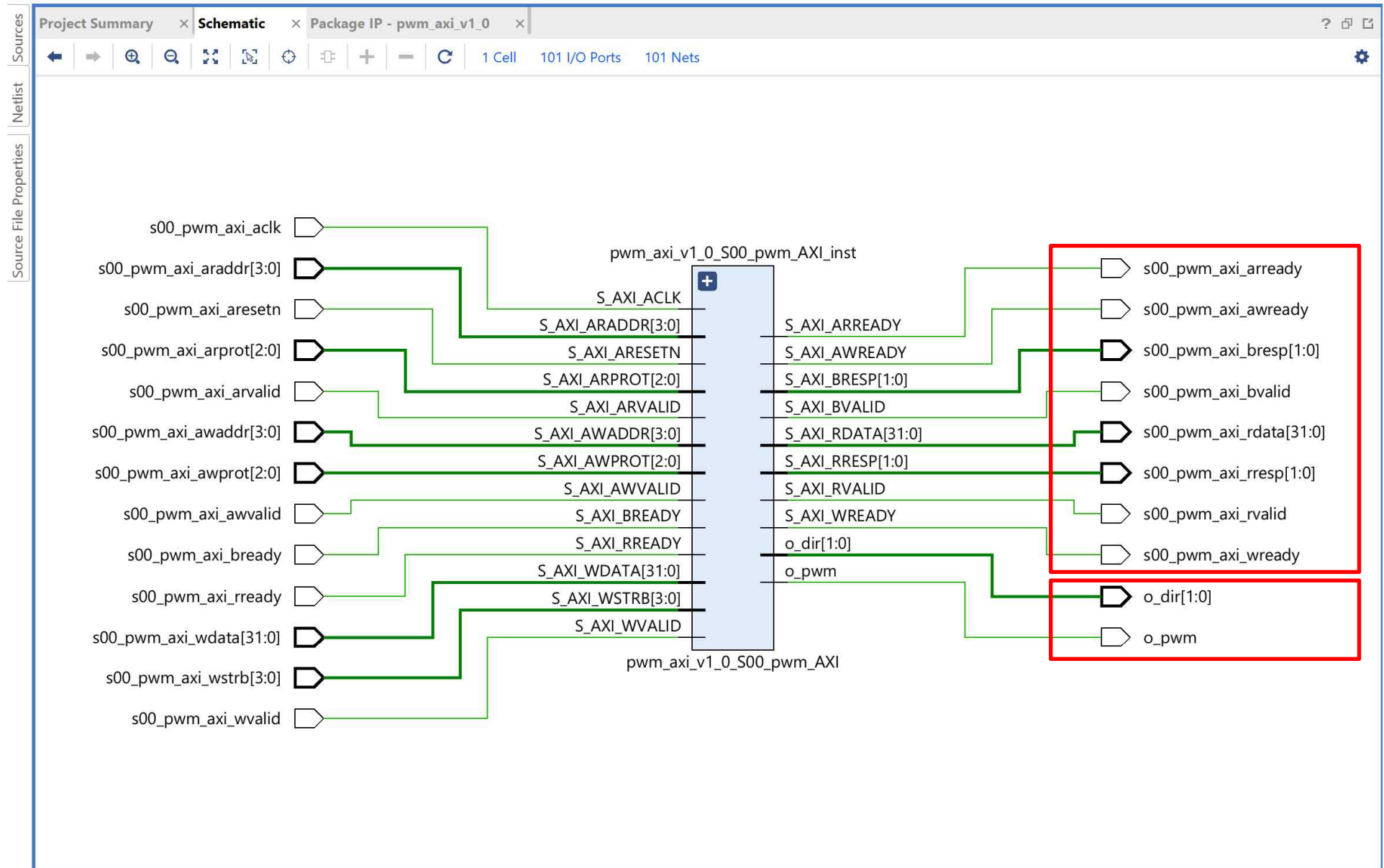


➤ AXI Connect ➔ PWM Control

The screenshot displays the Vivado 2022.1 Project Manager interface for a project named 'myip_pwm_axi_v1_0_project'. The 'PROJECT MANAGER' tab is active, showing a tree view of the project structure. The 'Sources' pane lists 'Design Sources (2)', including 'myip_pwm_axi_v1_0' and 'myip_pwm_axi_v1_0_S00_pwm_AXI.v'. The 'IP-XACT (1)' pane shows 'constr_1'. The 'Constraints' pane shows 'constr_1'. The 'Simulation Sources (1)' pane shows 'Utility Sources'. The 'Source File Properties' pane shows the properties for 'vm_axi_v1_0_S00_pwm_AXI.v', including 'Location', 'Type' (Verilog), 'Library', 'Size' (14.6 KB), and 'Modified' (Today at 14:26:26 PM). The 'Packaging Steps' pane is highlighted with a blue box, showing a list of steps: Identification, Compatibility, File Groups, Customization Parameters, Ports and Interfaces, Addressing and Memory, Customization GUI, and Review and Package. The 'Identification' pane is highlighted with a red box, showing fields for Vendor (xilinx.com), Library (user), Name (myip_pwm_axi_v1_0), Version (1.0), Display name (myip_pwm_axi_v1_0_v1_0), Description (myip_pwm_axi_v1_0_v1_0), Vendor display name, Company url, Root directory (g:/xilinx/newip/ip_repo/myip_pwm_axi_1.0), and Xml file name (g:/xilinx/newip/ip_repo/myip_pwm_axi_1.0/component.xml). The 'Categories' pane shows a list of categories, including '/UserIP'. A yellow box with the text 'MyIP 생성' is overlaid on the 'Packaging Steps' pane.

➤ AXI Connect ➔ PWM Control

ELABORATED DESIGN - xc7a35tcpg236-1



AMBA and AXI Connect Example

PWM Control

[*Create Block Design*]

➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control

Settings 클릭

Project Summary

Overview | Dashboard

Settings [Edit](#)

Project name: project_1
 Project location: C:/Users/parkj/test231013/project 1
 Product family: Artix-7
 Project part: Basys3 (xc7a35tcpg236-1)
 Top module name: Not defined
 Target language: Verilog
 Simulator language: Mixed

Board Part

Display name: Basys3
 Board part name: digilentinc.com:basys3:part0:1.2
 Board revision: C.0
 Connectors: No connections
 Repository path: E:/Xilinx/Vivado/2022.2/data/boards/board_files
 URL: <https://digilent.com/reference/programmable-logic/basys-3/start>
 Board overview: Basys3

Synthesis **Implementation**

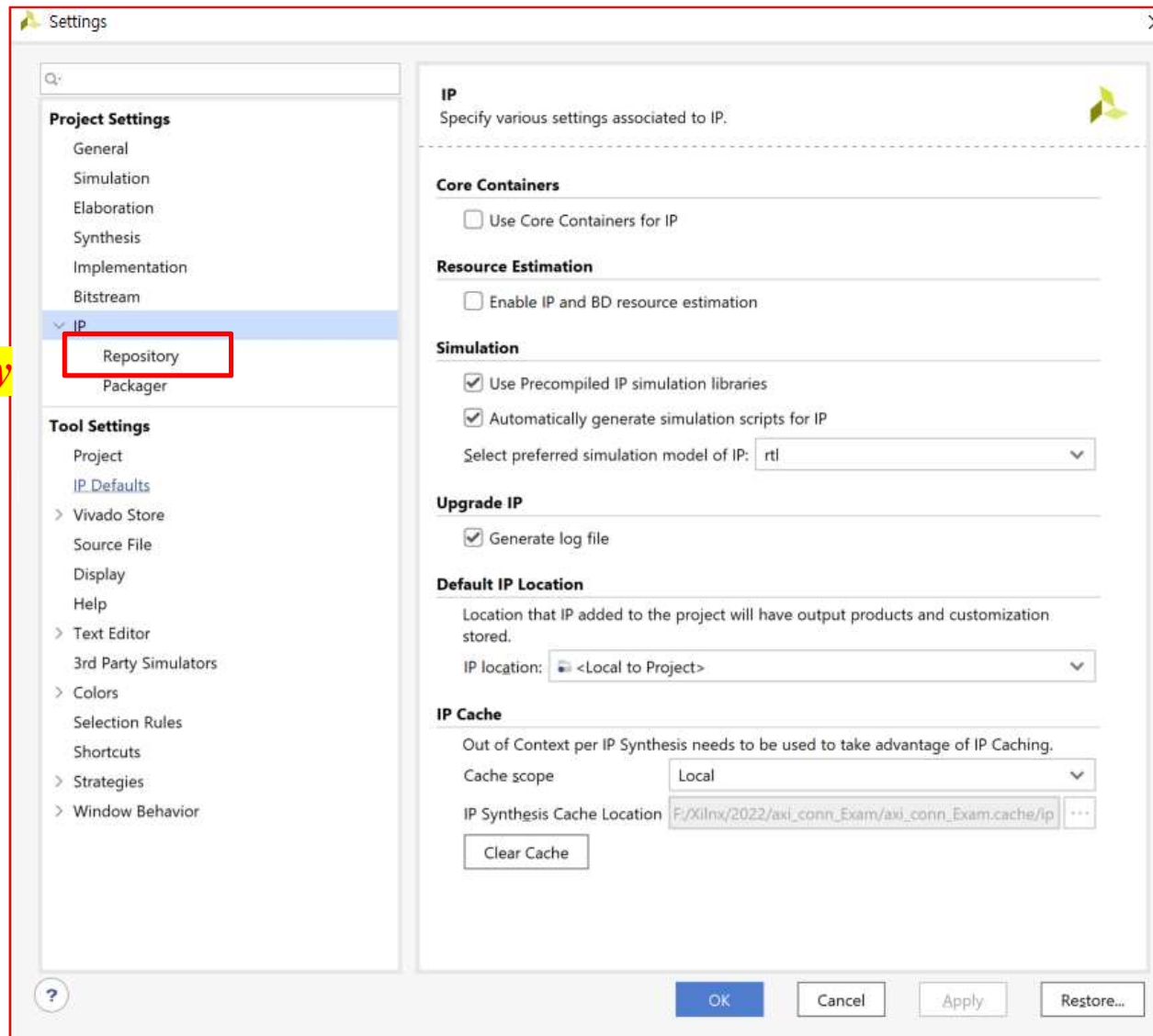
Status: Not started
 Messages: No errors or warnings

Design Runs

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy
synth_1	constrs_1	Not started																			Vivado Synthesis Defaults (Viva
impl_1	constrs_1	Not started																			Vivado Implementation Default

➤ AXI Connect ➔ PWM Control

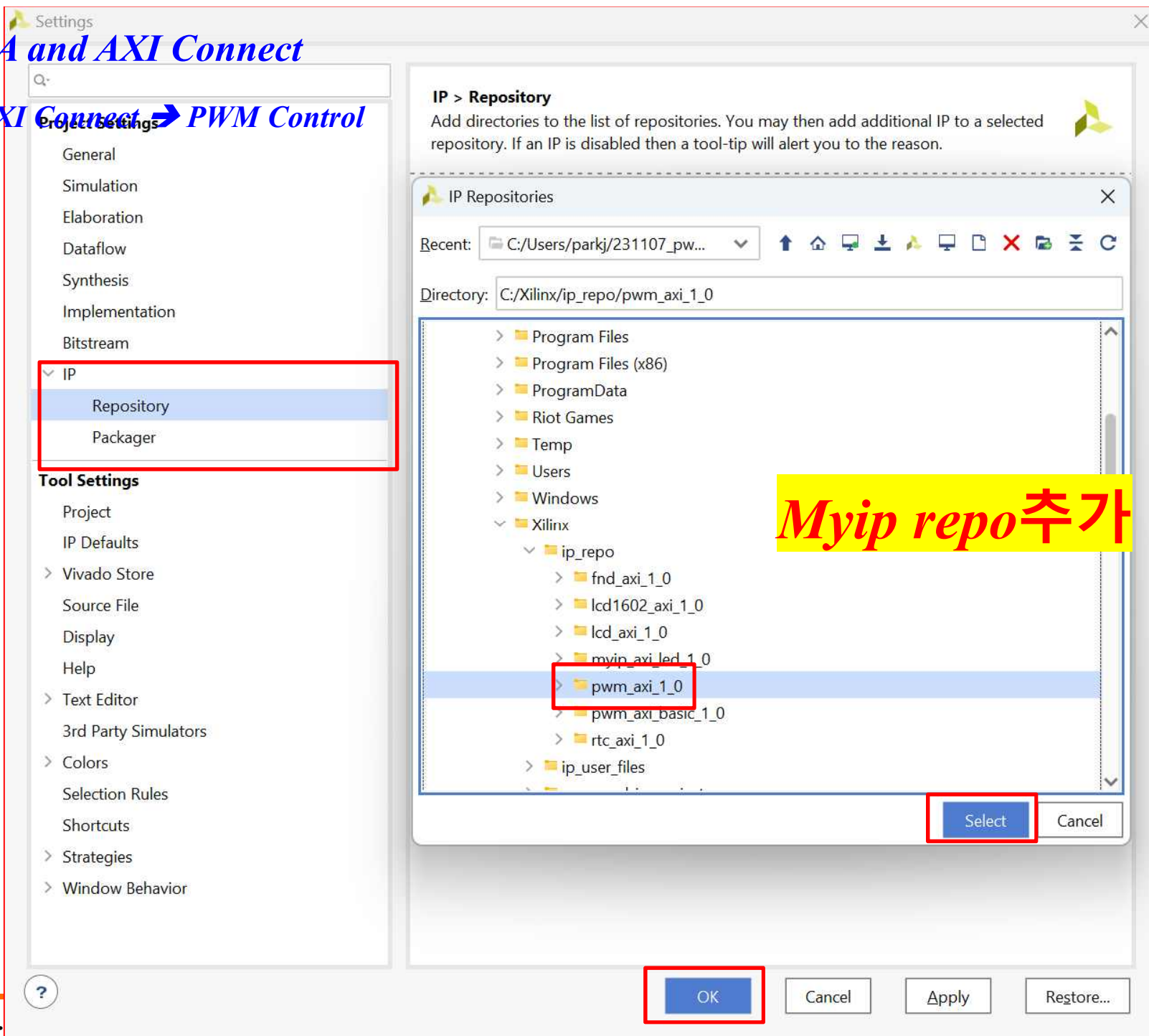
Repository



AMBA and AXI Connect

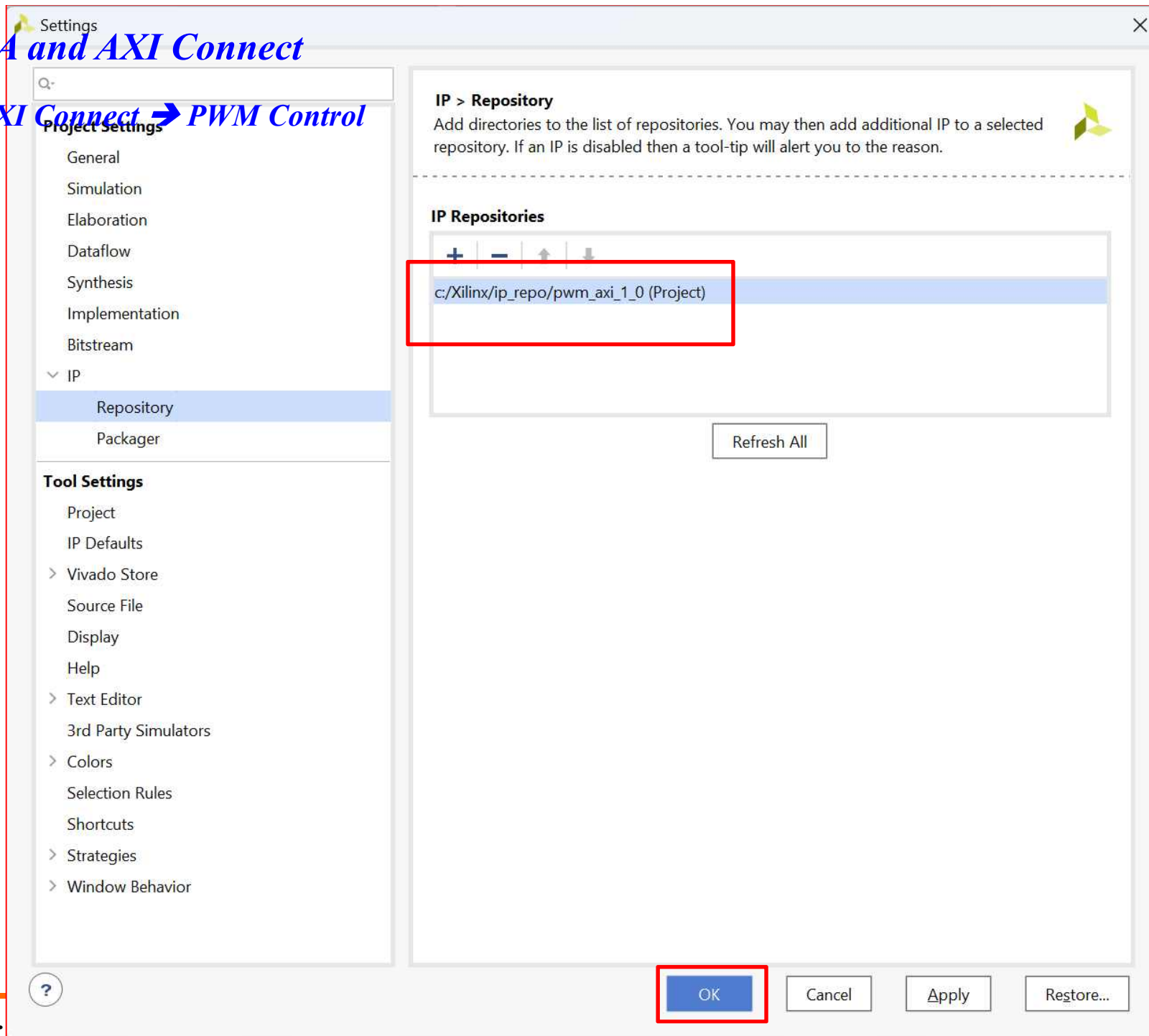
exam.xpr

➤ AXI Connect ➔ PWM Control



AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control



exam.xpr

➤ AXI Connect ➔ PWM Control

The screenshot shows the IP Catalog window in Vivado. The 'Cores' tab is selected, and the 'AXI Peripheral' section is expanded under the 'User Repository (c:/Xilinx/ip_repo/pwm_axi_1_0)'. The 'pwm_axi' entry is highlighted. A red box is drawn around the 'AXI Peripheral' section. A yellow box with the text 'IP Catalog에서 확인' is overlaid on the right side of the image.

Name	AXI4	Status	License	VLMV
✓ User Repository (c:/Xilinx/ip_repo/pwm_axi_1_0)				
✓ AXI Peripheral				
pwm_axi	AXI4	Product	Included	xilinx.cc
> Vivado Repository				

Details

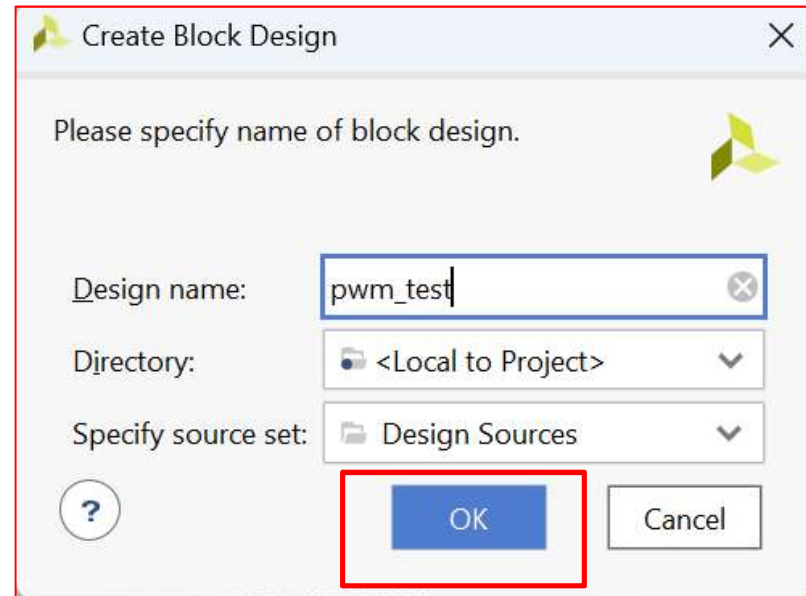
Select an IP or Interface or Repository to see details

➤ AXI Connect ➔ PWM Control

The screenshot displays the Xilinx Vivado IDE interface. On the left, the 'Flow Navigator' pane shows the 'IP INTEGRATOR' section with the 'Create Block Design' option highlighted. The main workspace is divided into several panes: 'Sources' (showing Design, Constraints, Simulation, and Utility sources), 'Repository Properties' (showing the path c:/Xilinx/ip_repo/axi_fnd_1_0), 'Project Summary' (showing project details like name, location, and board part), and 'Design Runs' (showing a table of synthesis and implementation runs). A large yellow text overlay 'Create Block Design' is positioned over the 'Sources' pane.

Create Block Design

➤ AXI Connect ➔ PWM Control



이름 설정 후 OK

➤ AXI Connect ➔ PWM Control

Microblaze, uart, pwm_axi, gpio 추가

Search: microblaze (3 matches)

- MicroBlaze
- MicroBlaze Debug Module (MDM)
- MicroBlaze MCS

ENTER to select, ESC to cancel, Ctrl+Q for IP

Search: uart (2 matches)

- AXI UART16550
- AXI Uartlite

ENTER to select, ESC to cancel, Ctrl+Q for IP

Search: pwm (1 match)

- pwm_axi_v1.0

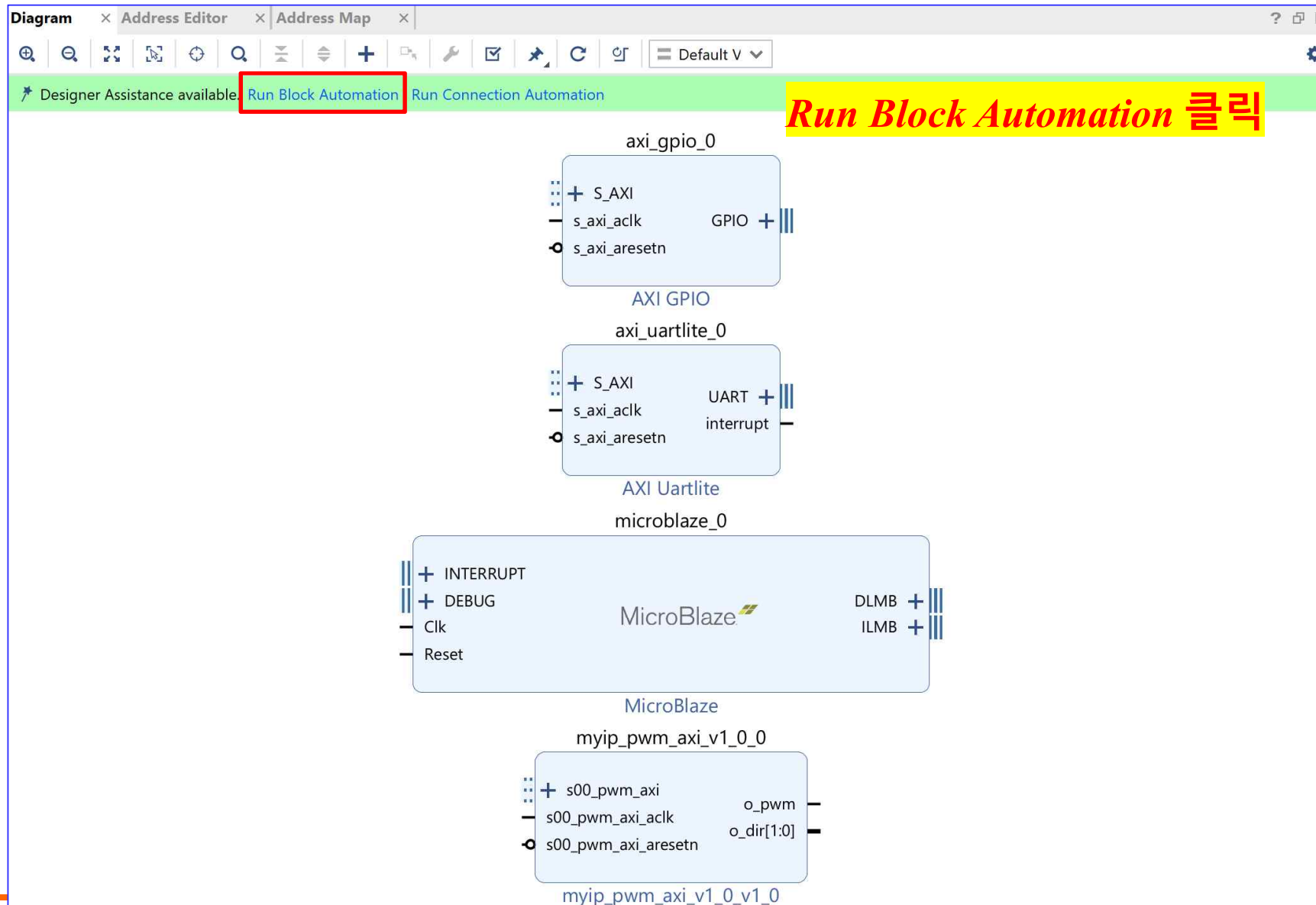
ENTER to select, ESC to cancel, Ctrl+Q for IP

Search: gpio (1 match)

- AXI GPIO

ENTER to select, ESC to cancel, Ctrl+Q for IP

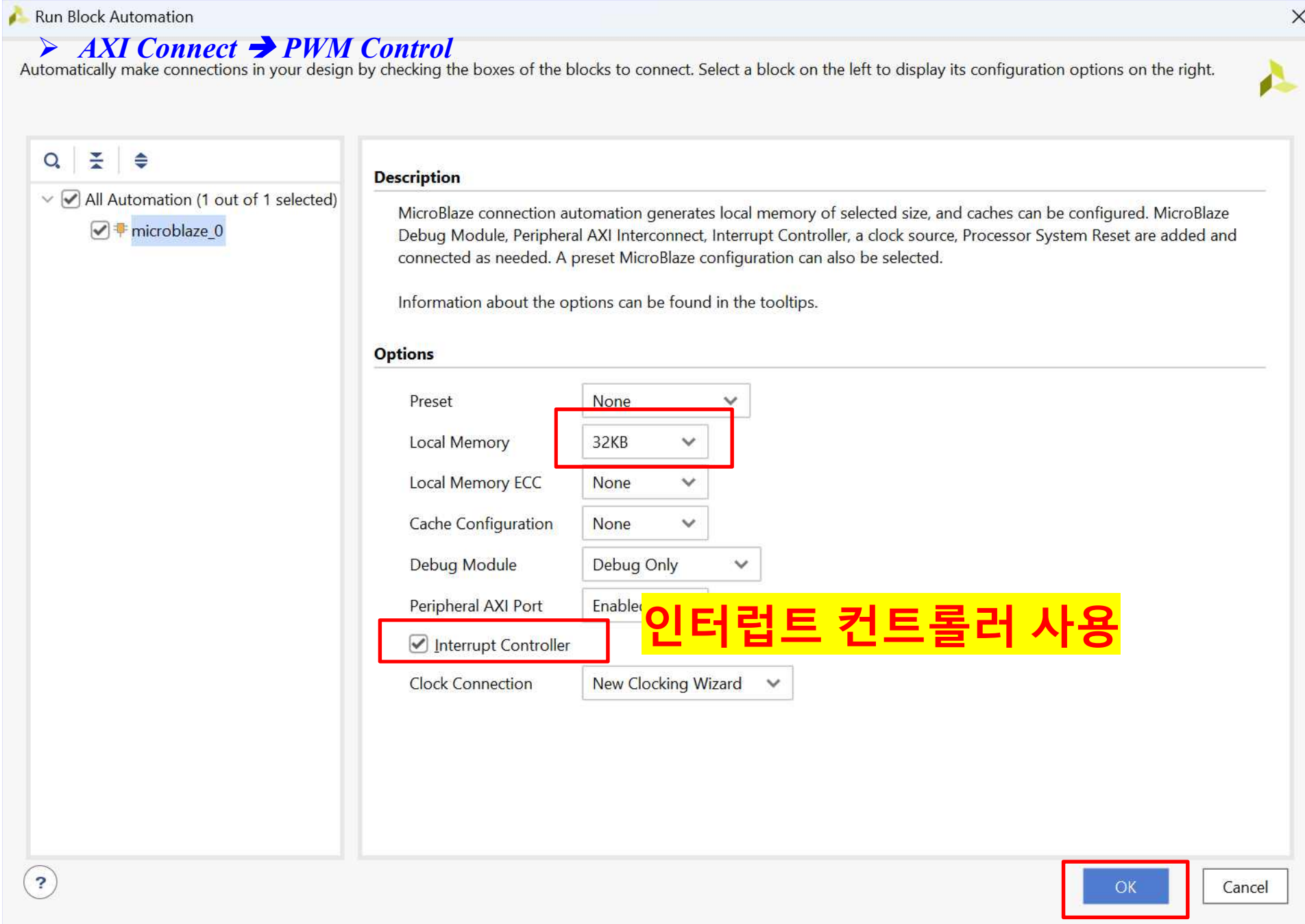
➤ AXI Connect ➔ PWM Control



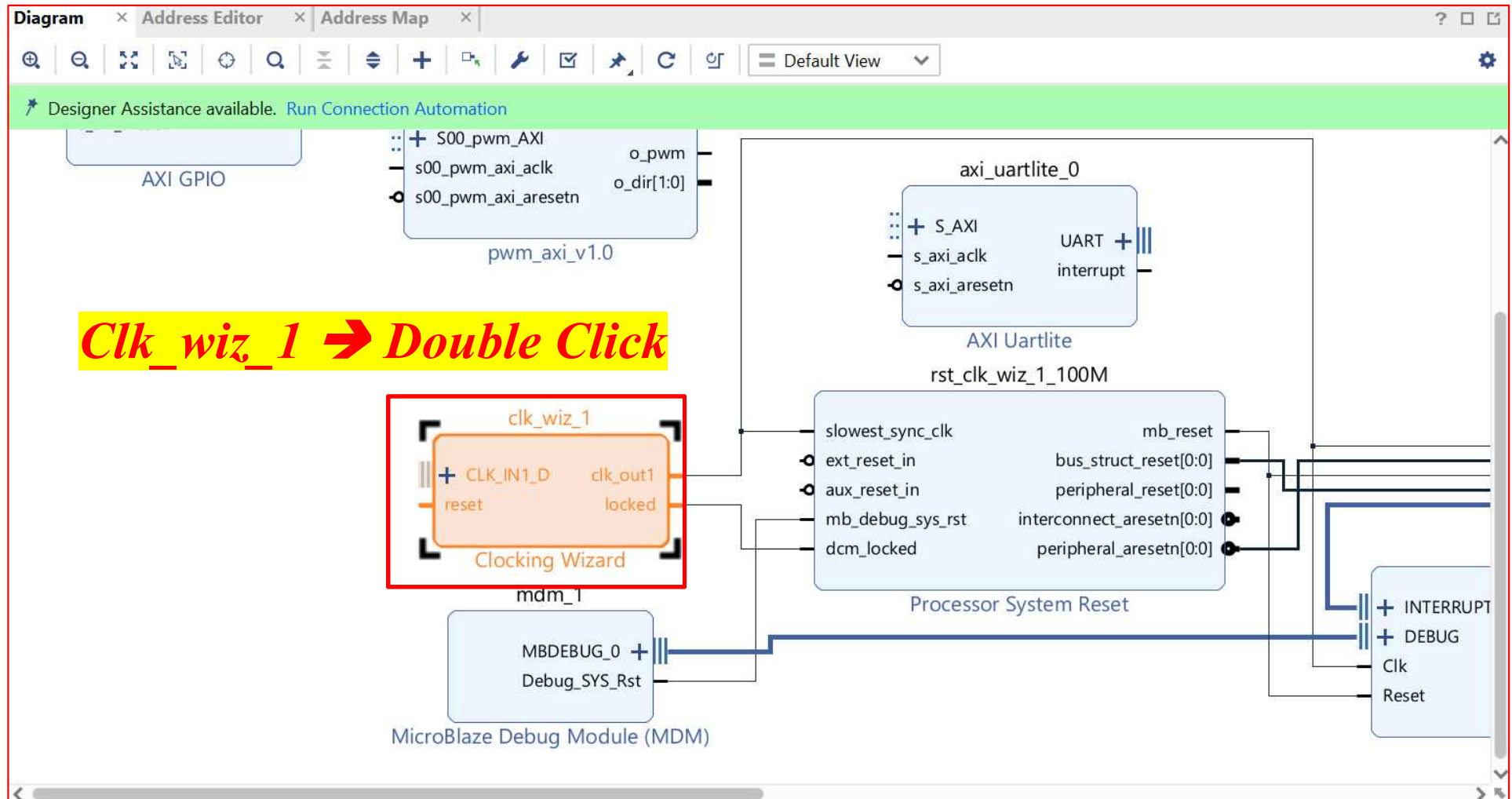
Run Block Automation 클릭

AMBA and AXI Connect

axi_pwm_exam.xpr



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control

Re-customize IP

Documentation IP Location

IP Symbol Resource

☒ Show disabled ports

Component Name: clk_wiz_1

Board **Clocking Options** Output Clocks MMCM Settings Summary

Clock Monitor

☐ Enable Clock Monitoring

Primitive

☒ MMCM ☐ PLL

Clocking Features

☒ Frequency Synthesis ☐ Minimize Power

☒ Phase Alignment ☐ Spread Spectrum

☐ Dynamic Reconfig ☐ Dynamic Phase Shift

☐ Safe Clock Startup

Jitter Optimization

☒ Balanced ☐ Minimize Output Jitter

☐ Maximize Input Jitter filtering

Dynamic Reconfig Interface Options

☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

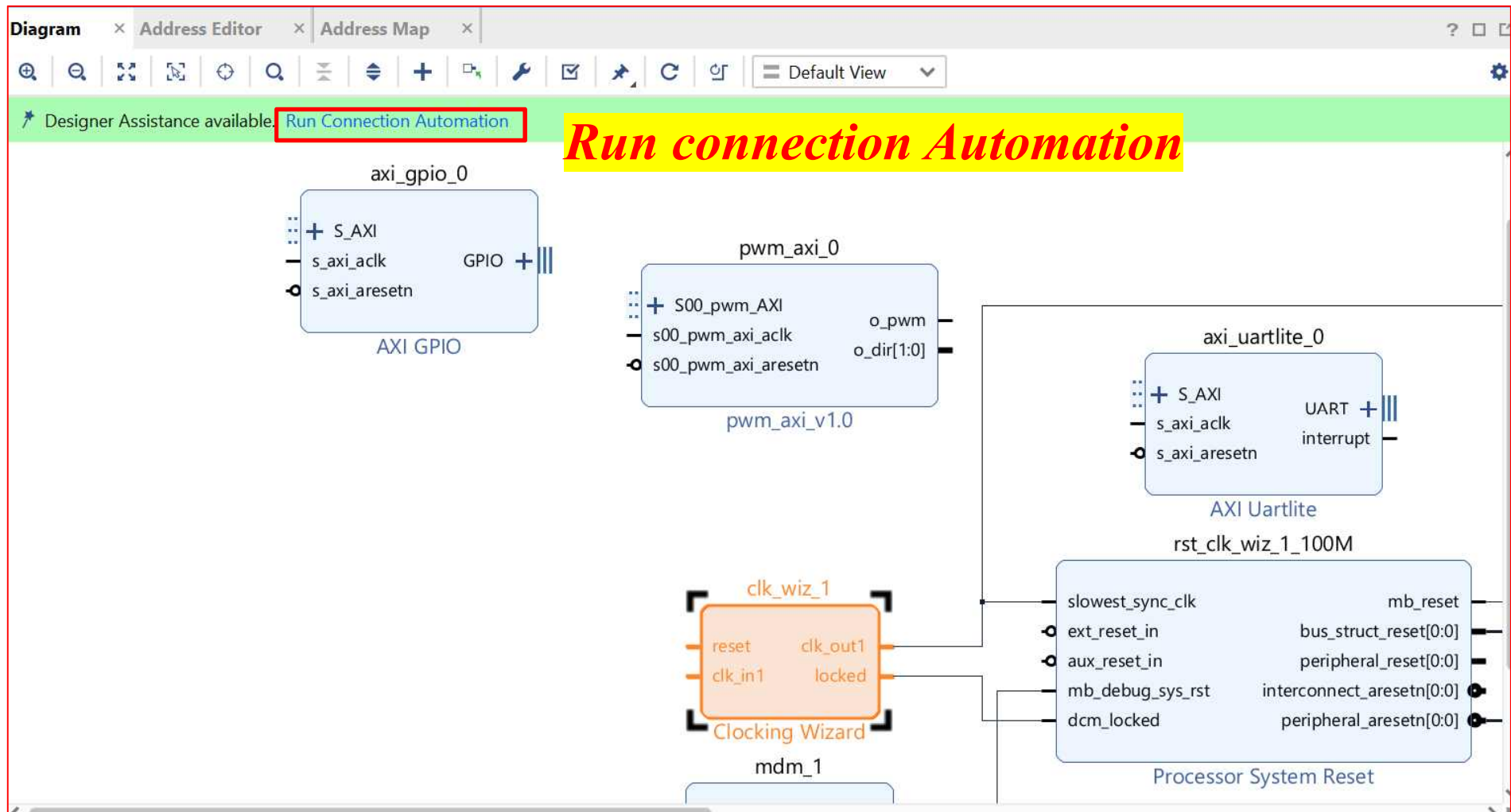
Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☒	Primary	clk_in1	AUTO 100.000	10.000 - 800.000	UI	0.010	Single ended clock capable p
☐	Secondary	clk_in2	AUTO 100.000	60.000 - 120.000		0.010	Single ended clock capable p

OK Cancel

Single ended clock 변경 후 OK

➤ AXI Connect ➔ PWM Control



Run Connection Automation



➤ AXI Connect ➔ PWM Control

Automatically make connections in your design by checking the boxes of the interfaces to connect. Select an interface on the left to display its configuration options on the right.



☒ All Automation (8 out of 8 selected)

☒ axi_gpio_0

- ☒ GPIO
- ☒ S_AXI

☒ axi_uartlite_0

- ☒ S_AXI
- ☒ UART

☒ clk_wiz_1

- ☒ clk_in1
- ☒ reset

☒ pwm_axi_0

- ☒ S00_pwm_AXI

☒ rst_clk_wiz_1_100M

- ☒ ext_reset_in

Select an interface pin on the left panel to view its options

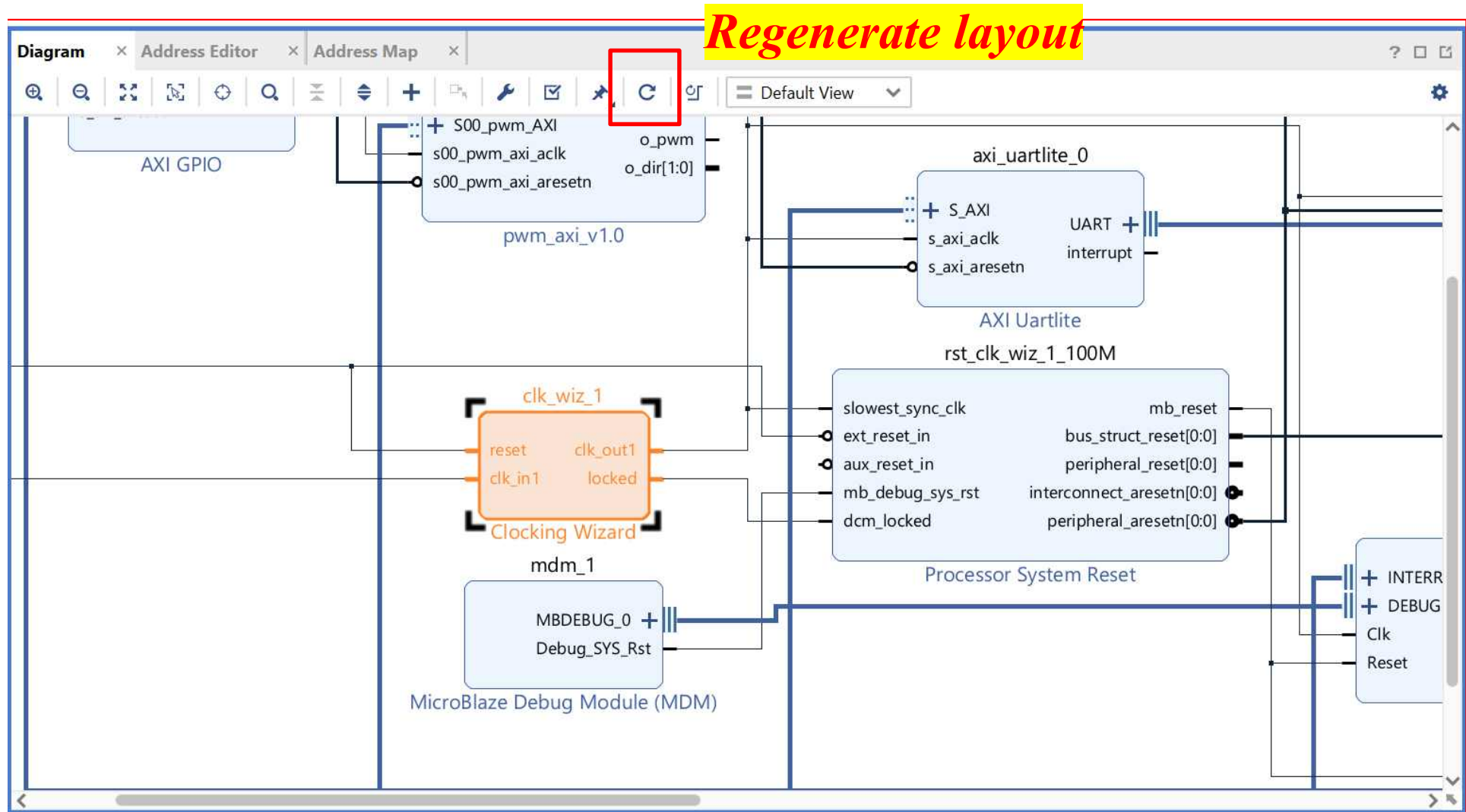
All Automation 선택 후 OK



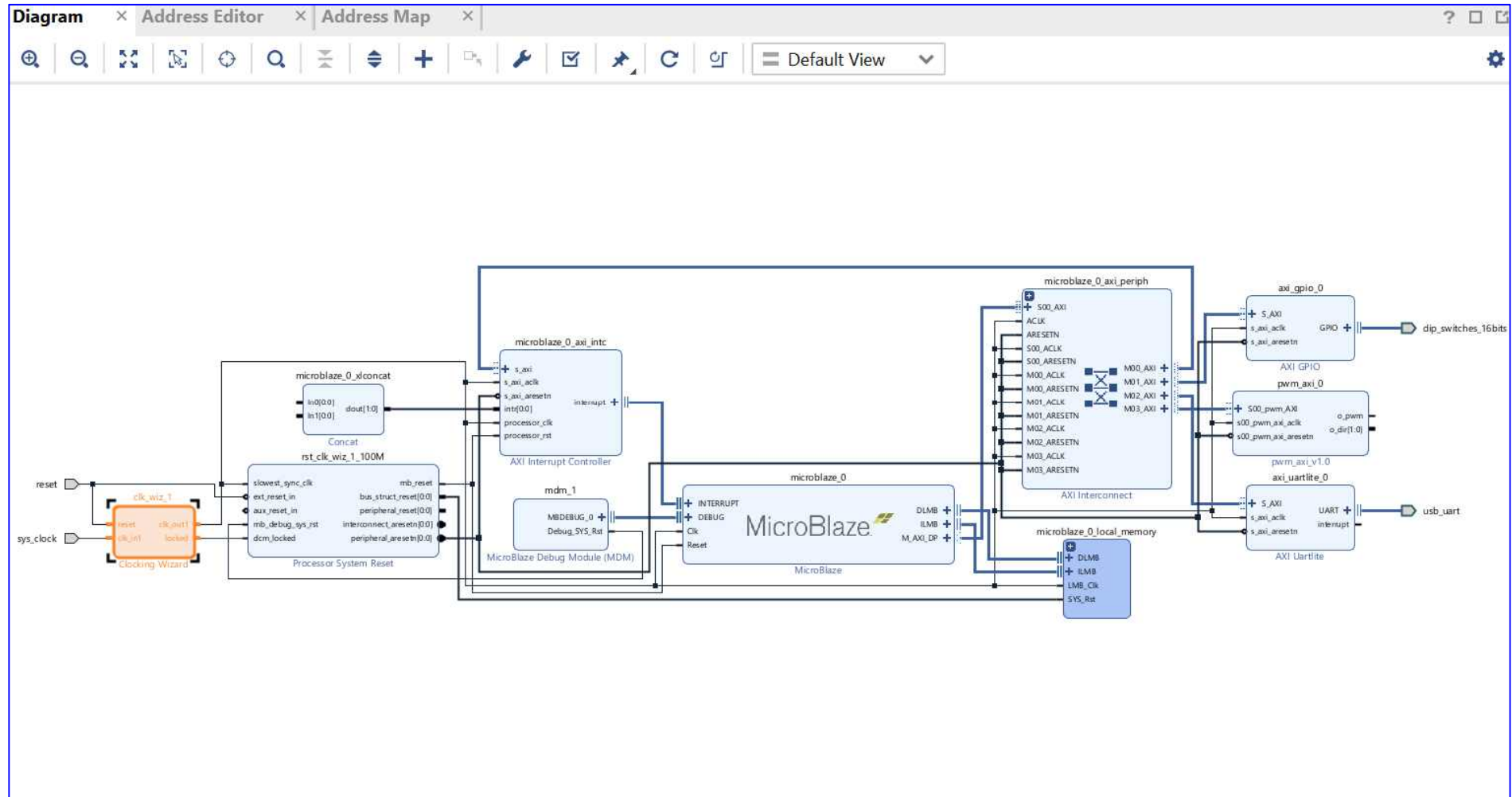
OK

Cancel

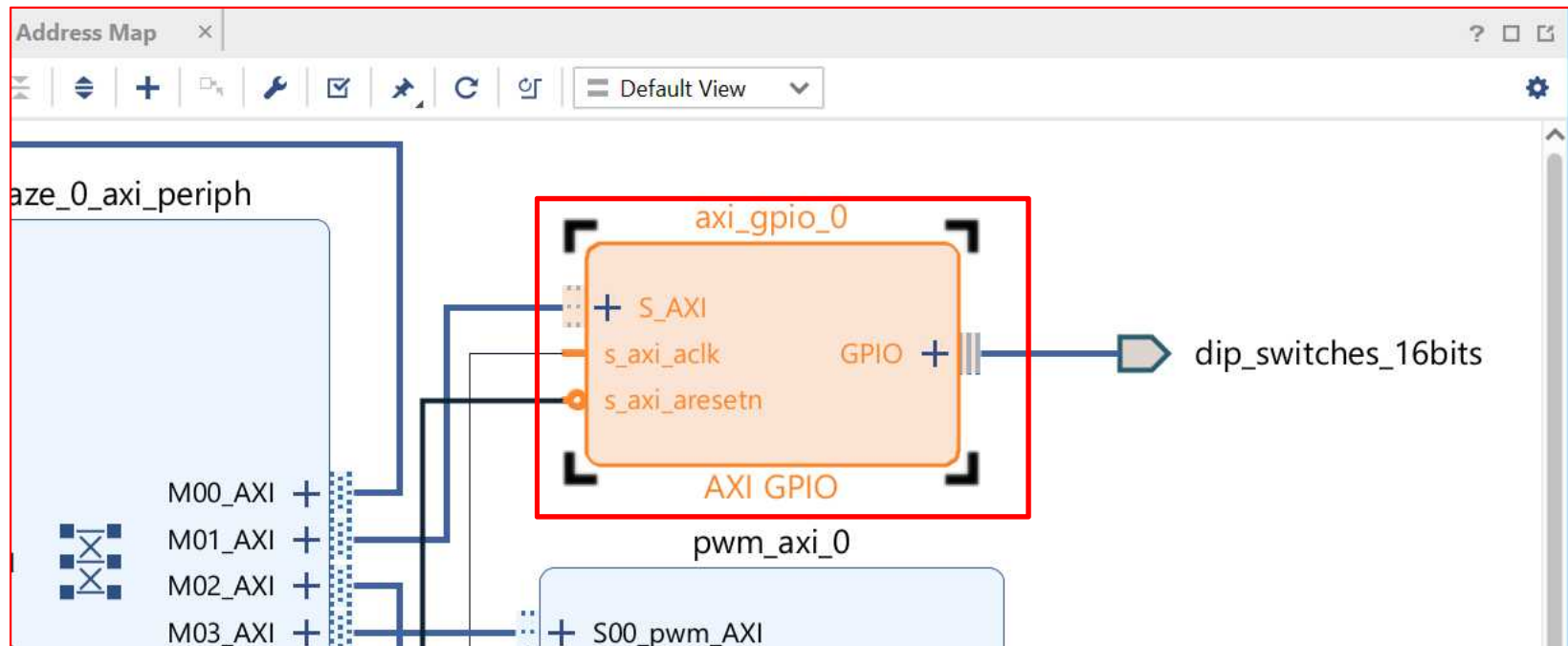
➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



Gpio 더블클릭 후 수정

AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control

Re-customize IP

AXI GPIO (2.0)

Documentation IP Location

☐ Show disabled ports

Component Name axi_gpio_0

Board IP Configuration

Associate IP interface with board interface

IP Interface	Board Interface
GPIO	dip switches 16bits
GPIO2	Custom

Clear Board Parameters

☐ Enable Interrupt

OK Cancel

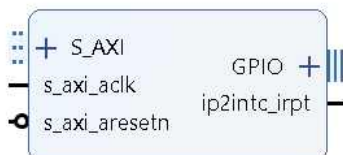
Gpio Custom 변경

AMBA and AXI Connect

➤ AXI GPIO (2.0) → PWM Control

Documentation IP Location

☐ Show disabled ports



Component Name axi_gpio_0

Board IP Configuration

GPIO

☒ All Inputs

☐ All Outputs

GPIO Width 4 [1 - 32]

Default Output Value 0x00000000 [0x00000000,0xFFFFFFFF]

Default Tri State Value 0xFFFFFFFF [0x00000000,0xFFFFFFFF]

☐ Enable Dual Channel

GPIO 2

☐ All Inputs

☐ All Outputs

GPIO Width 32 [1 - 32]

Default Output Value 0x00000000 [0x00000000,0xFFFFFFFF]

Default Tri State Value 0xFFFFFFFF [0x00000000,0xFFFFFFFF]

☒ Enable Interrupt

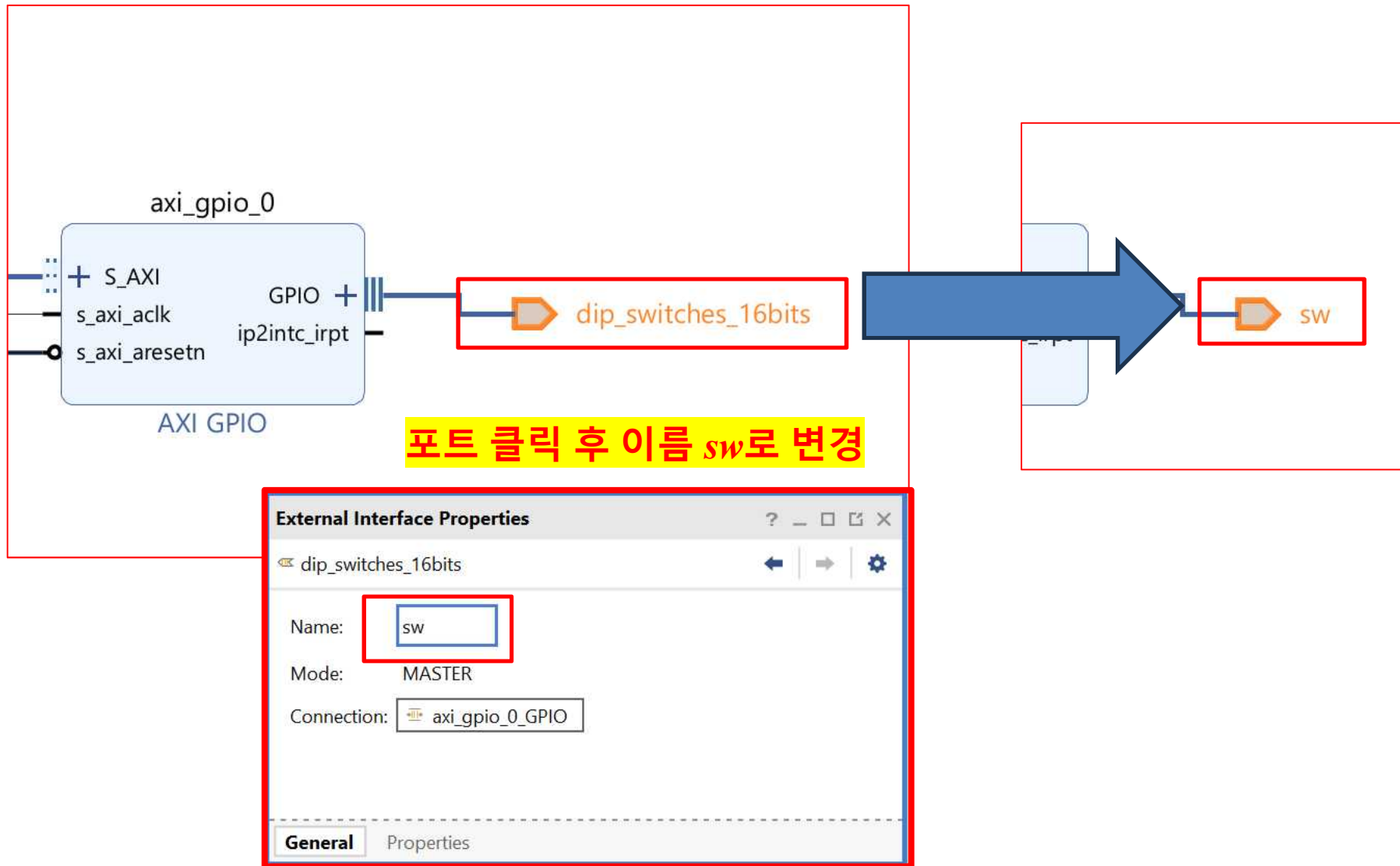
변경 후 OK

인터럽트 사용

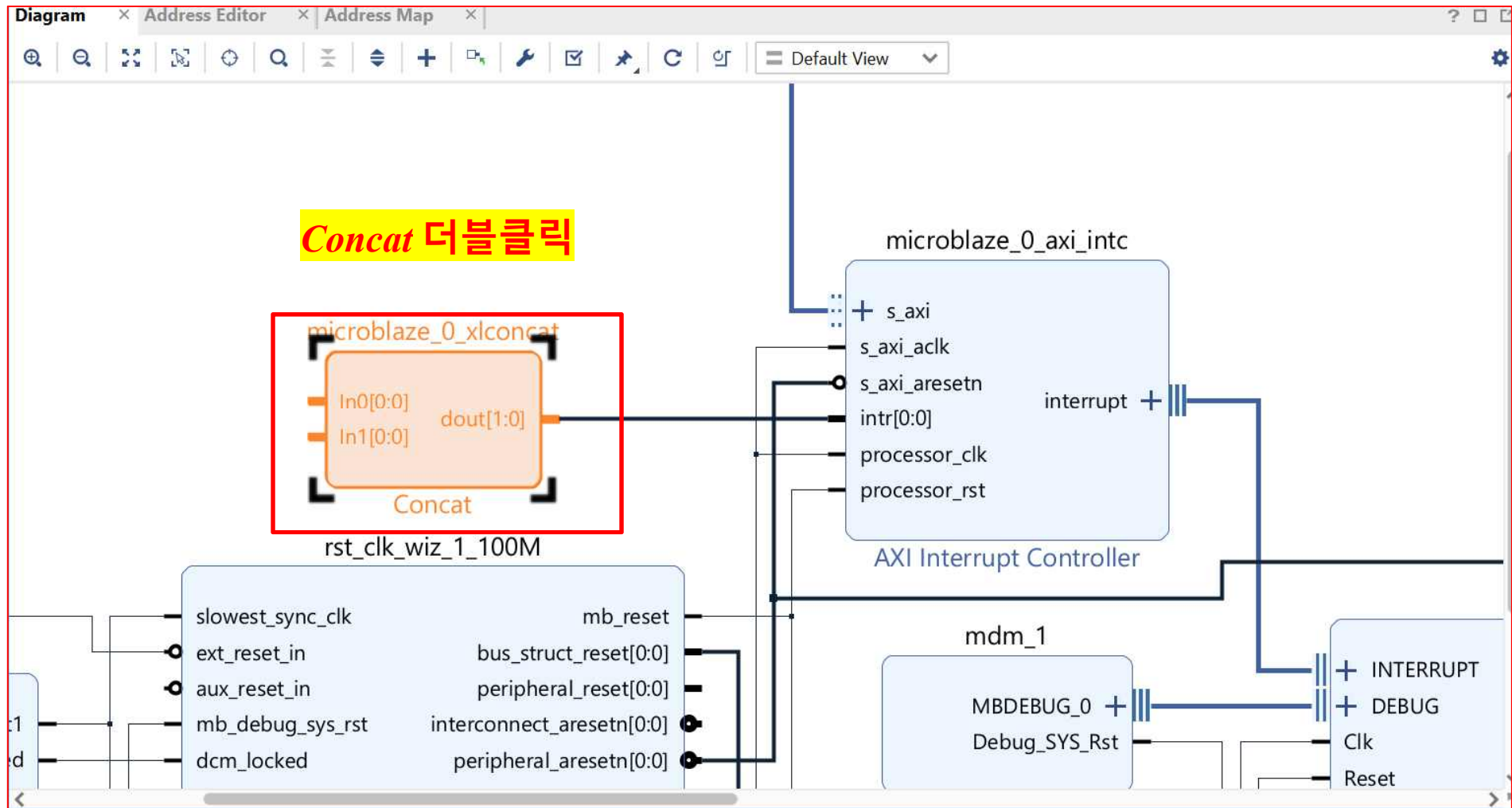
OK

Cancel

➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect → PWM Control

Concat (2.1)

Documentation IP Location

☐ Show disabled ports

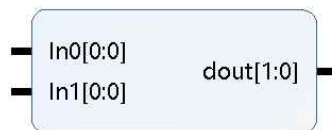
Component Name microblaze_0_xlconcat

Number of Ports 1 [1 - 128]

AUTO In0 Width 1 [1 - 4096]

AUTO In1 Width 1 [1 - 4096]

인터럽트를 하나만
사용하므로 (GPIO)
1로 변경 후 ok



Dout Width (Auto) 2

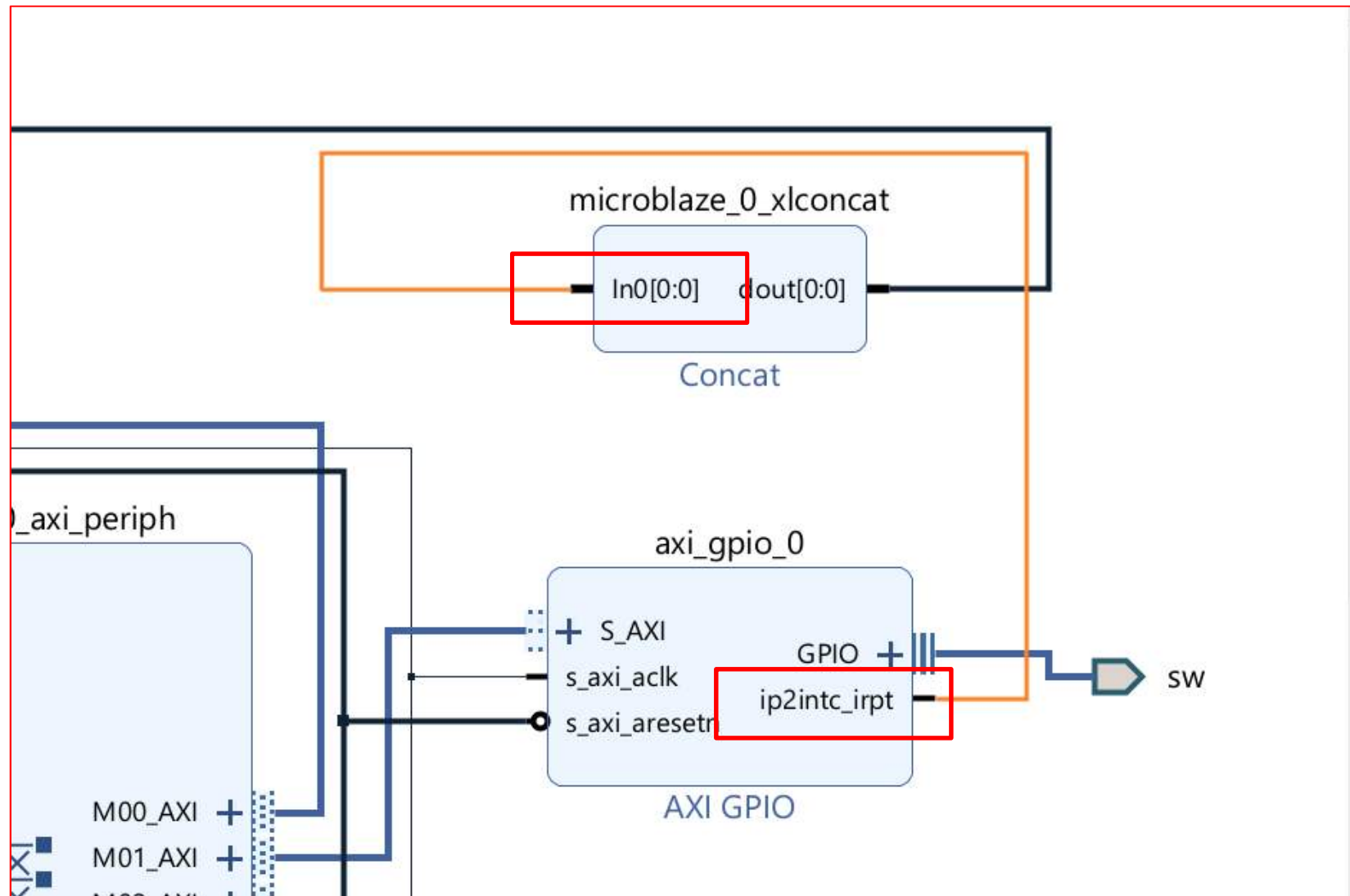
NOTE: The In0 port is connected to the LSB bits of the output, and the In[Number of Ports - 1] input port is connected to the MSB bits of the output.

변경 후 OK

OK

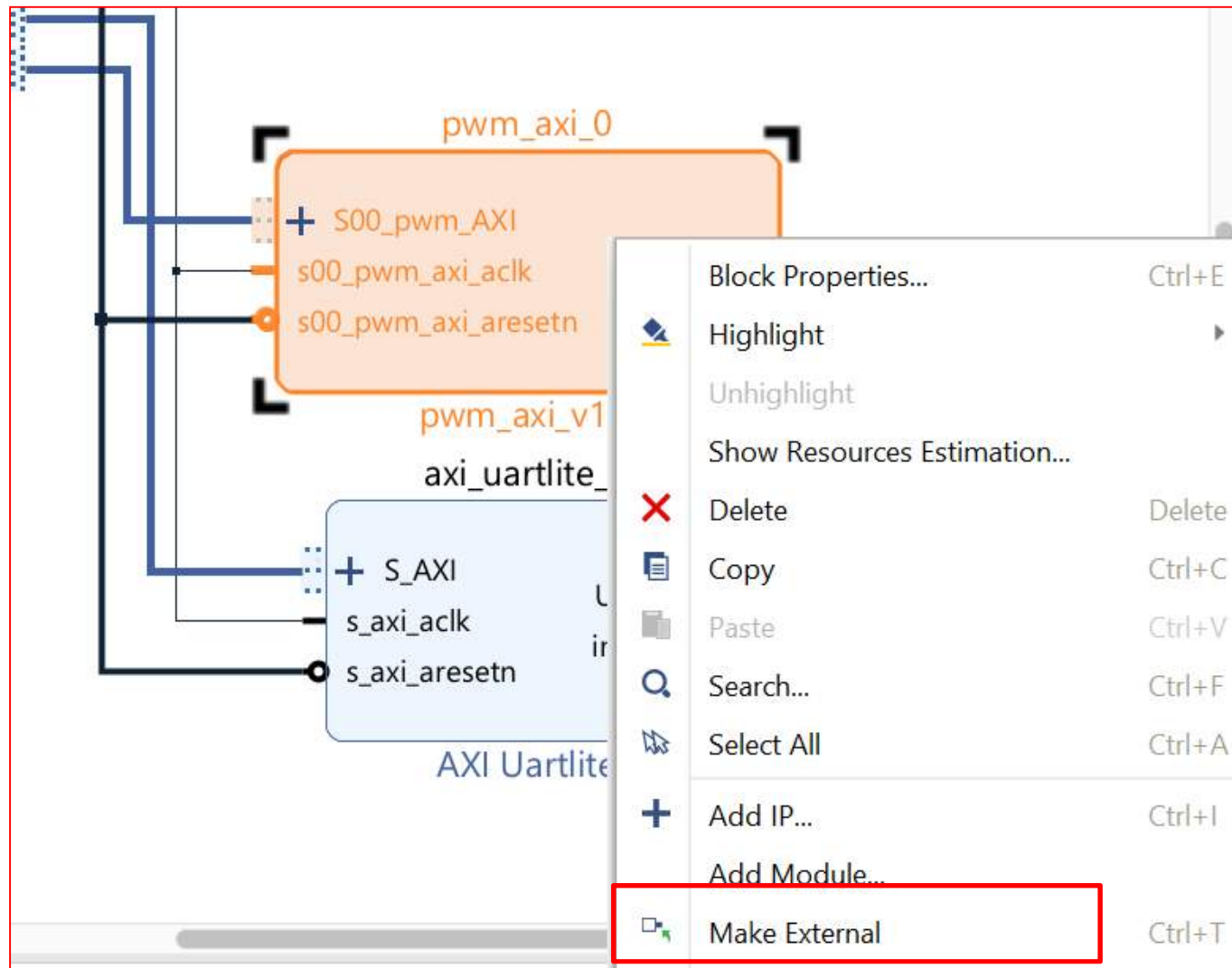
Cancel

➤ AXI Connect ➔ PWM Control



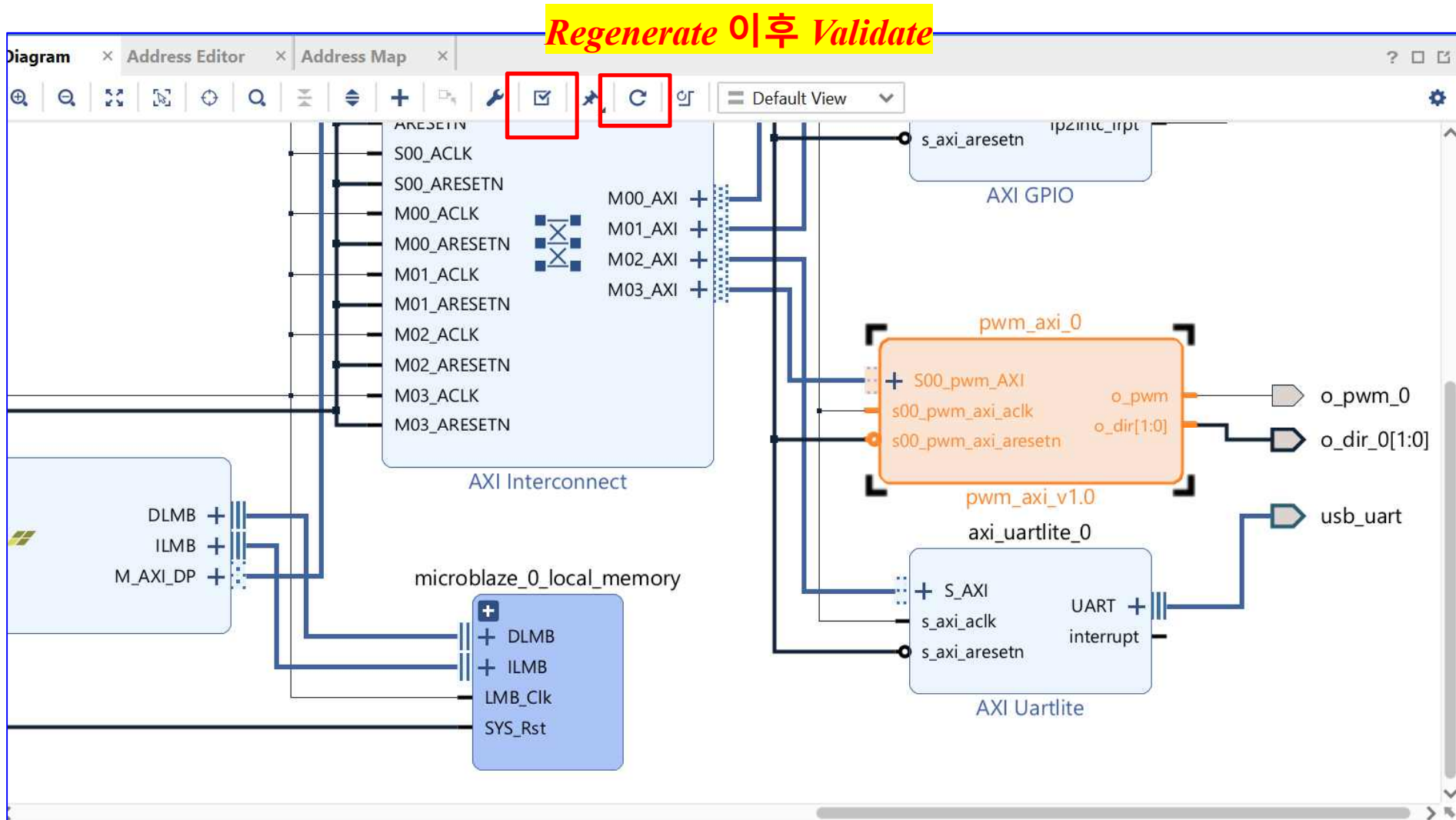
Concat과 GPIO의 인터럽트 연결

➤ AXI Connect ➔ PWM Control

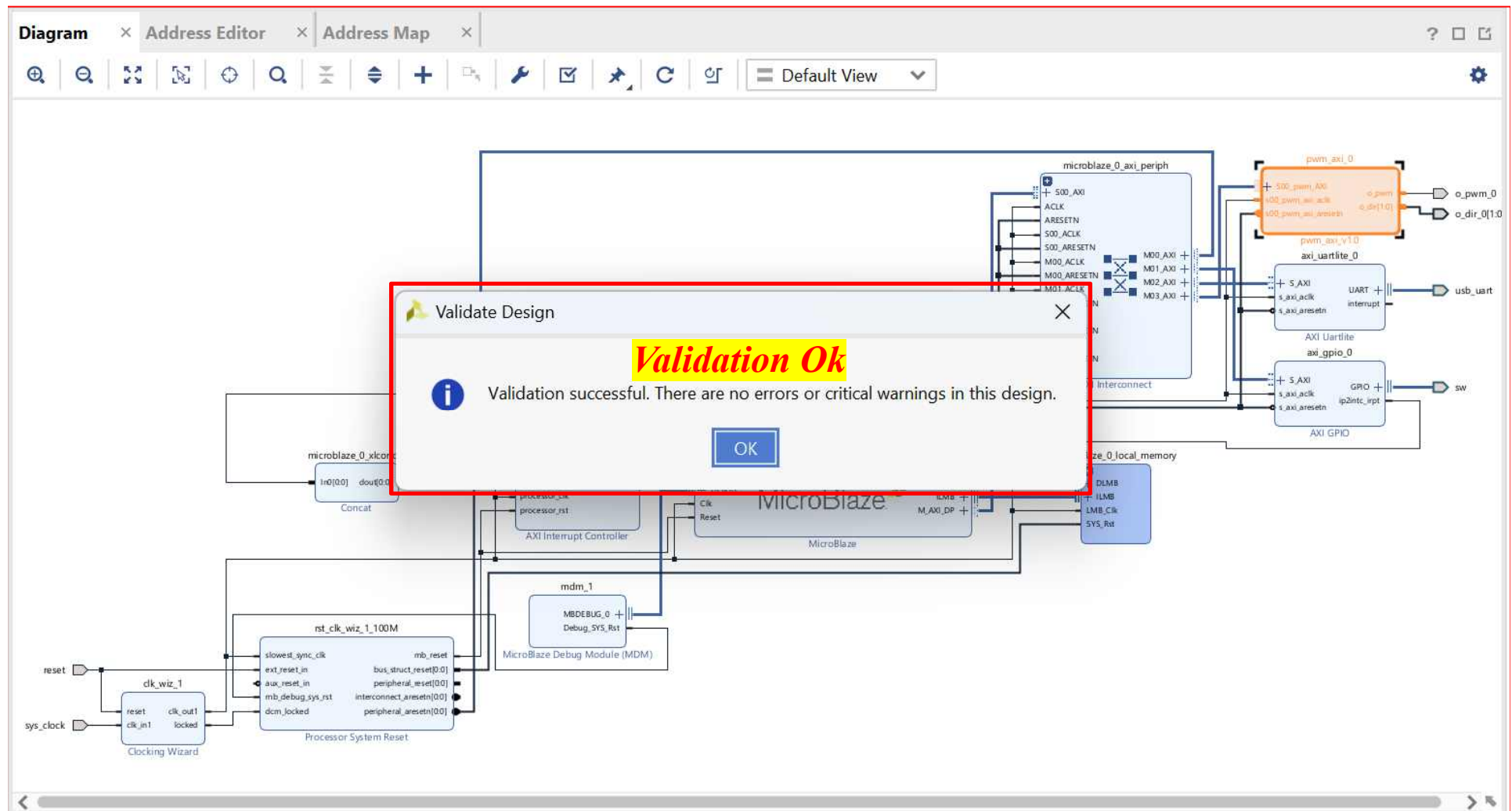


PWM 우클릭 후 Make External

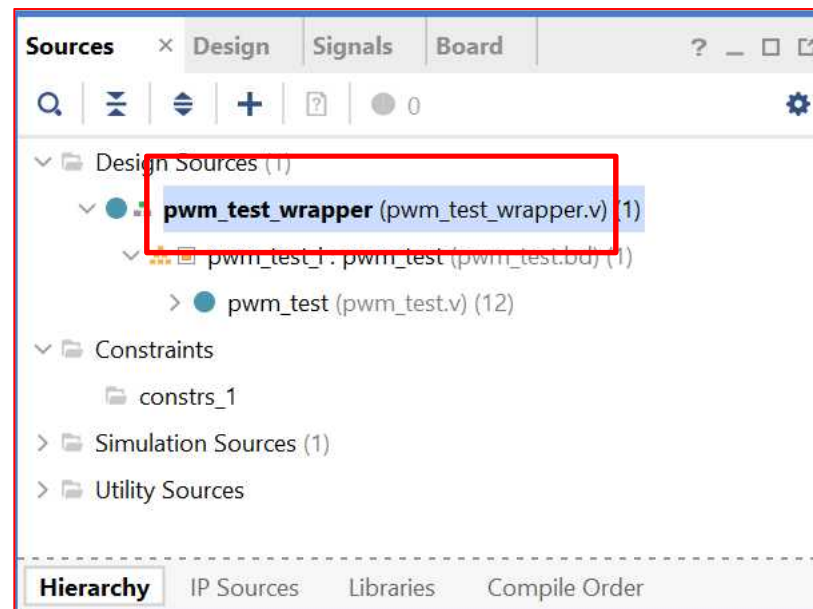
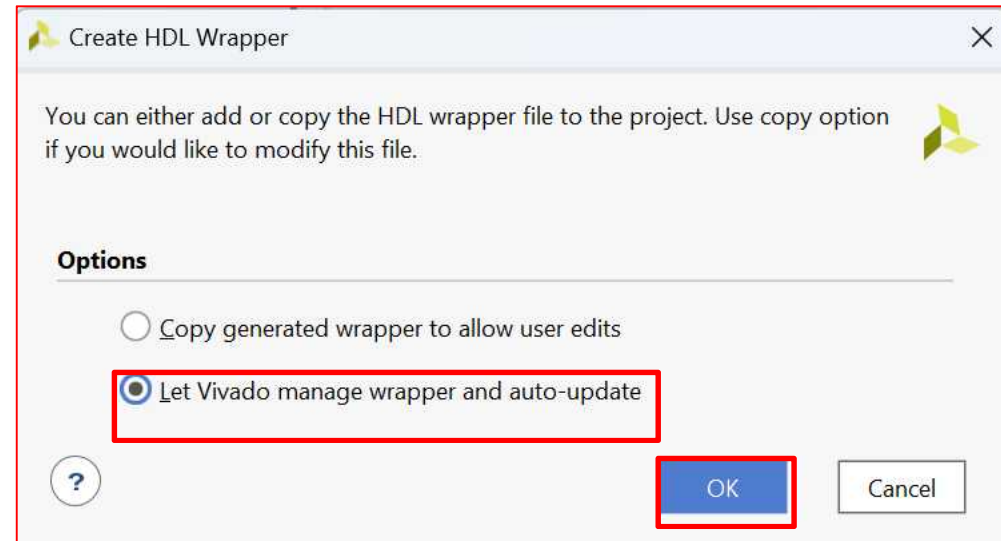
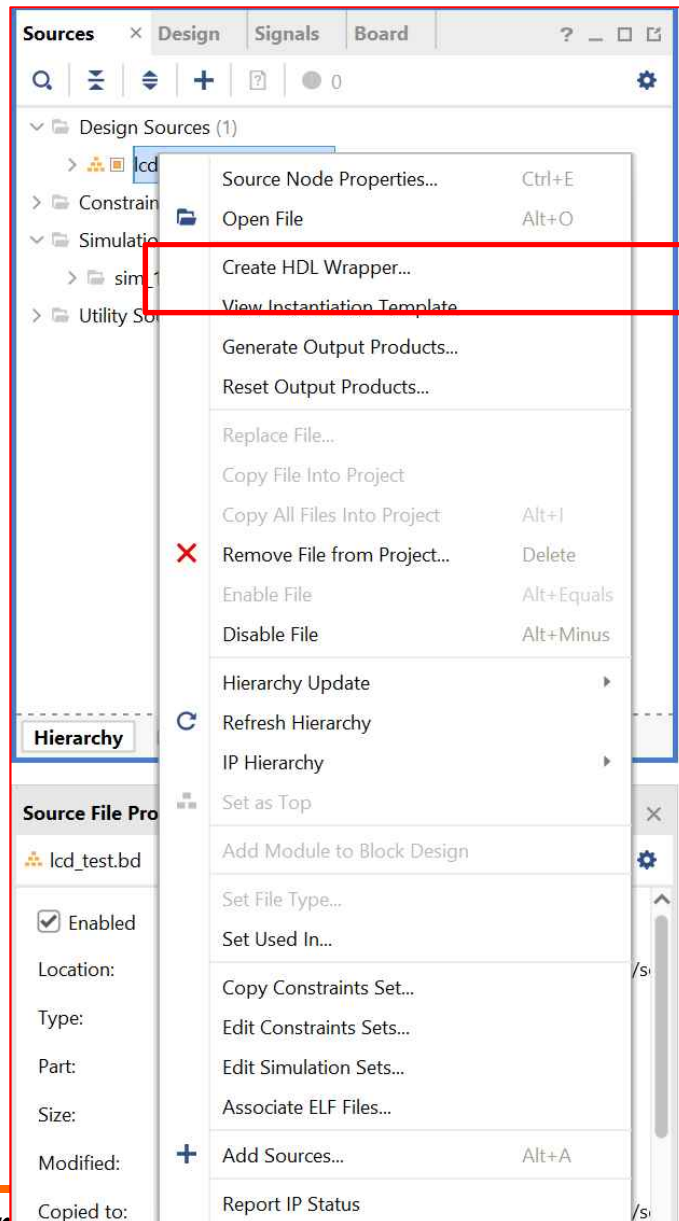
➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control

The screenshot displays the Xilinx Vivado IDE interface. The 'Sources' pane on the left shows the project hierarchy for 'pwm_test_wrapper'. A right-click context menu is open over the 'pwm_test_wrapper' source, with the 'Add Sources...' option highlighted at the bottom. The menu includes options like 'Source Node Properties...', 'Open File', 'Replace File...', 'Copy File Into Project', 'Copy All Files Into Project', 'Remove File from Project...', 'Enable File', 'Disable File', 'Move to Simulation Sources', 'Move to Design Sources', 'Hierarchy Update', 'Refresh Hierarchy', 'IP Hierarchy', 'Set as Top', 'Set Global Include', 'Clear Global Include', 'Add Module to Block Design', 'Set as Out-of-Context for Synthesis...', 'Set Library...', 'Set File Type...', 'Set Used In...', 'Copy Constraints Set...', 'Edit Constraints Sets...', 'Edit Simulation Sets...', and 'Report IP Status'. The 'Add Sources...' option is also labeled with the keyboard shortcut 'Alt+A'.

The 'Diagram' pane on the right shows the 'Address Map' for 'pwm_test_wrapper.v'. It displays the copyright information for Xilinx, Inc. (1986-2022) and the tool version (Vivado v.2022.2). The address map includes the following information:

- //Tool Version: Vivado v.2022.2 (win64) Build 3671981 Fri Oct 14 05
- //Date : Tue Nov 7 15:42:51 2023
- //Host : DESKTOP-10ETN9F running 64-bit major release (buil
- //Command : generate_target pwm_test_wrapper.bd
- //Design : pwm_test_wrapper
- //Purpose : IP block netlist

The address map also shows the timescale and the module definition for 'pwm_test_wrapper'.

```
timescale 1 ps / 1 ps

module pwm_test_wrapper
    (o_dir_0,
     o_pwm_0,
     reset,
     sw_tri_i,
     sys_clock,
     usb_uart_rxd,
     usb_uart_txd);
    output [1:0] o_dir_0;
    output o_pwm_0;
    input reset;
    input [3:0] sw_tri_i;
    input sys_clock;
```

Xdc 파일 추가

AMBA and AXI Connect

➤ AXI Connect ➔ PWM Control

Basys-3-Master.xdc

axi_pwm_exam.xpr

```
1.  ## This file is a general .xdc for the Basys3 rev B board
2.  ## To use it in a project:
3.  ## - uncomment the lines corresponding to used pins
4.  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the
    project

5.  ## Clock signal
6.  set_property -dict { PACKAGE_PIN W5  IOSTANDARD LVCMOS33 } [get_ports sys_clock]
7.  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports sys_clock]

8.  ## Switches
9.  set_property -dict { PACKAGE_PIN V17  IOSTANDARD LVCMOS33 } [get_ports {sw_tri_i[0]}]
10. set_property -dict { PACKAGE_PIN R2   IOSTANDARD LVCMOS33 } [get_ports {reset}]

11. ##Buttons
12. set_property -dict { PACKAGE_PIN U18  IOSTANDARD LVCMOS33 } [get_ports {sw_tri_i[3]}]
13. set_property -dict { PACKAGE_PIN W19  IOSTANDARD LVCMOS33 } [get_ports {sw_tri_i[2]}]
14. set_property -dict { PACKAGE_PIN T17  IOSTANDARD LVCMOS33 } [get_ports {sw_tri_i[1]}]

15. ##Pmod Header JC
16. set_property -dict { PACKAGE_PIN K17  IOSTANDARD LVCMOS33 } [get_ports {o_dir_0[0]}];#Sch
    name = JC1
17. set_property -dict { PACKAGE_PIN M18  IOSTANDARD LVCMOS33 } [get_ports {o_dir_0[1]}];#Sch
    name = JC2
18. set_property -dict { PACKAGE_PIN N17  IOSTANDARD LVCMOS33 } [get_ports {o_pwm_0}];#Sch
    name = JC3

19. ##USB-RS232 Interface
20. set_property -dict { PACKAGE_PIN B18  IOSTANDARD LVCMOS33 } [get_ports usb_uart_rxd]
21. set_property -dict { PACKAGE_PIN A18  IOSTANDARD LVCMOS33 } [get_ports usb_uart_txd]

22. ## Configuration options, can be used for all designs
23. set_property CONFIG_VOLTAGE 3.3 [current_design]
24. set_property CFGBVS VCCO [current_design]

25. ## SPI configuration mode options for QSPI boot, can be used for all designs
26. set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
27. set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
28. set_property CONFIG_MODE SPIx4 [current_design]
```

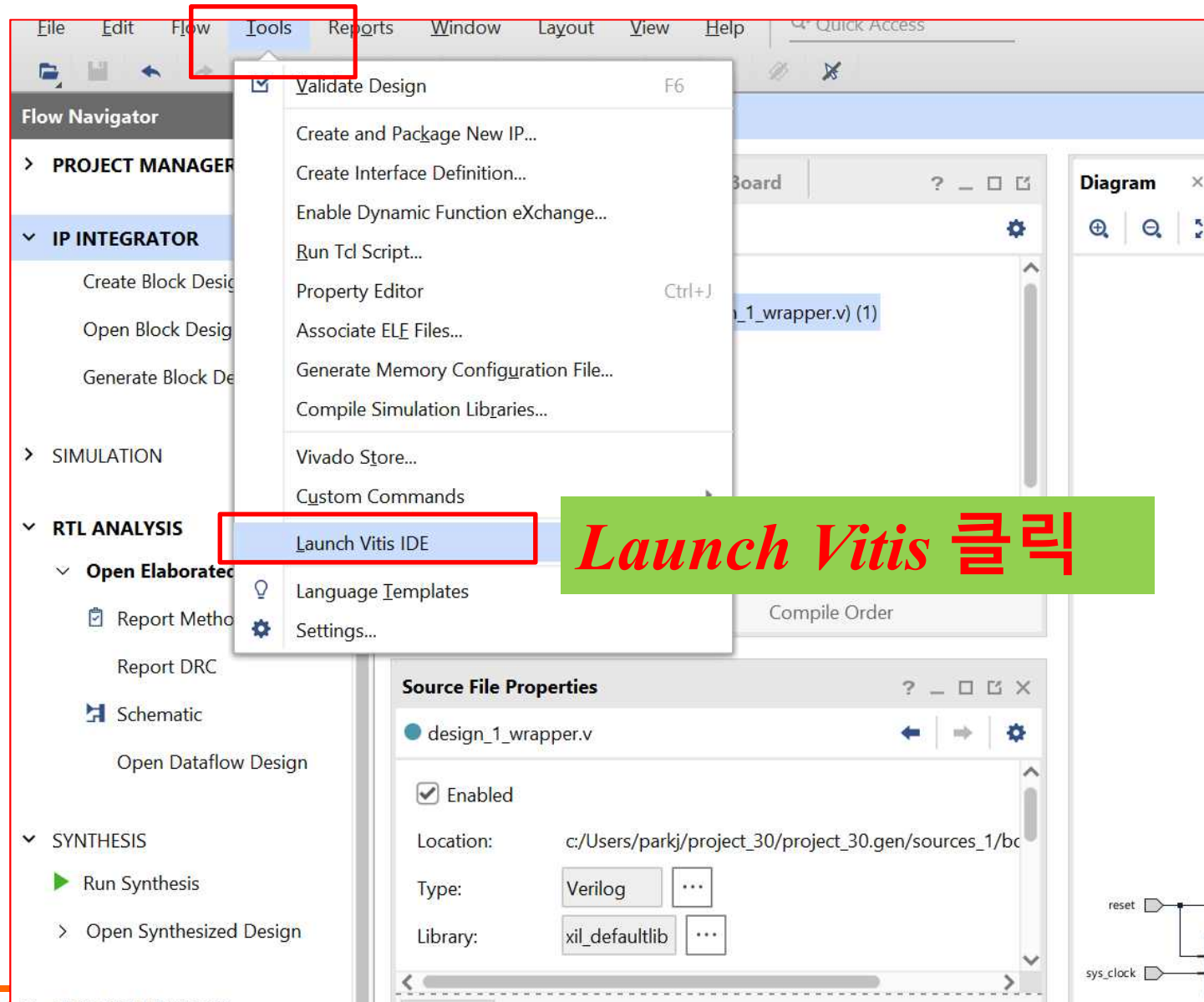
➤ AXI Connect ➔ PWM Control

- Create Block Design
- Blaze Uart & MyIP 추가 및 연결
- sys_clock & reset 설정
- MyIP Make External
- XDC Constraints 설정
- Bitstream
- Export Hardwar

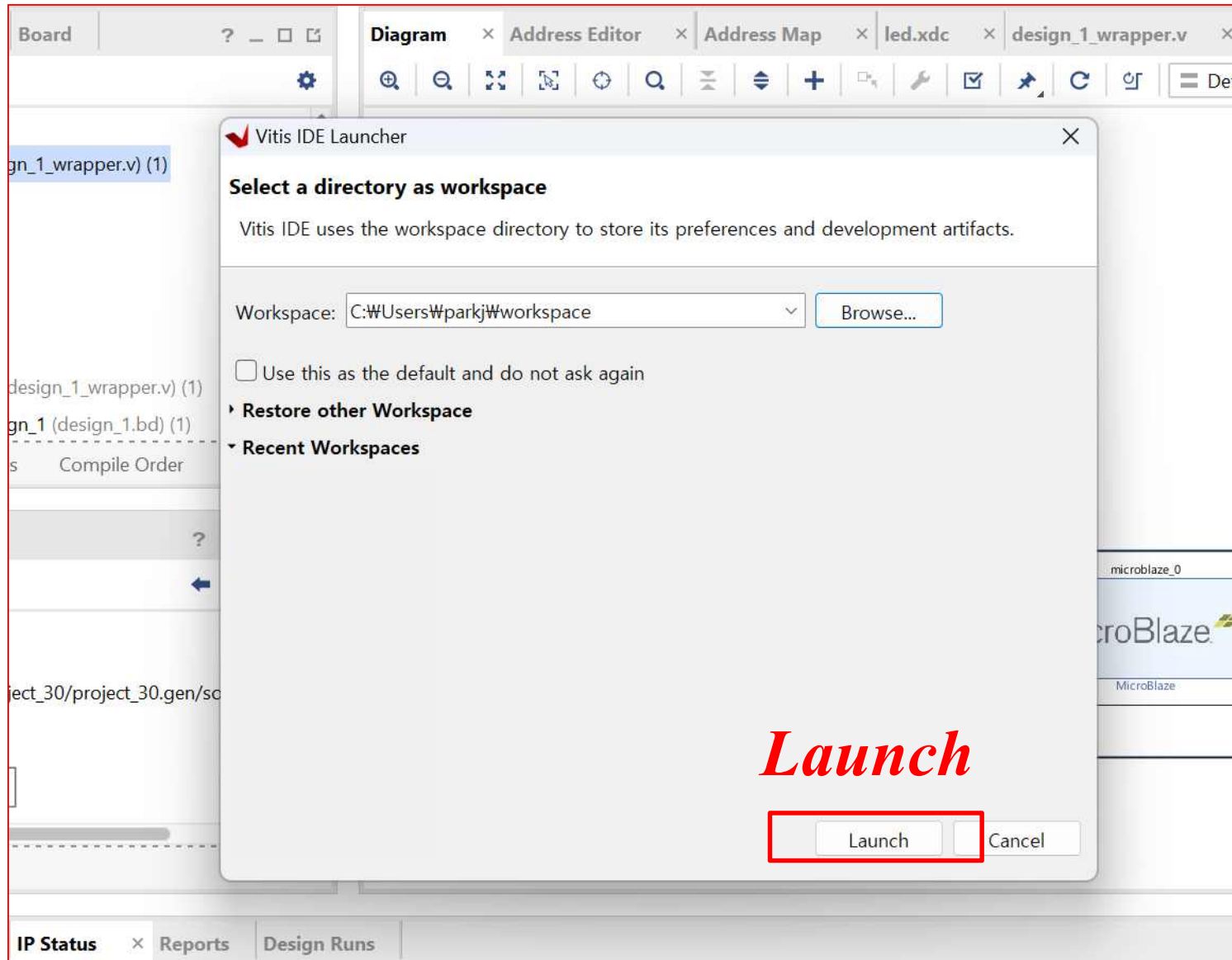
AMBA and AXI Connect

✓ *Vitis Application SW*

➤ AXI Connect ➔ PWM Control

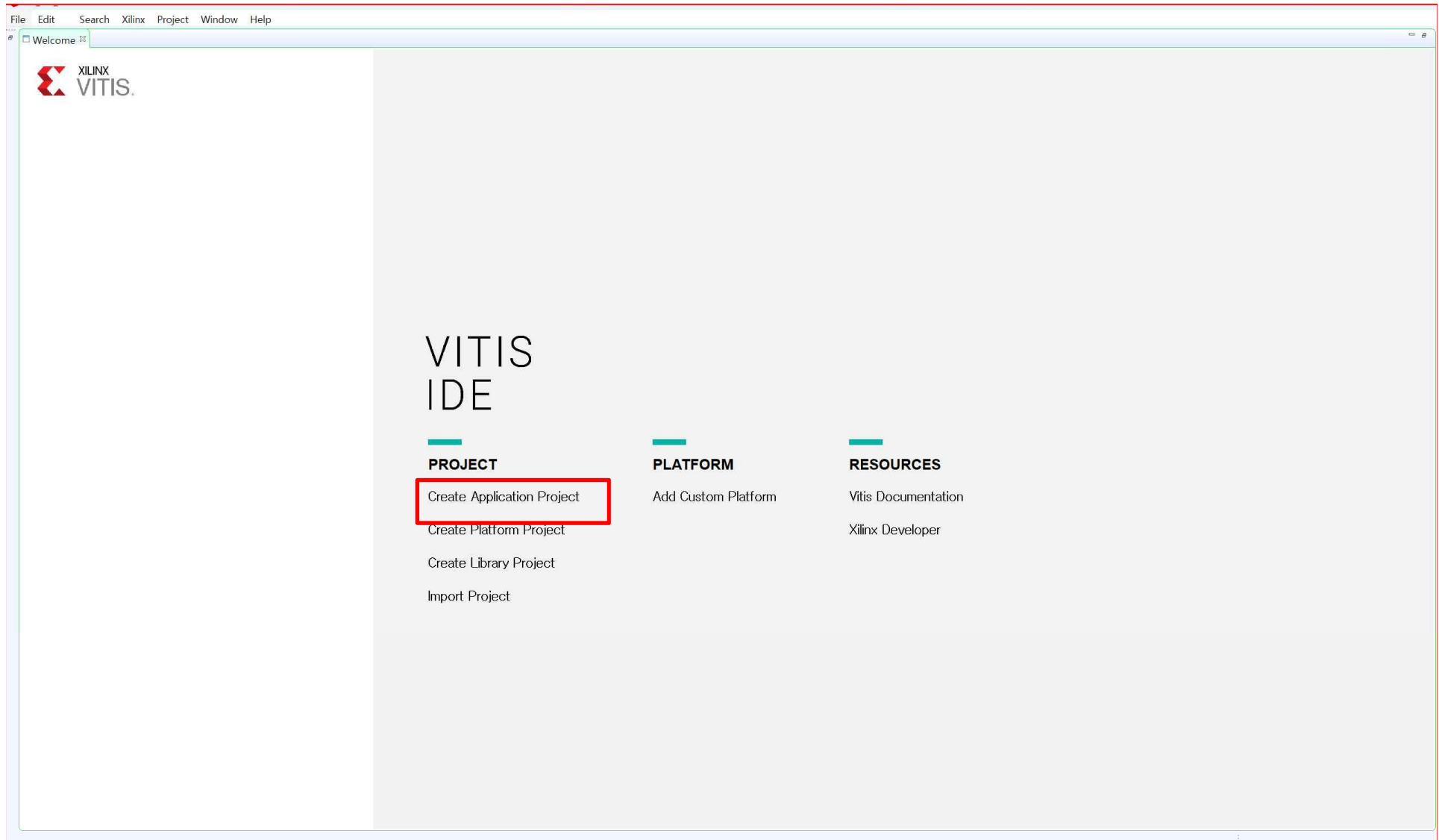


➤ AXI Connect ➔ PWM Control

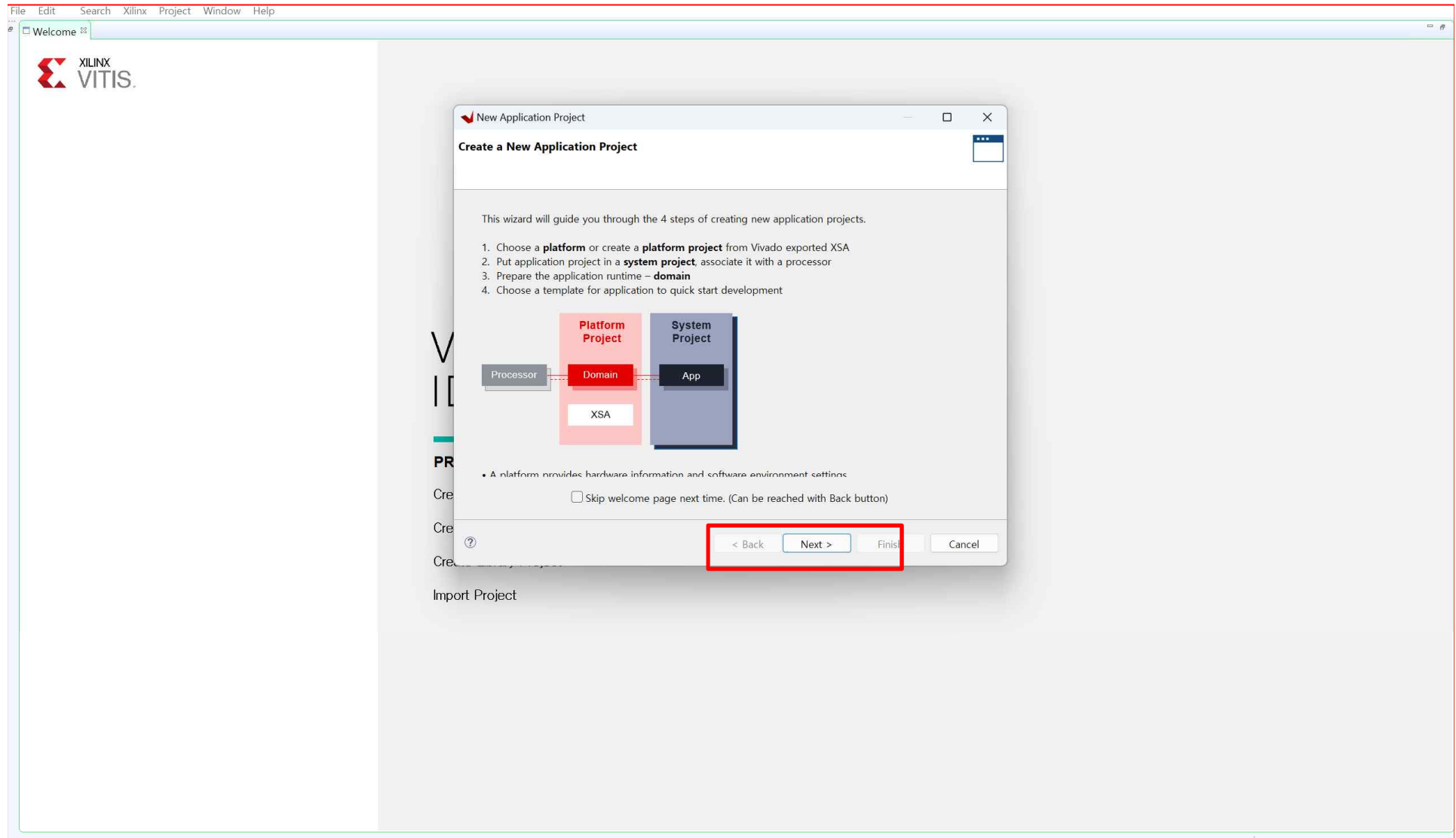


Launch

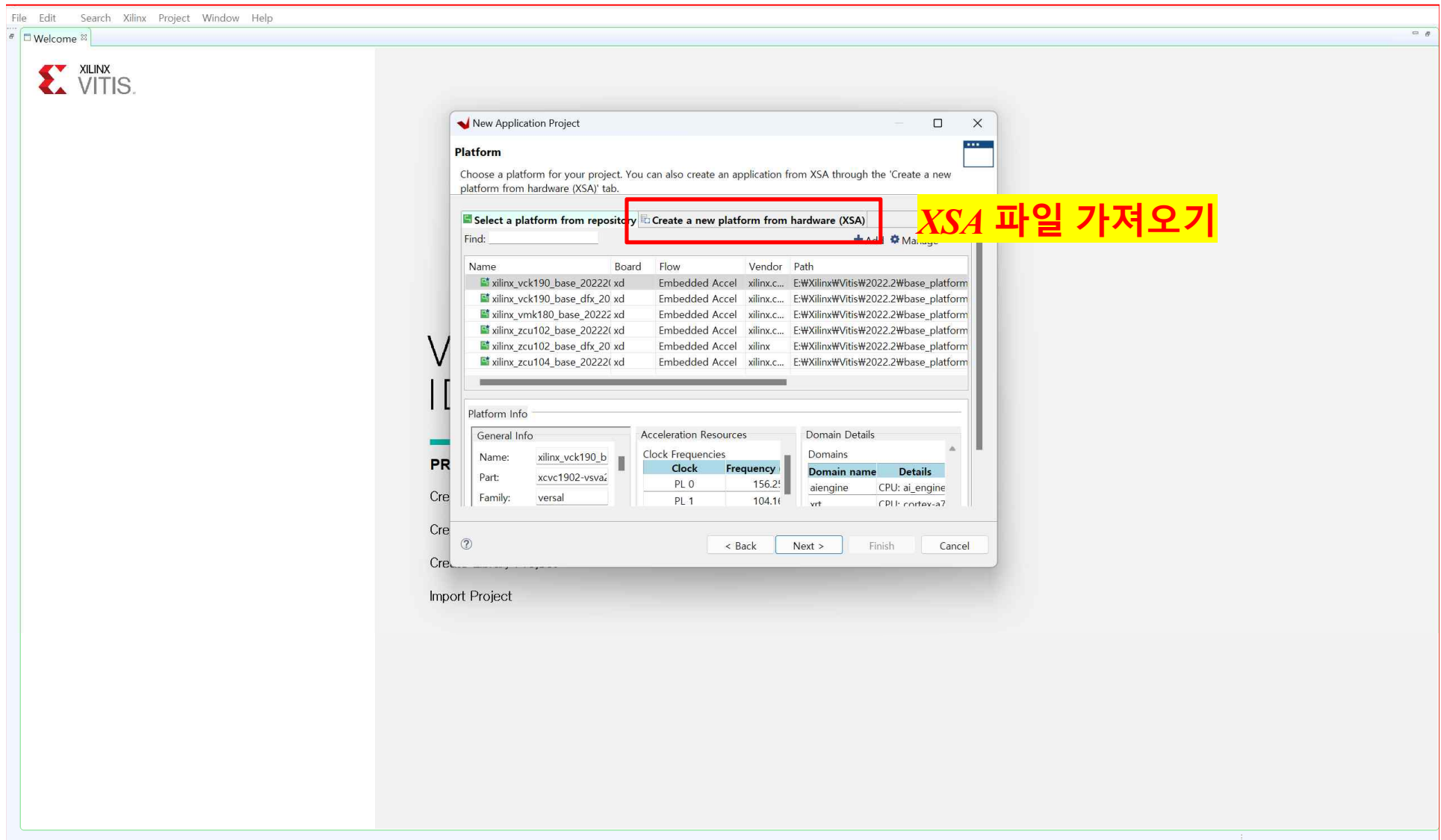
➤ AXI Connect ➔ PWM Control



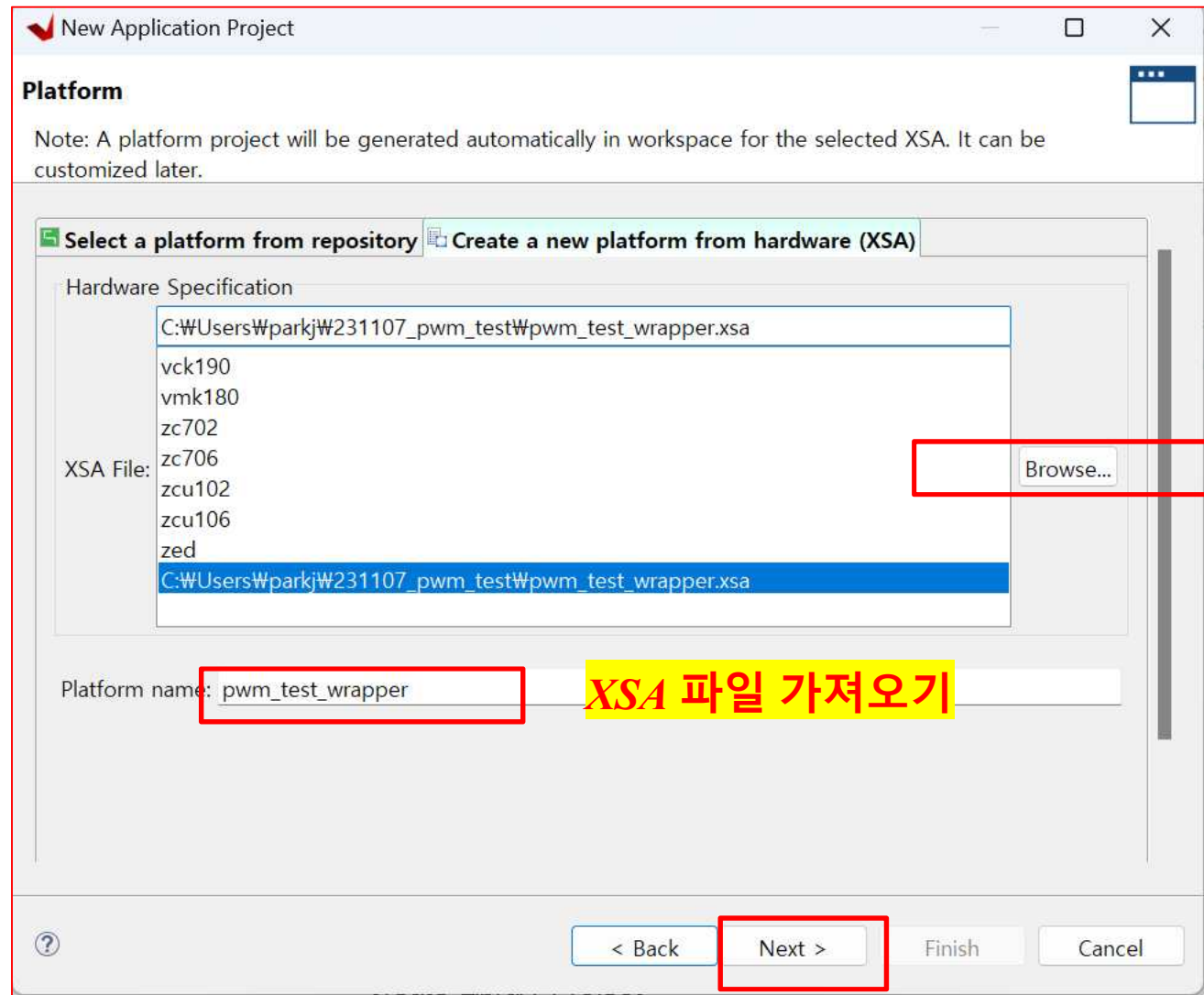
➤ AXI Connect ➔ PWM Control



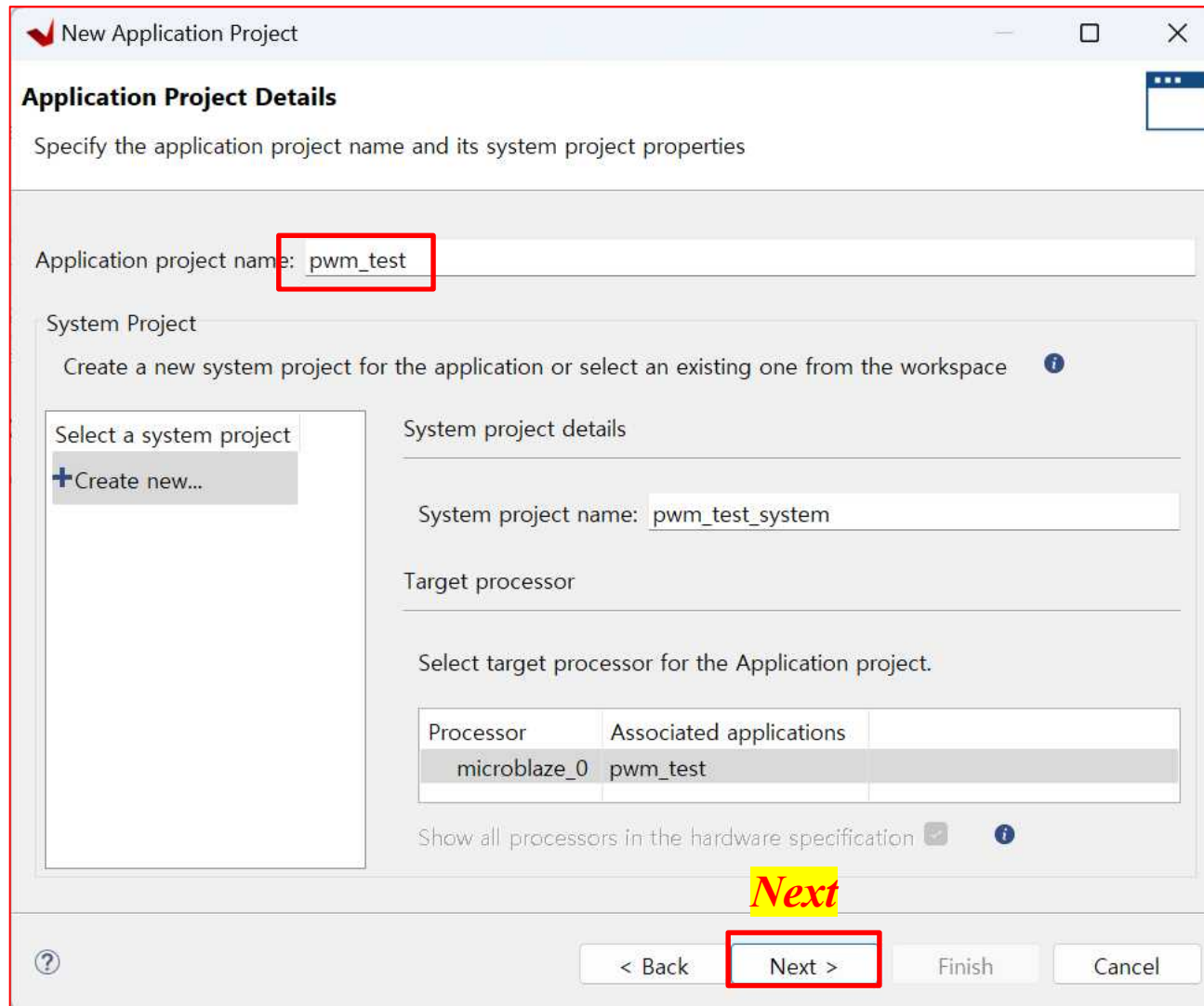
➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



The image shows a 'New Application Project' dialog box. The 'Application project name' field is set to 'pwm_test'. The 'System Project' section shows a list of system projects with 'pwm_test_system' selected. The 'Target processor' section shows a table with 'microblaze_0' and 'pwm_test' selected. The 'Next >' button is highlighted with a red box and the word 'Next' is written in red above it.

New Application Project

Application Project Details
Specify the application project name and its system project properties

Application project name:

System Project
Create a new system project for the application or select an existing one from the workspace

Select a system project
+ Create new...

System project details

System project name:

Target processor

Select target processor for the Application project.

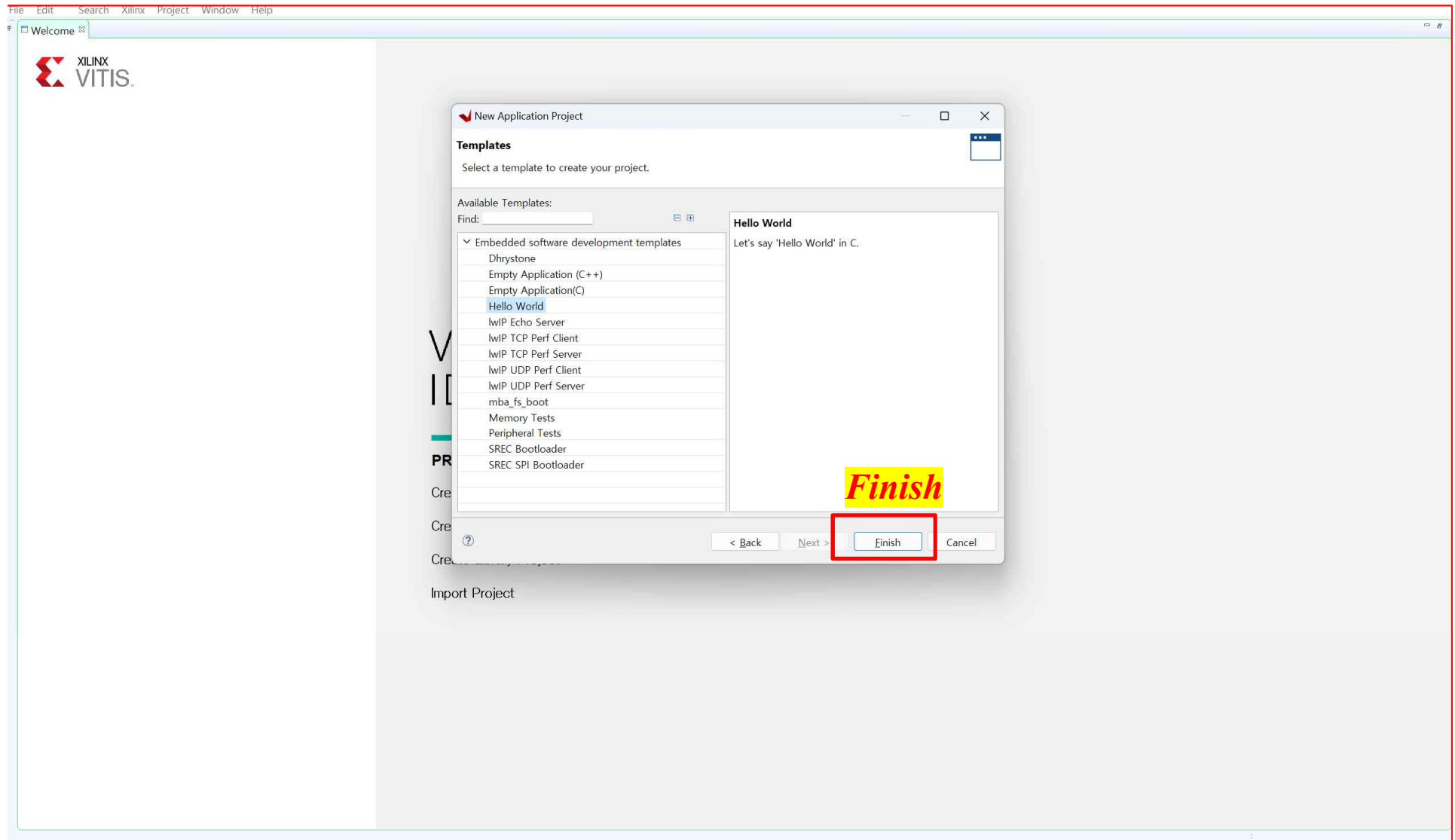
Processor	Associated applications
microblaze_0	pwm_test

Show all processors in the hardware specification ☒

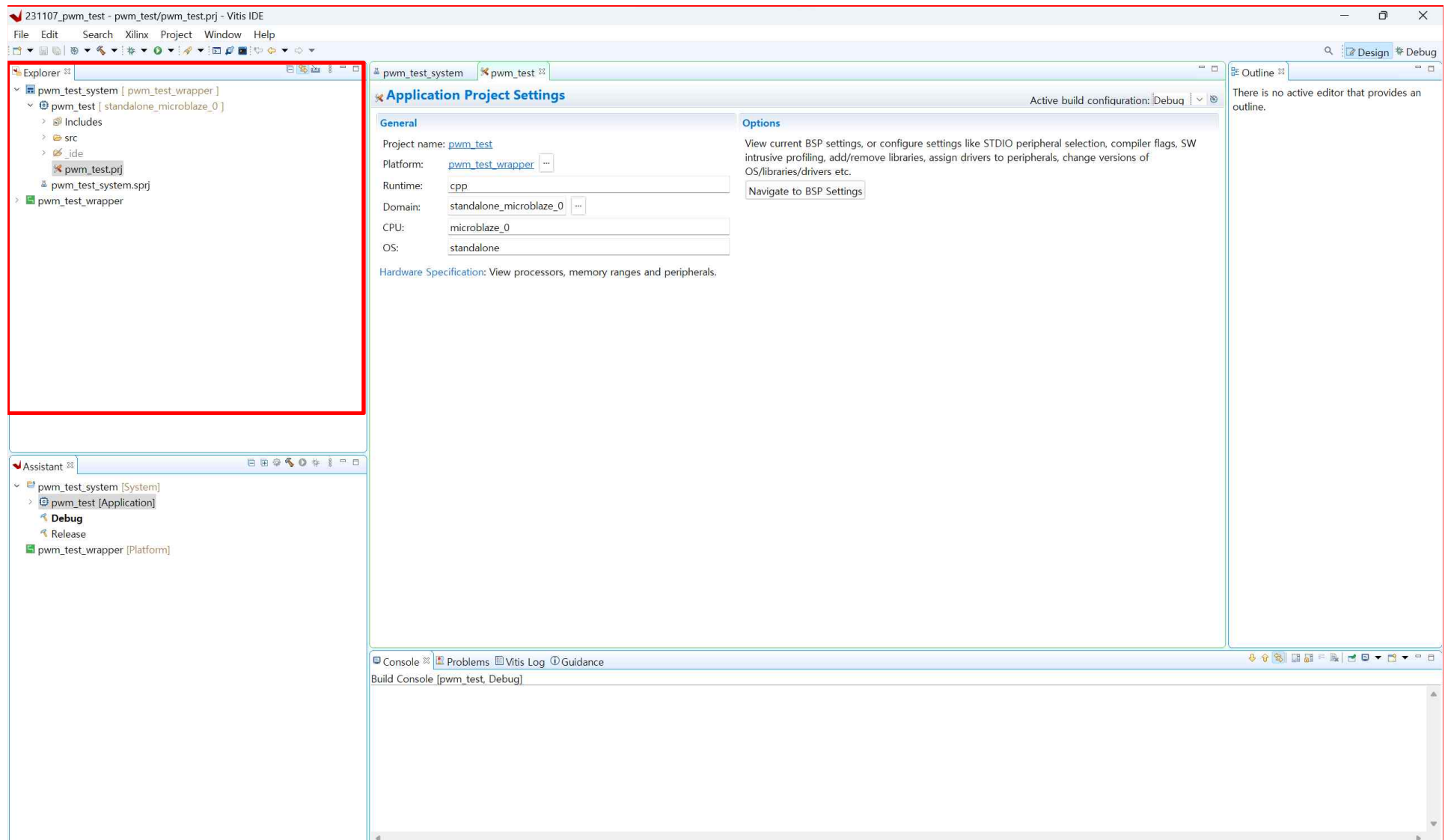
Next

< Back **Next >** Finish Cancel

➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control



➤ AXI Connect ➔ PWM Control

BASEADDR 확인

The screenshot shows a project explorer on the left and a code editor on the right. The project explorer displays the hierarchy of the project, with the following items highlighted by red boxes:

- pwm_test_system [pwm_test_wrapper]** (Project Root)
- pwm_test [standalone_microblaze_0]** (Sub-project)
- pwm_test_wrapper** (Wrapper)
- microblaze_0** (Microblaze Instance)
- standalone_microblaze_0** (Sub-instance)
- bsp** (Board Support Package)
- microblaze_0** (Sub-bsp)
- include** (Include Directory)
- xparameters.h** (Header File)

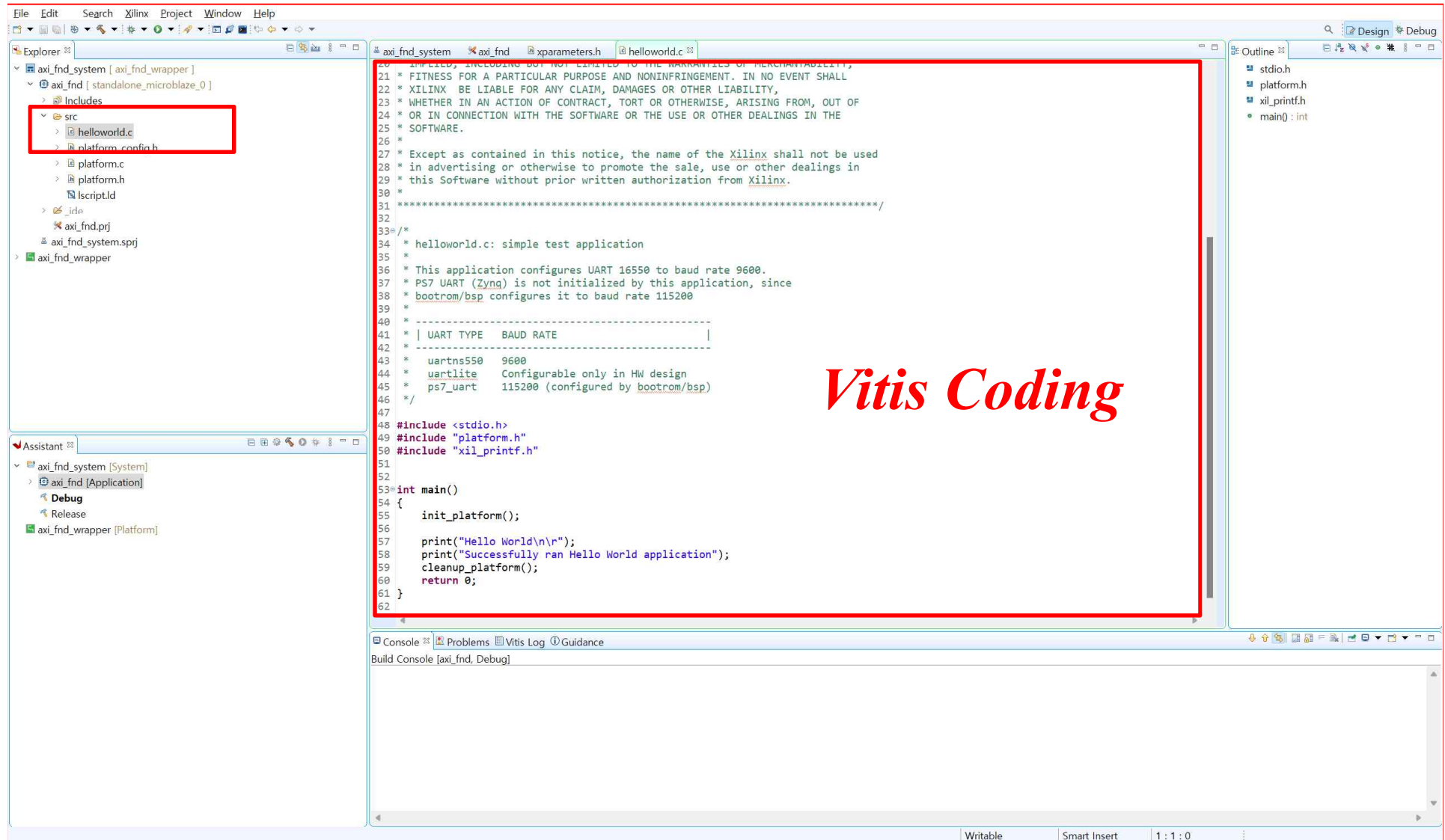
The code editor displays the contents of **xparameters.h**, with the following lines highlighted by a red box:

```

564
565
566 /* Peripheral Definitions for peripheral PWM_AXI_V1_0_0 */
567 #define XPAR_PWM_AXI_V1_0_0_BASEADDR 0x00010000
568 #define XPAR_PWM_AXI_V1_0_0_HIGHADDR 0x00010FFF
569
570
571 /*****
572
573
574 /* Canonical Definitions for peripheral PWM_AXI_V1_0_0 */
575 #define XPAR_PWM_AXI_V1_0_BASEADDR 0x00010000
576 #define XPAR_PWM_AXI_V1_0_HIGHADDR 0x00010FFF
577
578 /*****
579
580
581 /* Definitions for driver GPIO */
582 #define XPAR_XGPIO_NUM_INSTANCES 1
583
584 /* Definitions for peripheral AXI_GPIO_0 */
585 #define XPAR_AXI_GPIO_0_BASEADDR 0x40000000
586 #define XPAR_AXI_GPIO_0_HIGHADDR 0x4000FFFF
587 #define XPAR_AXI_GPIO_0_DEVICE_ID 0

```

➤ AXI Connect ➔ PWM Control



Vitis Coding

➤ AXI Connect ➔ PWM Control

Helloworld.c (1)

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xgpio.h"
#include "sleep.h"
#include "xil_exception.h"
#include "xintc.h"
```

주소 확인

```
#define AXI_PWM0_BASEADDR 0x44A00000
#define PWM0_enable AXI_PWM0_BASEADDR + 0x00
#define PWM0_dir AXI_PWM0_BASEADDR + 0x04
#define PWM0_ocr AXI_PWM0_BASEADDR + 0x08

/* definitions for btn0, AXI_GPIO_1 */
#define GPIO_BTN0_DEVICE_ID XPAR_GPIO_0_DEVICE_ID
#define BTN0_CHANNEL1 1
#define BTN0_INT_MSK XGPIO_IR_CH1_MASK /* Channel 1
Interrupt Mask */

/* definitions for Interrupt */
#define INTC_DEVICE_ID XPAR_INTC_0_DEVICE_ID
#define INTC_BTN0_INT_VEC_ID XPAR_INTC_0_GPIO_0_VEC_ID

XGpio GPIO_BTN; /* The instance of the btn */
XIntc Intc;
```

PWM IP 참고

```
assign i_en = slv_reg0;
assign i_dir = slv_reg1;
assign i_ocr = slv_reg2;
// User logic ends
```

Helloworld.c (2)

```
void Gpio1Handler(void *CallBackRef);
```

```
void interrupt_init(void)
{
```

```
    XIntc_Initialize(&Intc, INTC_DEVICE_ID);
    XIntc_Connect(&Intc, INTC_BTN0_INT_VEC_ID,
(XInterruptHandler)Gpio1Handler, &GPIO_BTN);
```

```
    XIntc_Start(&Intc, XIN_REAL_MODE);
    XIntc_Enable(&Intc, INTC_BTN0_INT_VEC_ID);
}
```

```
void exception_init(void)
```

```
{
    Xil_ExceptionInit();
    Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
(Xil_ExceptionHandler)XIntc_InterruptHandler, &Intc);
    Xil_ExceptionEnable();
}
```

```
void gpio_init(void)
```

```
{
    XGpio_Initialize(&GPIO_BTN, GPIO_BTN0_DEVICE_ID);

    XGpio_InterruptEnable(&GPIO_BTN, BTN0_CHANNEL1);

    XGpio_InterruptGlobalEnable(&GPIO_BTN);
}
```

➤ AXI Connect ➔ PWM Control

Helloworld.c (3)

```
u8 direction=0b01;
u8 duty=0;
u8 RunFlag=0;

void Gpio1Handler(void *CallbackRef)
{
    XGpio *GpioPtr = (XGpio *)CallbackRef;

    if ((XGpio_DiscreteRead(&GPIO_BTN, BTN0_CHANNEL1) & (1<<1)))
    {
        if(duty>=80)
            duty=80;
        else
            duty=duty+20;
        Xil_Out32(PWM0_ocr,duty);
        usleep(300000);
    }

    ///
    if ((XGpio_DiscreteRead(&GPIO_BTN, BTN0_CHANNEL1) & (1<<2)))
    {
        if(duty==0)
            duty=0;
        else
            duty=duty-20;
        Xil_Out32(PWM0_ocr,duty);
        usleep(300000);
    }
}
```

- ❖ **u8 direction** : 모터의 방향 제어를 위한 변수
- ❖ **u8 duty** : 모터의 방향 제어를 위한 변수
- ❖ **u8 RunFlag** : 모터의 방향 제어를 위한 변수

❖ **GPIO SW1: 듀티비 증가**

- ✓ 스위치를 누를 때마다 듀티비가 20씩 증가
- ✓ 듀티비가 80이 되면 스위치를 눌러도 듀티비가 증가하지 않고 80으로 유지

❖ **GPIO SW2: 듀티비 감소**

- ✓ 스위치를 누를 때마다 듀티비가 20씩 감소
- ✓ 듀티비가 0이 되면 스위치를 눌러도 듀티비가 감소하지 않고 0으로 유지

➤ AXI Connect ➔ PWM Control

Helloworld.c (4)

```
if ((XGpio_DiscreteRead(&GPIO_BTN,  
BTN0_CHANNEL1) & (1<<3))) {  
  
    if (RunFlag==1)  
    {  
        RunFlag=0;  
        Xil_Out32(PWM0_enable,0);  
        Xil_Out32(PWM0_ocr,0);  
        usleep(300000);  
    }  
  
    else if (RunFlag==0)  
    {  
        RunFlag=1;  
        Xil_Out32(PWM0_enable,1);  
        Xil_Out32(PWM0_ocr,duty);  
        usleep(300000);  
    }  
  
    }  
  
    XGpio_InterruptClear(GpioPtr, BTN0_CHANNEL1);  
    /* Clear the Interrupt */  
}
```

❖ GPIO SW3: RUN/STOP

- ✓ RUN Flag가 1일때 (모터 동작상태)
➔ 모터를 정지시키고, RUN Flag=0

- ✓ RUN Flag가 0일때 (모터 정지상태)
➔ 모터를 동작시키고, RUN Flag=1

➤ AXI Connect ➔ PWM Control

Helloworld.c (5)

```
int main()
{
    init_platform();
    interrupt_init();
    gpio_init();
    exception_init();
    print("Hello World\n\r");
    print("PWM RUN \n\r");
    cleanup_platform();

    while(1){

        if ((XGpio_DiscreteRead(&GPIO_BTN, BTN0_CHANNEL1) & (1<<0)))
            Xil_Out32(PWM0_dir,0b10);
        else
            Xil_Out32(PWM0_dir,0b01);

        usleep(300000);

    }
    return 0;
}
```

❖ Main 함수

- ✓ 앞에서 정의한 init 함수들 실행.

❖ GPIO SW0: 정/역회전 제어

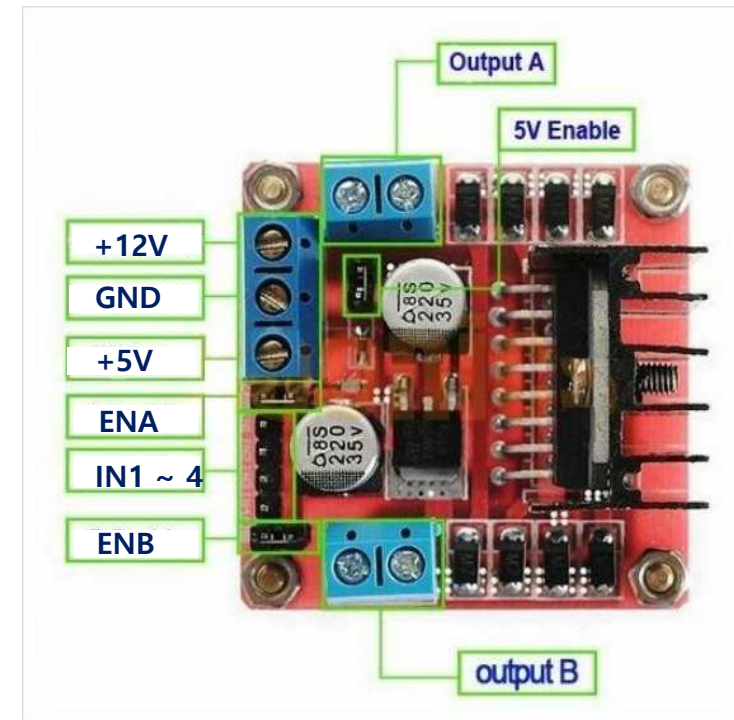
- ✓ SW0 토글시 모터드라이버 *IN1,IN2*에 신호 **10**을 인가
- ✓ 기본 상태에서는 모터드라이버 *IN1,IN2*에 신호 **01**을 인가

➤ 모터 구동을 위한 모듈 : **L298N**(모터 드라이버)[1/2]

- **L298N 기능** : 1. DC, 스텝 모터의 방향 및 속도 제어
 - 2. H-브리지 구조: 모터를 양방향으로 제어가능. 즉, 모터의 회전방향을 제어하거나 정지
 - 3. 두개의 모터를 독립적으로 제어. 두개의 출력 핀 제공(Output A, Output B)
 - 4. 주로 PWM신호를 사용하여 모터 속도를 조절

▪ **L298N** 핀 구성 :

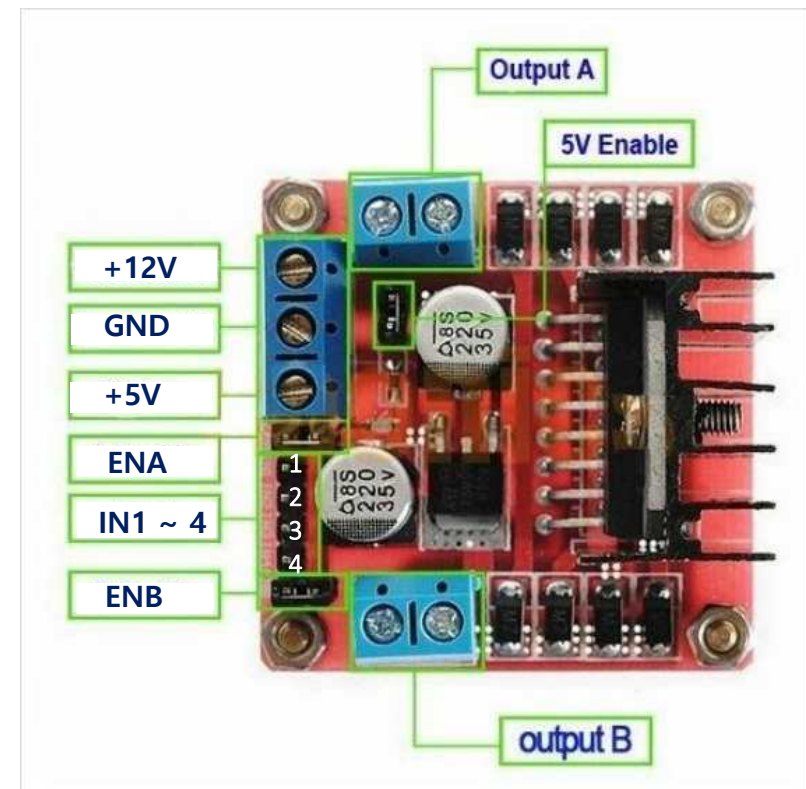
1. **IN1, IN2** : 첫 번째 모터의 회전 방향 제어
2. **IN3, IN4** : 두 번째 모터의 회전 방향 제어
3. **OUTA** : 첫 번째 모터의 출력 핀(모터와 연결)
4. **OUTB** : 두 번째 모터의 출력 핀(모터와 연결)
5. **ENA** : 첫 번째 모터의 속도 제어를 위한 PWM 입력 핀
6. **ENB** : 두 번째 모터의 속도 제어를 위한 PWM 입력 핀
7. **+12V** : 모터 전원 입력(+5V ~ 35V)
8. **+5V** : 내부 논리 회로 전원 입력 (+5V)
9. **GND** : 공통 접지



➤ 모터 구동을 위한 모듈 : **L298N**(모터 드라이버)[2/2]

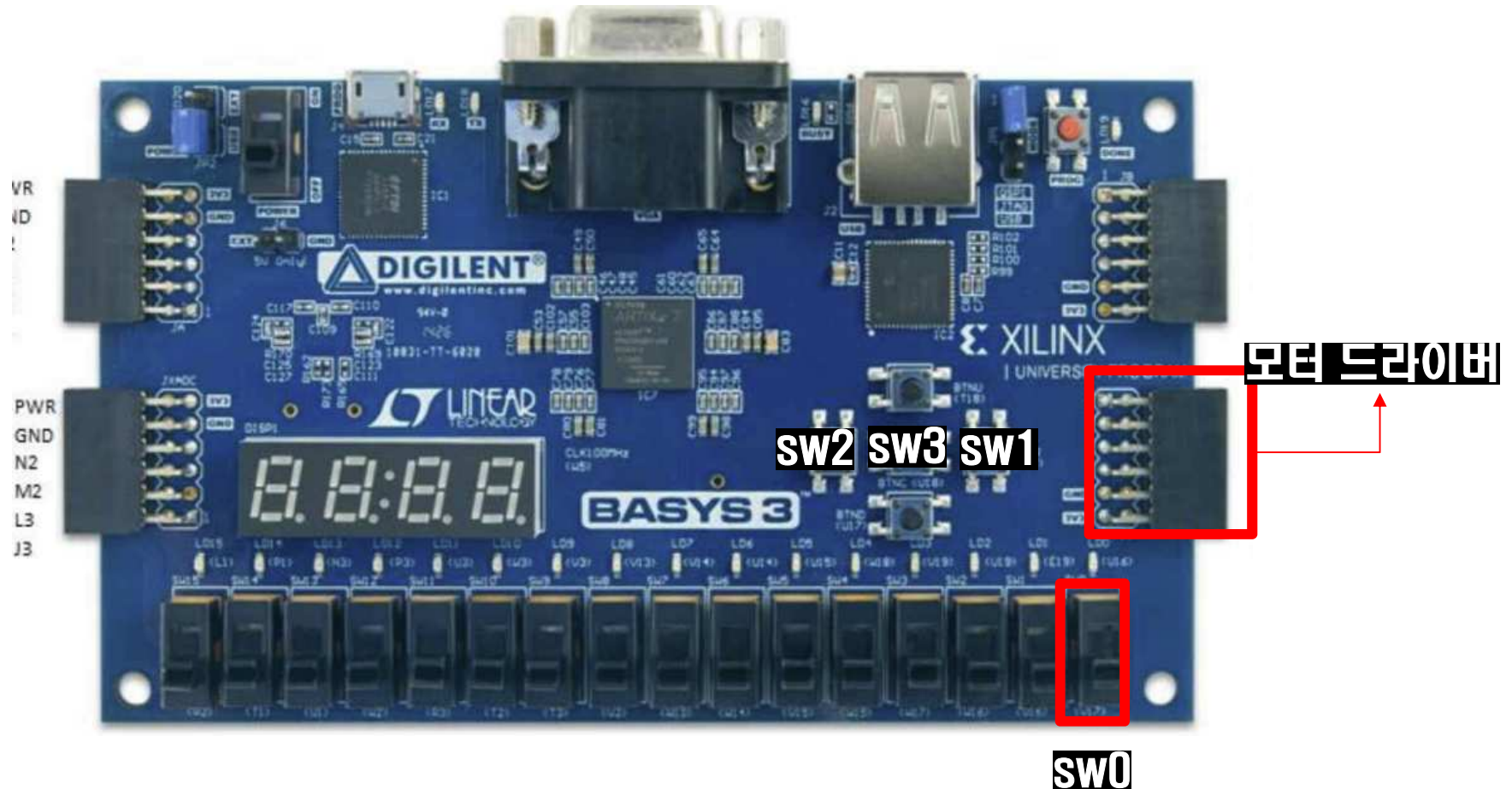
▪ 모터 구동을 위한 결선

1. **IN1, IN2**: **DC MOTER** 결선(**DC MOTOR**방향제어)
2. **OUTA**: **DC MOTER** 결선 (**MOTOR** 출력 핀)
3. **ENA** : **DC MOTER** 결선(속도제어)
4. **+12V** : 외부전원 +5V인가
5. **+5V** : 결선X
6. **GND** : 외부전원과 **BASYS3**을 공통 접지



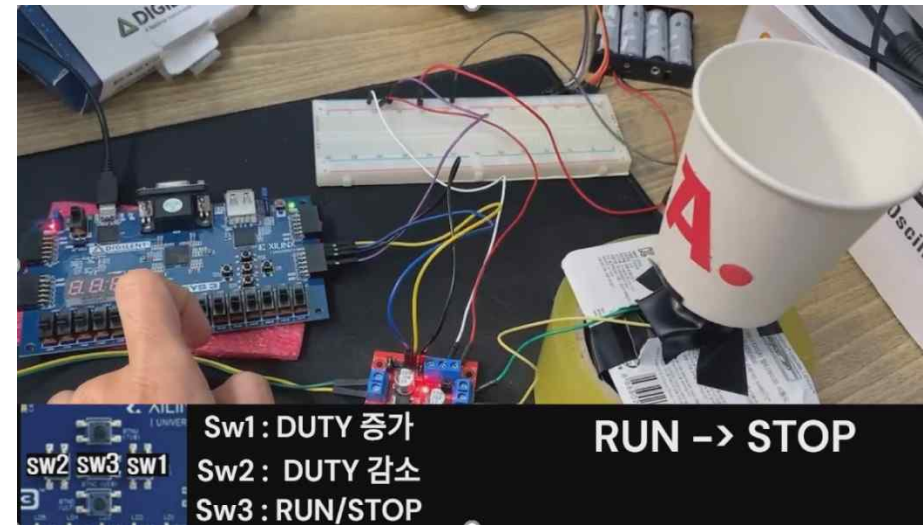
➤ Design Goal

- ***GPIO를 활용한 PWM Control 실습***

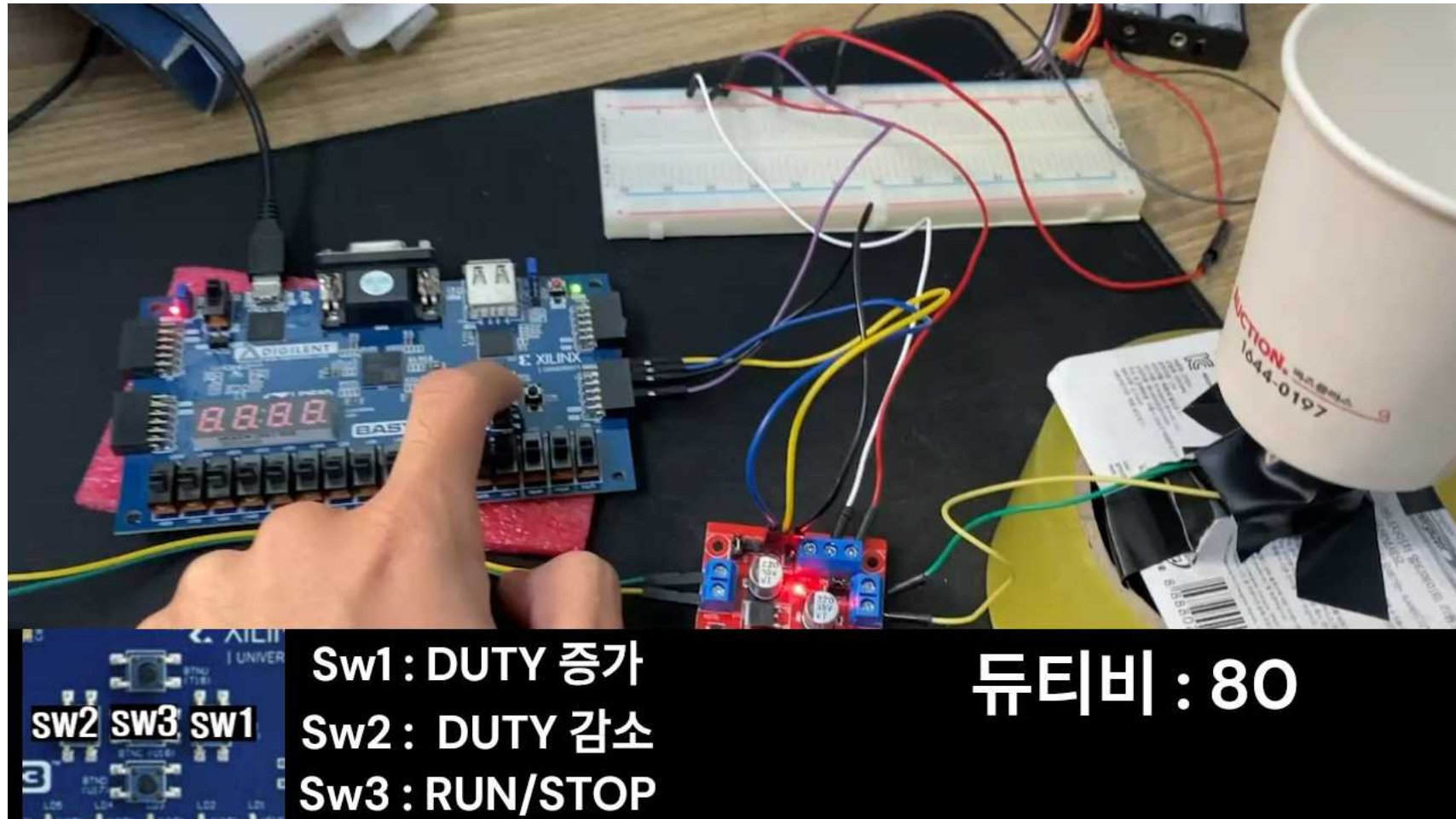


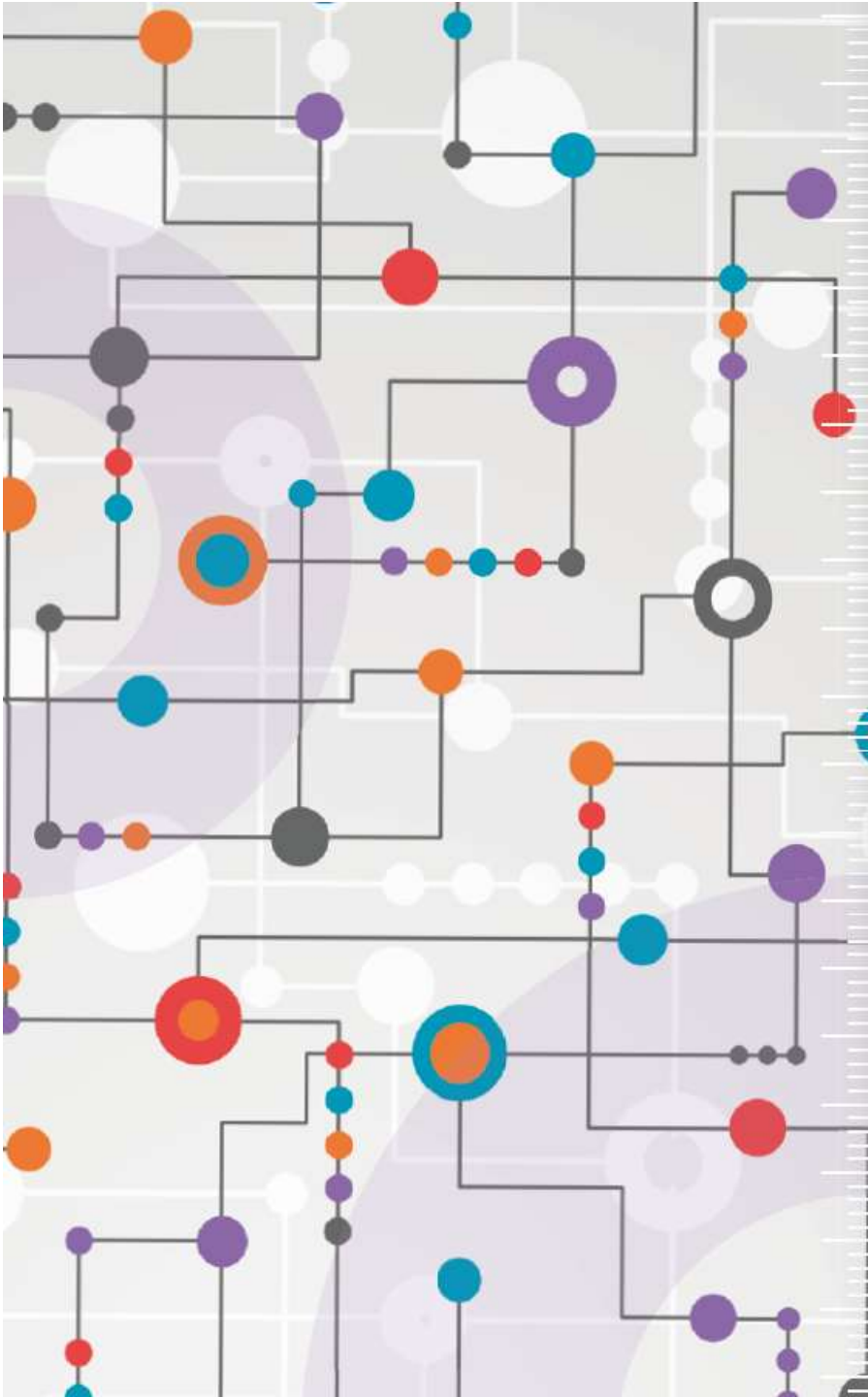
- ***SW0*** : 기본상태에서 0b01, 토글일때 0b10
- ***SW1*** : PWM 듀티비 증가 ***SW2*** : PWM 듀티비 감소 ***SW3*** : RUN/STOP

➤ AXI Connect ➔ PWM Control ➔ Program Device and Implementation Result



➤ AXI Connect ➔ PWM Control ➔ Program Device and Implementation Result





수고하셨습니다.